



Università degli Studi di Napoli
FEDERICO II

A.A. 2025-2026

RETI E PROGRAMMAZIONE DISTRIBUITA

0 NOTE SUL CORSO	3
1 LEZIONE 1: INTRODUZIONE ALLE RETI DI CALCOLATORI	4
Cosa sono le reti di calcolatori	4
Usi delle reti	4
Storia di Internet	4
Componenti della rete	4
Reti e grafi	4
Comunicazione dei dati	5
Velocità di trasmissione, banda e throughput	6
Tipi di connessione	6
Architetture di comunicazione	6
Topologie di rete	6
Bus	6
Internet Service Provider (ISP)	9
Connessione a Internet	10
2 LEZIONE 2: INTERNET, PROTOCOLLI E LIVELLI	11
Le due visioni di Internet	11
Protocolli	11
Dai protocolli umani ai protocolli di rete	11
Chi definisce gli standard	11
Il modello a livelli	12
Connessioni e affidabilità	12
Servizio orientato alla connessione	12
Servizio senza connessione	12
Il modello OSI	12
1. Livello fisico	13
2. Livello data link	13
3. Livello di rete	14
4. Livello di trasporto	14
5. Livello di sessione	15
6. Livello di presentazione	15
7. Livello applicazione	15
Critiche ai modelli	16
Critiche al modello OSI	16
Critiche al modello TCP/IP	16
TCP/IP vs OSI	16
3 LEZIONE 3: IL LIVELLO APPLICAZIONE	17
Applicazioni di rete	17
Architetture delle applicazioni	17
Applicazioni client-server	17
Applicazioni peer-to-peer	17
Comunicazione tra processi	18
Socket	18
Indirizzamento dei processi	18
Qualità del servizio (QoS) dal livello di trasporto	18
Protocolli applicativi	18
Alcuni protocolli applicativi	19

FTP e FTPS	19
Esempio FTP	19
Comandi FTP comuni	19
4 LEZIONE 4: IL LIVELLO APPLICAZIONE: HTTP	21
Web e HTTP	21
URL	21
Esempi	21
HTTP	21
Protocollo di trasporto	22
Assenza di stato (statelessness)	22
Connessioni persistenti e non persistenti	22
Esempio di connessione non persistente	22
RTT (Round-Trip Time)	23
Connessioni persistenti	23
Pipelining	23
Confronto: persistenti vs non persistenti	23
5 LEZIONE 5: FORMATO DEI MESSAGGI HTTP, COOKIE, CACHING	24
Ripasso di HTTP	24
Formato dei messaggi HTTP	24
Formato del messaggio di richiesta	24
Metodi	24
Esempio di richiesta	24
Formato del messaggio di risposta	25
Codici di stato	25
Esempio di risposta	25
Cookie	25
Esempio	26
Usi dei cookie	26
Web caching	26
Funzionamento	26
Hit rate	26
Il problema dell'aggiornamento: conditional GET	27
Caching nel browser	27
Oltre il caching	27
6 LEZIONE 6: E-MAIL, P2P, BITTORRENT	28
Posta elettronica e SMTP	28
Scenario tipico	28
Dettagli della connessione	28
Accesso alla posta	28
POP3	29
IMAP	29
Accesso via HTTP	29
Peer-to-peer	29
BitTorrent	29
Il tracker	29
Il client BitTorrent	30
7 LEZIONE 7: IL DNS	31
Introduzione al DNS	31
Il resolver	31
Lo spazio dei nomi DNS	31
Domini generici di primo livello	31
Nomi di dominio	32
Record di risorsa (resource records)	32
Tipi di record	32

Esempio (dati autorevoli per cs.vu.nl)	33
Name server	33
Risoluzione dei nomi	33
Esempio: nslookup	33
Tipi di query e record	34
Il resolver locale	34
Alias	34
Distribuzione del carico	34
8 LEZIONE 8: PROGRAMMAZIONE DI RETE (SOCKET)	35
Creare applicazioni di rete	35
Scelta dei protocolli	35
Socket	35
Definizioni (strutture in C)	35
Creazione: socket()	35
Associazione: bind()	36
Invio e ricezione (UDP)	36
Chiusura: close()	36
Esempio UDP in C/C++	36
Client	36
Server	37
Da UDP a TCP	37
Connessione lato client: connect()	38
Attesa di connessione lato server: listen() e accept()	38
Invio e ricezione (TCP)	38
Esempio TCP in C/C++	38
Client	38
Server	39
9 LEZIONE 9: PROGRAMMAZIONE DI RETE E DNS	41
Ripasso: socket	41
Comandi socket UDP	41
Comandi socket TCP	41
Traduzione DNS in C/C++	41
La struttura hostent	41
gethostbyname()	42
Esempio completo	42
Esempio: richiesta HTTP via socket	43
Creazione del socket TCP	43
Connessione al server web	43
Richiesta e risposta	43
10 LEZIONE 10: IL LIVELLO DI TRASPORTO	45
Dal livello applicazione al livello di trasporto	45
Responsabilità del livello di trasporto	45
Consegna processo-a-processo	45
Porte	45
Nmap	46
Multiplazione e demultiplazione	46
In UDP (C/C++)	46
In TCP (C/C++)	47
Esempio HTTP	47
UDP	48
Perché usare UDP invece di TCP?	48
Esempio: DNS su UDP	48
Uso di UDP	48
Controversia	48
Formato del segmento UDP	49
Checksum UDP	49

11 LEZIONE 11: TRASPORTO AFFIDABILE DEI DATI	50
Il problema del trasferimento affidabile	50
Corruzione dei pacchetti e Stop-and-wait	50
Problemi di stop-and-wait	50
Perdita di pacchetti e timeout	50
Analogia con l'acqua	51
Prestazioni dello stop-and-wait	51
Pipelining	51
Go-Back-N (GBN)	51
Il ricevente in GBN	51
Selective Repeat (SR)	52
Limite della finestra in SR	52
12 LEZIONE 12: IL LIVELLO DI TRASPORTO --- TCP	53
TCP rispetto a UDP	53
Buffer TCP	53
Maximum Segment Size (MSS)	53
Il segmento TCP	53
Numeri di sequenza e acknowledgment	54
Esempio	54
Dati in entrambe le direzioni	54
Esempio semplificato (echo)	54
Retransmission Timeout	54
Stima dell'RTT	54
Timeout di ritrasmissione	55
Fast Retransmit	55
Esempio	55
13 LEZIONE 13: TCP --- GESTIONE DELLA CONNESSIONE	56
Richiamo: responsabilità del livello di trasporto	56
Richiamo: il segmento TCP	56
Problemi di gestione della connessione TCP	56
Il problema dei due eserciti (two-army problem)	56
Instaurazione della connessione TCP	57
Dettagli del three-way handshake	57
Rilascio della connessione TCP	57
Gestione delle perdite nel rilascio	57
14 LEZIONE 14: TCP --- CONTROLLO DI FLUSSO E DI CONGESTIONE	58
Il problema della congestione e del controllo di flusso	58
Controllo di flusso	58
Meccanica	58
Il problema della congestione	58
Vista semplificata	58
Controllo di congestione	59
Il controllo di congestione in TCP	59
Regolazione della velocità	59
Rilevamento della congestione	59
Adeguamento della velocità	59
15 LEZIONE 15: IL LIVELLO DI RETE	60
Dal trasporto alla rete	60
Il livello di rete in Internet	60
I router	60
Tipi di router	60
Piano dati e piano di controllo	60
Componenti principali di un router	61
Forwarding	61

Switching fabric	61
Code sulle porte	62
Scenario worst-case	62
Scheduling dei pacchetti	62
FIFO	62
Priority queuing	62
Round-robin	62
16 LEZIONE 17: LIVELLO DI RETE --- GESTIONE DEGLI INDIRIZZI	63
Ripasso	63
Internet Protocol (IP)	63
Indirizzi IPv4	63
Subnetting	63
Subnet mask	63
Stessa subnet o no?	64
Router e subnet	64
Esempi	64
ifconfig	65
Indirizzamento in Internet	65
Classful addressing	65
Classless addressing (CIDR)	65
Aggregazione degli indirizzi	65
Ottenere un blocco	65
Il datagram IPv4	66
Frammentazione IPv4	66
17 LEZIONE 18: LIVELLO DI RETE --- DHCP, LAN, IPV6	67
Assegnazione degli IP: DHCP	67
Funzionamento	67
Comandi Linux	67
Visibilità degli IP e NAT	67
Network Address Translation (NAT)	67
Ping	68
Nmap (parte 2)	68
Indirizzi IP riservati	68
Esempio: setup di una LAN	68
Configurazione lato WAN	68
Configurazione lato LAN	69
Configurazione di un host	69
IPv6	69
Il datagram IPv6	69
Cosa manca rispetto a IPv4	69
Da IPv4 a IPv6	70
18 LEZIONE 20: LIVELLO DI RETE --- PIANO DI CONTROLLO E ROUTING	71
Introduzione al routing	71
Flooding	71
Formalizzazione del problema	71
Algoritmo Distance-Vector (DV)	72
L'equazione di Bellman	72
Esempio	72
Pseudocodice	72
Esempio sul cammino u-z	72
Complessità, pro e contro	73
Il problema del count-to-infinity	73
Esempio semplificato	73
Come evitarlo	73

19 LEZIONE 21: LIVELLO DI RETE --- ALGORITMO LINK-STATE	74
L'algoritmo Link-State (LS)	74
La fase di scambio dei messaggi	74
Pseudocodice	74
Esempio (6 nodi, algoritmo invocato su u)	74
Complessità, pro e contro	75
DV vs. LS	75
20 LEZIONE 22: LIVELLO DI COLLEGAMENTO --- FRAMING E GESTIONE DEGLI ERRORI	76
Dal livello di rete ai livelli di collegamento e fisico	76
Servizi del livello di collegamento	76
Implementazione	76
Software	77
Hardware	77
Tecnologie fisiche	77
Frame (richiamo)	77
Metodi di framing	77
Conteggio di byte (mai usato)	77
Byte stuffing	77
Bit stuffing	78
Violazione fisica	78
Rilevamento e correzione degli errori	78
Formulazione del problema	78
Parity check	78
Parity check bidimensionale	79
Cyclic Redundancy Check (CRC)	79
Distanza di Hamming	79
21 LEZIONE 23: LIVELLO DI COLLEGAMENTO --- SOTTO-LIVELLO MAC	80
Richiamo: servizi del livello di collegamento	80
Tipi di collegamento	80
Collisioni	80
Protocolli di accesso multiplo	80
Ruolo e approcci	80
Desiderata	81
Protocolli a partizionamento del canale	81
TDMA (Time Division Multiple Access)	81
FDMA (Frequency Division Multiple Access)	81
CDMA (Code Division Multiple Access)	81
Protocolli ad accesso casuale	81
Slotted ALOHA	81
CSMA e CSMA/CD	82
22 LEZIONE 24: SOTTO-LIVELLO MAC, ACCESSO CASUALE, PROTOCOLLI SENZA COLLISIONI, SWITCH E ARP	83
Protocolli ad accesso casuale: CSMA	83
Perché le collisioni avvengono nonostante il carrier sensing?	83
CSMA puro	83
CSMA/CD (implementato attualmente su Ethernet)	83
Protocolli a turni (taking-turns)	83
Protocollo di polling	83
Protocollo token-passing	83
Indirizzi MAC	83
Comunicazione in una LAN	84
Switch	84
LAN con switch	84
ARP (Address Resolution Protocol)	84
Esempio: da C verso A	84

ARP e gateway (IP non locale)	85
Frame Ethernet	85

23 LEZIONE 25: SICUREZZA DI RETE --- FONDAMENTI 86

Richiamo: alcuni protocolli applicativi	86
Introduzione	86
Attacchi: malware	86
Attacchi: DoS	87
Attacchi: packet sniffing	87
Attacchi: IP spoofing	87
Fondamenti della sicurezza di rete	87
Crittografia	87
Chiavi	88
Crittografia simmetrica	88
Crittografia asimmetrica	88
Certification Authority (CA)	88
Integrità dei messaggi	88
Hash semplice (approccio fallace)	88
Message Authentication Code (MAC)	89

24 LEZIONE 26: SICUREZZA DI RETE --- AUTENTICAZIONE, SSL, FIREWALL 90

Richiamo: fondamenti della sicurezza di rete	90
Autenticazione degli endpoint	90
Autenticazione tramite indirizzo IP	90
Autenticazione tramite password	90
Attacco playback (replay)	90
Nonce	91
SSL (Secure Socket Layer)	91
Fase 1: Handshake	91
Fase 2: Key Derivation	91
Fase 3: Trasferimento dati	91
Firewall	92
Filtraggio tradizionale dei pacchetti	92
Filtraggio stateful	92
Application Gateway	93
Intrusion Detection Systems (IDS)	93
Demilitarized Zone (DMZ)	93

Note sul corso

I seguenti appunti sono stati presi durante le lezioni del corso di Tecnologie Web.

Nota:

Lezione 1: Introduzione alle reti di calcolatori

Cosa sono le reti di calcolatori

Il networking è il processo di collegare tra loro calcolatori affinché possano condividere informazioni. Una **rete di calcolatori** è un insieme di dispositivi di calcolo interconnessi e autonomi: un numero elevato di calcolatori separati ma connessi svolge un lavoro comune e può scambiarsi informazioni.

Internet è la rete di calcolatori più importante al mondo e, anzi, è una **rete di reti**: tecnicamente si tratta di un'enorme *Wide Area Network* (WAN) che connette sotto-reti e dispositivi tra nazioni e continenti diversi. Comprende centinaia di milioni di dispositivi di rete, collegamenti e calcolatori che offrono centinaia di servizi agli utenti; esistono comunque anche reti più piccole, locali o disconnesse da Internet.

Usi delle reti

Le reti servono per accedere alle informazioni, per la comunicazione da persona a persona, per il commercio elettronico, per l'intrattenimento e per l'**Internet delle cose** (IoT).

Storia di Internet

L'idea di una rete "universale" fu teorizzata dai ricercatori (in particolare J.C.R. Licklider) intorno al 1950 e iniziò a diventare realtà quasi dieci anni dopo. Le tappe fondamentali sono:

- L'**Advanced Research Projects Agency** (ARPA) del Dipartimento della Difesa statunitense assegnò contratti per lo sviluppo del progetto **ARPANET** (1969).
- R. Kahn (ARPA) e V. Cerf (Stanford) progettaron il Transmission Control Protocol (TCP) e l'Internet Protocol (IP), due protocolli della **suite di protocolli Internet** (1974).
- La National Science Foundation (NSF) finanziò il progetto **NSFNET**, una rete basata su TCP/IP (1986).
- **ARPANET fu dismesso** nel 1990; **NSFNET fu dismesso** nel 1995, rimuovendo le ultime restrizioni all'uso di Internet per il traffico commerciale.

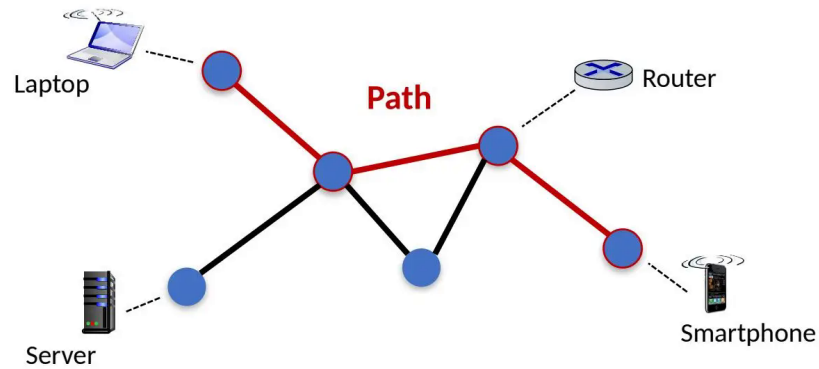
L'ARPANET iniziale aveva quattro nodi (SRI, UCLA, UCSB e UTAH, 1969) e nasceva come rete chiusa, pensata per università e centri di ricerca; reti simili furono create indipendentemente in tutta Europa e negli USA. NSFNET e il protocollo TCP/IP realizzarono l'idea di *rete di reti*, ma sia ARPANET che NSFNET presentavano **restrizioni commerciali**.

Componenti della rete

I dispositivi di rete e i collegamenti (*link*) costituiscono l'infrastruttura che permette agli host di connettersi: router, access point, switch e hub. Gli **host** sono invece i dispositivi su cui girano le applicazioni, come laptop, server, smartphone e PC.

Reti e grafi

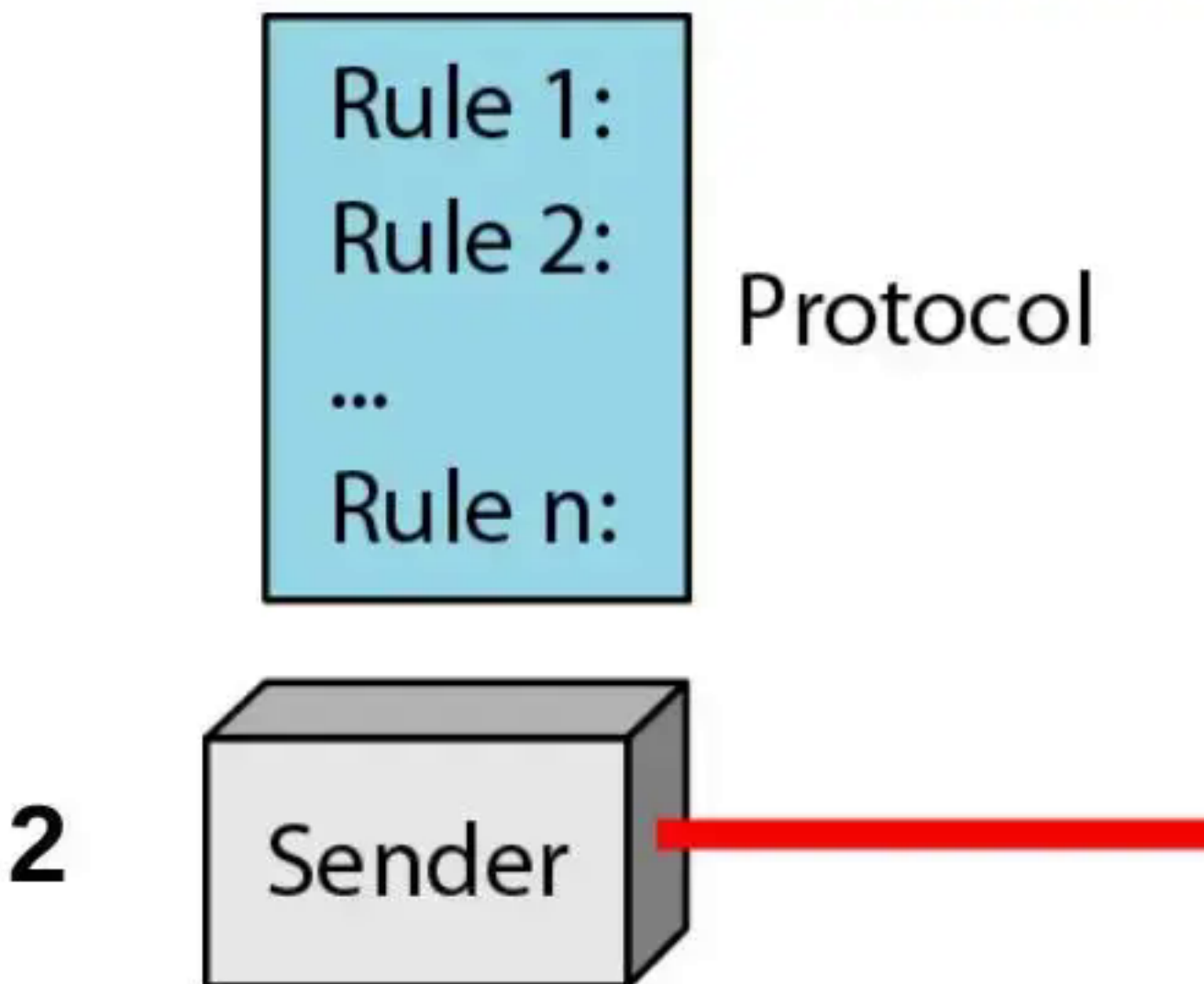
Le reti di calcolatori condividono la terminologia con la teoria dei grafi. I dispositivi connessi tramite la rete sono chiamati **nodi**, mentre le connessioni tra nodi sono chiamate **collegamenti** (o **canali**) di comunicazione. Una sequenza di nodi e collegamenti forma un **cammino** (*path*). I punti terminali della rete, che forniscono o usano **servizi**, sono nodi speciali chiamati **host**: idealmente sono le foglie della rete (anche se spesso non è vero), mentre i nodi intermedi sono dispositivi di routing come router e switch.



Comunicazione dei dati

La comunicazione dati coinvolge cinque componenti principali:

- Il **messaggio**, che contiene i dati da trasmettere.
- Il **mittente**, l'entità che invia il messaggio.
- Il **destinatario**, l'entità che dovrebbe riceverlo.
- Il **mezzo**, il canale tra mittente e destinatario in cui viaggia il messaggio.
- Il **protocollo**, un insieme di regole di comunicazione note a mittente e destinatario.



I dati da scambiare possono essere rappresentati in forme diverse (testo, numeri, immagini, audio, video). A seconda del tipo e dello scopo della comunicazione, il **flusso dei dati** può essere **simplex** (monodirezionale), **half-duplex** (bidirezionale a turni) o **full-duplex** (bidirezionale simultaneo).

Velocità di trasmissione, banda e throughput

Questi tre termini si distinguono per ciò che misurano.

- La **velocità di trasmissione** è la quantità **massima** di informazione che un **dispositivo** può trasmettere, misurata in bit/s (o byte/s).
- La **banda** (*bandwidth*) è la quantità **massima** di informazione che un **cammino** (link + nodi) può trasmettere, sempre in bit/s.
- Il **throughput**, infine, è la quantità **effettiva** (istantanea) di informazione che viene realmente trasmessa su un cammino.

Tipi di connessione

Gli host di una rete possono essere connessi in modi diversi: **punto-punto**, quando viene fornito un collegamento dedicato tra due dispositivi (wireless o cablato), oppure **multipunto** (broadcast), quando più di due dispositivi condividono un singolo collegamento.

Architetture di comunicazione

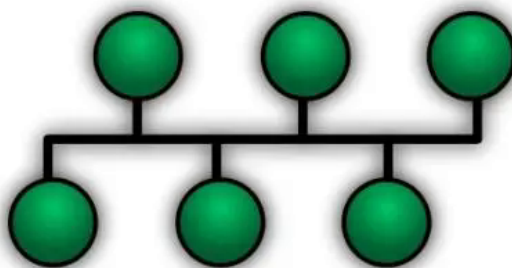
Nel modello **client-server**, un client richiede esplicitamente informazioni a un server che le ospita: il processo client invia un messaggio attraverso la rete al processo server e attende un messaggio di risposta. In un sistema **peer-to-peer**, invece, non esistono client e server fissi.

Topologie di rete

La **topologia di rete** è la disposizione degli elementi (link, nodi, ecc.) di una rete di comunicazione. Le topologie principali sono:

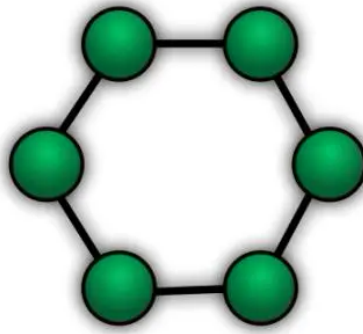
Bus

Nella topologia **bus** gli host sono connessi a un cavo dorsale centrale, e quindi sono possibili **collisioni**. È una soluzione semplice ed economica, adatta a reti piccole; il difetto principale è che il bus è un singolo punto di guasto, anche se le sotto-reti possono rimanere disponibili. Una rete bus ha un collegamento fisico duplex (una dorsale più n link).



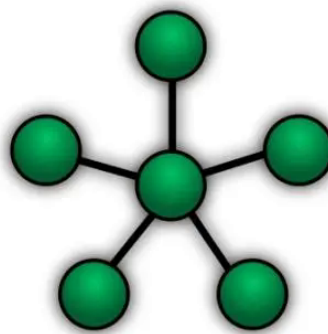
Ring

Nella topologia **ring** ogni host è connesso punto-punto esattamente ad altri due, e il segnale viene **inoltrato** lungo l'anello, da dispositivo a dispositivo, fino a raggiungere la destinazione. È semplice ed economica, con prestazioni migliori del bus; tuttavia aggiungere nuovi nodi è più difficile e i nodi guasti possono compromettere la rete. Una rete ring ha n collegamenti duplex.



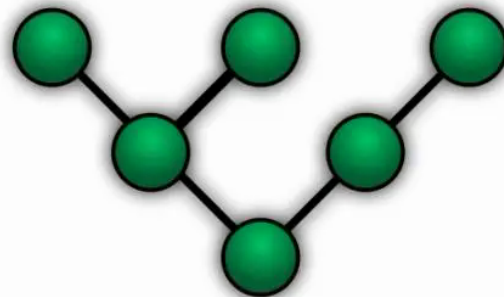
Star

Nella topologia **star** gli host sono collegati a un controllore centrale (hub, switch o router), senza collegamenti diretti tra host: è il controllore a reindirizzare i messaggi. È meno costosa, semplice, robusta e più scalabile delle precedenti, ma il controllore deve essere raggiungibile da tutti gli host e rappresenta un singolo punto di guasto. Una rete star ha un controllore e n collegamenti fisici duplex.



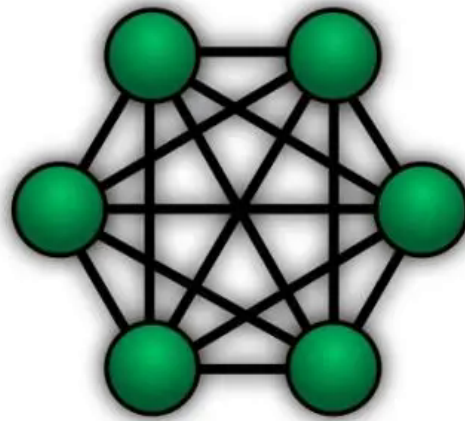
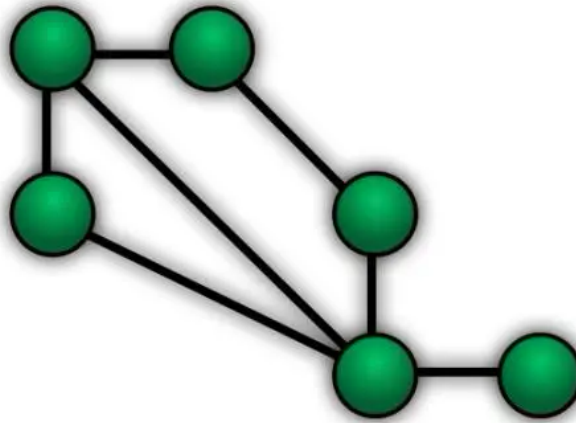
Tree

La topologia **tree** integra più topologie star, tipicamente tramite un cavo bus. È versatile, scalabile e robusta, ed è ben supportata dai fornitori di hardware e software; in compenso è difficile da configurare ed eredita la debolezza del bus.



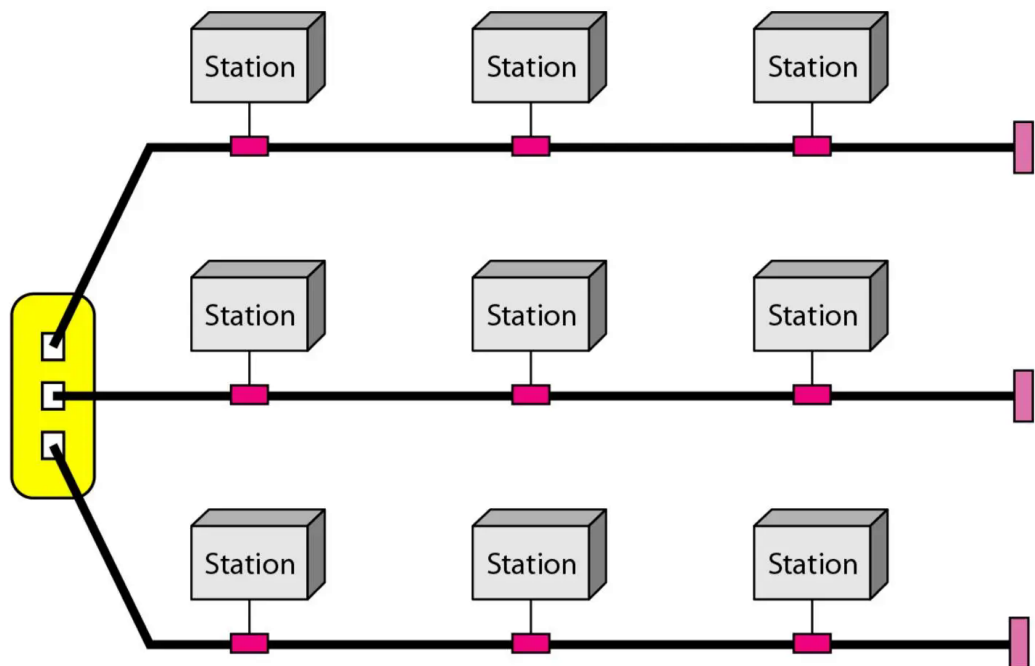
Mesh

Nella topologia **mesh** gli host sono collegati punto-punto in modo non gerarchico. Si distingue tra **full mesh**, in cui tutti i nodi sono pienamente connessi, e **partial mesh**, in cui i nodi sono connessi solo ad alcuni altri. Offre traffico ridotto, robustezza, sicurezza e collegamenti dedicati, ma è difficilmente scalabile e costosa, perché richiede dispositivi con più porte. Una rete full mesh ha $n(n-1)/2$ collegamenti fisici duplex.



Ibrida

Le topologie possono essere **combinare** in una rete ibrida, per esempio una dorsale star con tre reti bus.



Categorie di reti

Le reti si classificano per dimensione, numero di host e banda in **PAN** (Personal Area Network), **LAN** (Local Area Network) o **WLAN** (Wireless LAN), **MAN** (Metropolitan Area Network) e **WAN** (Wide Area Network).

PAN

Le PAN permettono ai dispositivi di comunicare entro il raggio di una persona. **Bluetooth** è una rete wireless a corto raggio usata per connettere componenti senza cavi.

LAN e rete casalinga

Una LAN domestica connette una gamma ampia e diversificata di dispositivi a Internet, e deve essere gestibile, affidabile e sicura; l'IoT permette di connettere quasi qualsiasi dispositivo. Le proprietà richieste a una rete domestica sono: facile da installare, sicura e affidabile, con interfacce compatibili tra tutti i prodotti e costi ridotti dei dispositivi.

MAN

In una MAN sia i segnali televisivi che Internet vengono immessi nel centralino via cavo (*cable head-end*, o *cable modem termination system*) per la successiva distribuzione alle abitazioni.

WAN

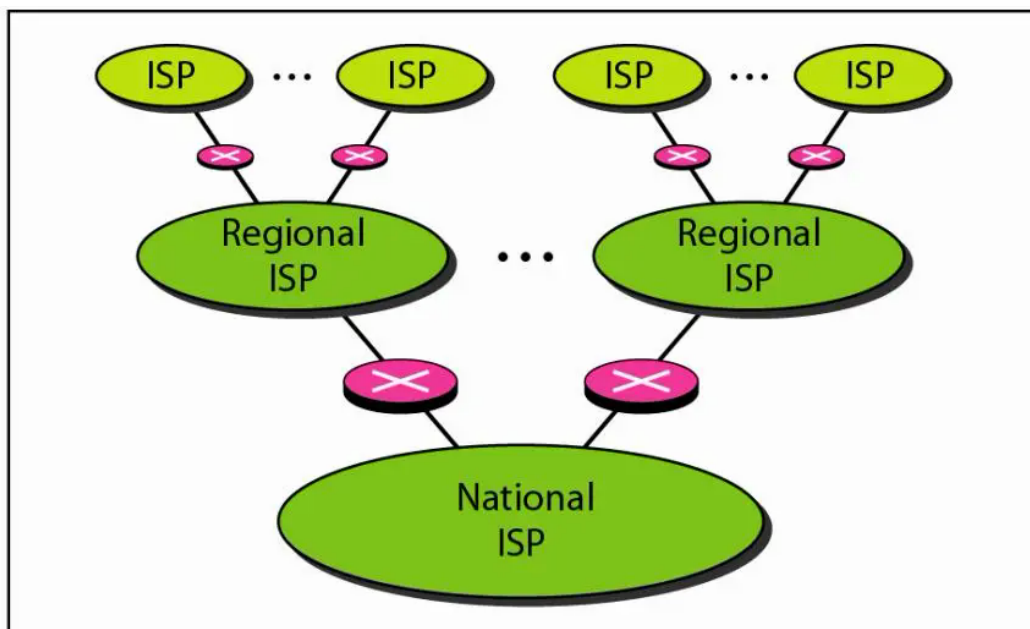
Le WAN mostrano come host in città diverse possano comunicare tramite linee dedicate (*leased line*), via Internet o tramite un ISP. La complessità della topologia cresce con le dimensioni della rete: le WAN che connettono nazioni o continenti possono essere molto complesse ed eterogenee.

Internet Service Provider (ISP)

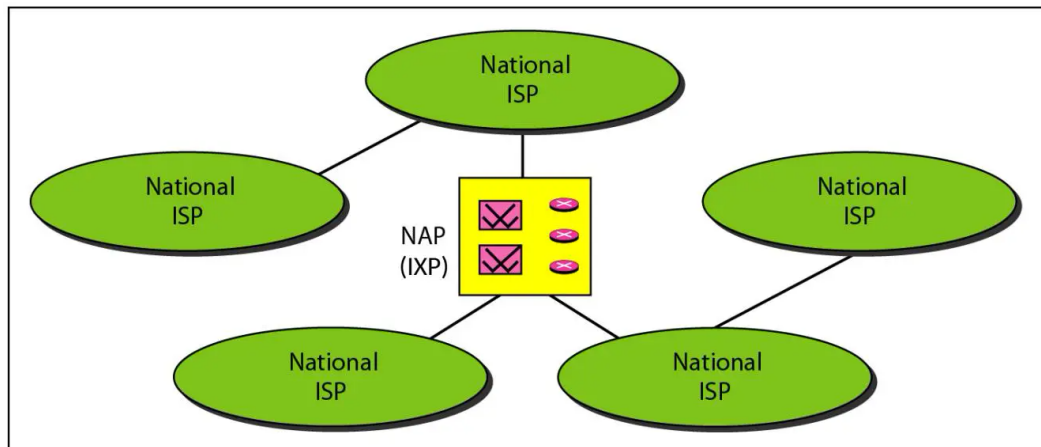
L'accesso a Internet è il servizio base. Un **Internet Service Provider (ISP)** è un'organizzazione che fornisce servizi per accedere, utilizzare o partecipare a Internet; può essere commerciale, comunitario, no-profit o privato (per esempio un'azienda o un'università). Diversi ISP si scambiano dati tramite Network Access Point (NAP) neutri o Internet Exchange Point (IXP).

La rete di un ISP è definita gerarchicamente:

- Un punto di presenza (**PoP**) è un gruppo di uno o più router usati dall'ISP per raggiungere i clienti.
- Gli **access ISP** coprono aree locali.
- I **regional ISP** coprono aree più ampie.
- I **national ISP** coprono intere nazioni.
- Un **Internet Exchange Point (IXP)** funge da punto d'incontro tra più ISP, spesso non gestito dagli ISP coinvolti.



Structure of a national ISP



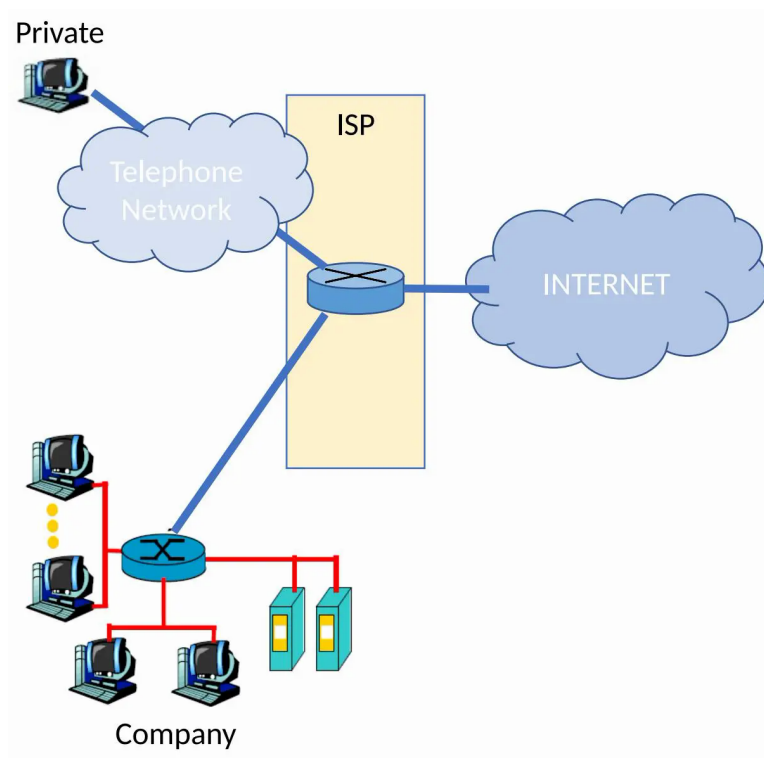
Interconnection of national ISPs

Connessione a Internet

Le connessioni private a Internet si stabiliscono tramite modem o ONT collegato alla rete telefonica. Le tecnologie, in ordine cronologico e di capacità, sono:

- Analogica (56 kbps).
- ISDN (Integrated Services Digital Network, 128 kbps).
- ADSL (Asymmetric Digital Subscriber Line, da 1 a 20 Mbps).
- Doppino in rame (da 10 a 100 Mbps).
- Fibra ottica (da 50 Mbps a 40 Gbps), tramite ottico di rete (ONT), senza (de)modulazione.

Le aziende, soprattutto medie e grandi, possono invece avere un collegamento diretto o dedicato all'ISP.



Lezione 2: Internet, protocolli e livelli

Le due visioni di Internet

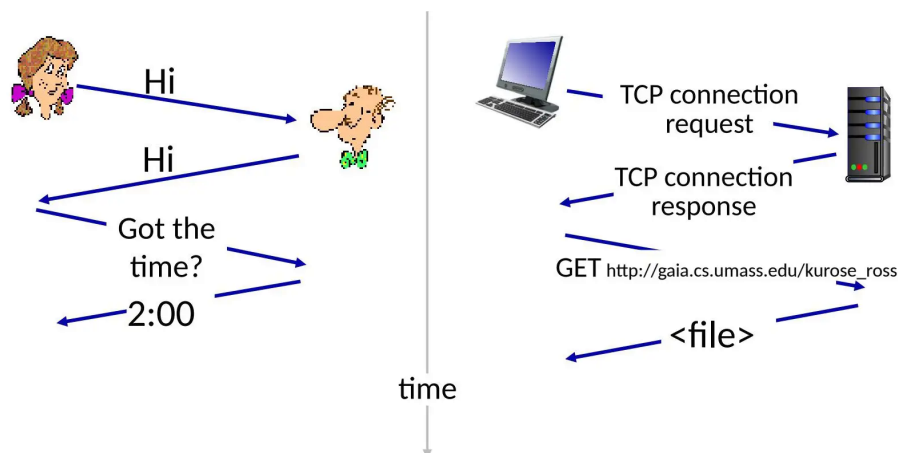
Internet può essere descritta in due modi complementari. Nella visione “concreta” (*nuts and bolts*) essa è costituita da miliardi di **dispositivi di calcolo connessi**: gli **host** (o sistemi finali, *end system*) eseguono applicazioni di rete al bordo (*edge*) di Internet, mentre i **commutatori di pacchetti** (*packet switch*), come router e switch, inoltrano blocchi di dati chiamati pacchetti. I dispositivi sono collegati da **collegamenti di comunicazione** di vario mezzo (fibra, rame, radio, satellite), ciascuno caratterizzato dalla propria **banda** (velocità di trasmissione). Un’ultima componente è l’insieme delle **reti**: insiemi di dispositivi, router e link gestiti da un’organizzazione. Tra i dispositivi oggi connessi si va ben oltre i PC: assistenti vocali, elettrodomestici, telecamere, pacemaker, dispositivi indossabili e tostapane connessi.

Nella visione “dei servizi”, invece, Internet è un’**infrastruttura che fornisce servizi alle applicazioni**: Web, video in streaming, teleconferenze, e-mail, giochi, e-commerce e social media. Fornisce inoltre un’**interfaccia di programmazione** alle applicazioni distribuite, ossia degli “agganci” (*hook*) che permettono alle applicazioni che inviano e ricevono di connettersi a Internet e usare il servizio di trasporto, con opzioni di servizio analoghe a quelle del servizio postale.

Protocolli

Dai protocolli umani ai protocolli di rete

I protocolli umani, come chiedere l’ora o presentarsi, consistono nell’inviare messaggi specifici e nel compiere azioni specifiche alla ricezione di un messaggio o in risposta ad altri eventi. I **protocolli di rete** seguono lo stesso schema, ma coinvolgono calcolatori anziché esseri umani: tutta l’attività di comunicazione in Internet è governata da protocolli. Un protocollo definisce il **formato e l’ordine dei messaggi** scambiati tra le entità di rete, e le **azioni compiute** alla trasmissione e alla ricezione dei messaggi.



Jakobson Communication model

Chi definisce gli standard

Gli standard e i protocolli di Internet sono definiti da una comunità di esperti, la **Internet Engineering Task Force** (IETF), organizzata in gruppi di lavoro aperti, ciascuno dedicato ad aspetti specifici di Internet. Ogni gruppo produce documenti numerati chiamati **Request For Comments** (RFC), che descrivono e definiscono protocolli, concetti e metodi alla base di uno standard: ad esempio il protocollo IPv4 è stato definito nell’RFC 791 (1981).

Il modello a livelli

Le reti sono sistemi complessi: host, router, link di vario mezzo, applicazioni, protocolli, hardware e software. Per dominare questa complessità si ricorre a un approccio architetturale chiamato **modello a livelli**, una strategia *divide et impera* in cui ogni livello costituisce un livello di astrazione: ogni livello si appoggia sui servizi del livello inferiore e fornisce servizi al livello superiore, attraverso interfacce ben definite, dal livello alto (le applicazioni) verso quello basso (l'hardware).

Il layering è la chiave per gestire sistemi complessi. Una struttura esplicita permette di identificare le componenti del sistema e le loro relazioni, fornendo così un modello di riferimento per la discussione; la **modularizzazione**, inoltre, facilita manutenzione e aggiornamento, poiché un cambiamento nell'*implementazione* del servizio di un livello è trasparente al resto del sistema. Va notato che il layering può avere un costo in termini di efficienza, e che strutture a livelli esistono anche in molti altri sistemi complessi. Le entità usano i protocolli per implementare le proprie definizioni di servizio; quando due dispositivi comunicano, ogni livello del mittente comunica con lo stesso livello del destinatario tramite un protocollo specifico.

Connessioni e affidabilità

Servizio orientato alla connessione

Il servizio orientato alla connessione è modellato sul sistema telefonico: l'utente stabilisce prima una connessione, la usa, poi la rilascia, e durante l'instaurazione può condurre una negoziazione sui parametri da usare.

Servizio senza connessione

Il servizio senza connessione è modellato sul sistema postale, dove il pacchetto è un messaggio al livello di rete. Lo scambio può avvenire in due modi: con **store-and-forward switching** i nodi intermedi ricevono il messaggio per intero prima di inoltrarlo al nodo successivo, mentre con **cut-through switching** la trasmissione da un nodo inizia prima che il nodo abbia ricevuto completamente il messaggio. Il **servizio datagram** è un servizio senza connessione non affidabile, cioè privo di *acknowledgement*. L'affidabilità, in generale, caratterizza sia i servizi orientati alla connessione sia quelli senza connessione.

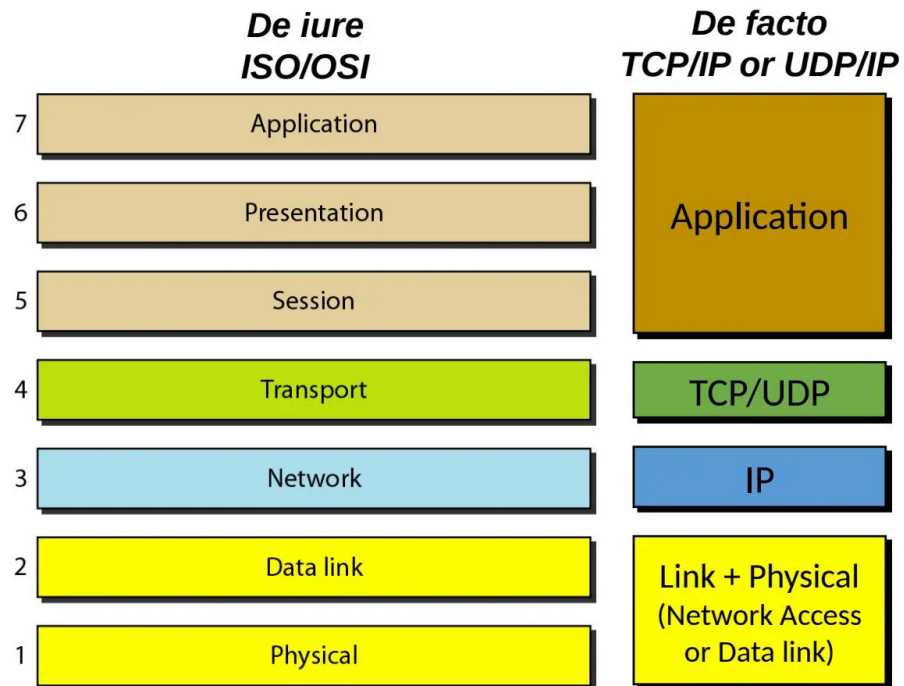
Sei servizi comuni

Tipo	Servizio	Esempio
Orientato alla connessione	Flusso di messaggi affidabile	Sequenza di pagine
Orientato alla connessione	Flusso di byte affidabile	Download di un film
Orientato alla connessione	Connessione non affidabile	Voice over IP
Senza connessione	Datagram non affidabile	Posta elettronica spazzatura
Senza connessione	Datagram confermato	Messaggistica testuale
Senza connessione	Richiesta-risposta	Query su database

Il modello OSI

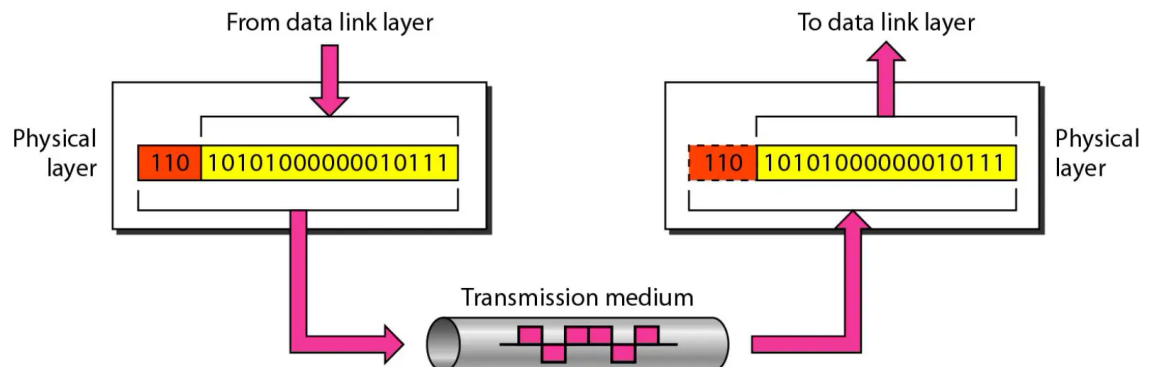
Negli anni '80 la ISO (*International Standards Organization*) definì un modello a livelli per le reti di calcolatori: il modello **OSI** (*Open System Interconnection*), composto da 7 livelli, che vanno dalla trasmissione fisica dei bit "grezzi" (livello 1) fino all'uso dei dati e all'interazione uomo-macchina (livello 7). I livelli intermedi si occupano, nell'ordine, del formato dei dati (frame), dell'instradamento dei messaggi, dei protocolli di trasmissione, della gestione delle porte e delle sessioni, e della traduzione dei messaggi (codifica, compressione, decifrazione).

Non sempre tutti i livelli sono implementati: tipicamente nei nodi intermedi (switch, router) sono implementati solo 2 o 3 livelli (i livelli dei mezzi), mentre solo gli endpoint implementano l'intero stack fino ai livelli applicativi. La comunicazione tra livelli segue due meccanismi speculari: con l'**incapsulamento** (*top-down*) ogni livello del mittente aggiunge al messaggio (payload) un campo proprio, sotto forma di header (H) o trailer (T); con il **decapsulamento** (*bottom-up*) quei campi sono rimossi e interpretati dallo stesso livello nel destinatario.



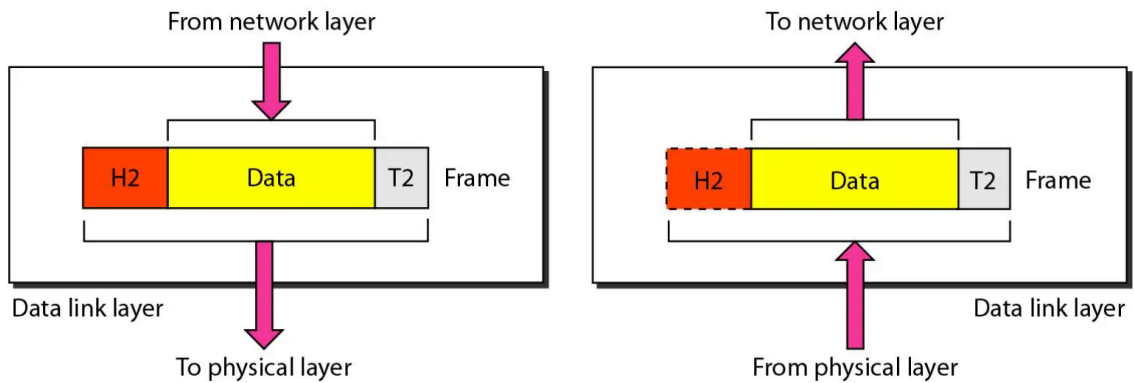
1. Livello fisico

Il **livello fisico** è responsabile del movimento dei singoli bit da un hop (nodo) al successivo. Regola le caratteristiche fisiche di interfacce e mezzi (connettori, cavi, segnali elettrici), la configurazione della linea (punto-punto o multipunto), la topologia fisica (mesh, star, ring o bus), la modalità di trasmissione (simplex, half-duplex o duplex), la rappresentazione dei bit, la velocità di trasmissione (*data rate*) e la sincronizzazione dei bit.



2. Livello data link

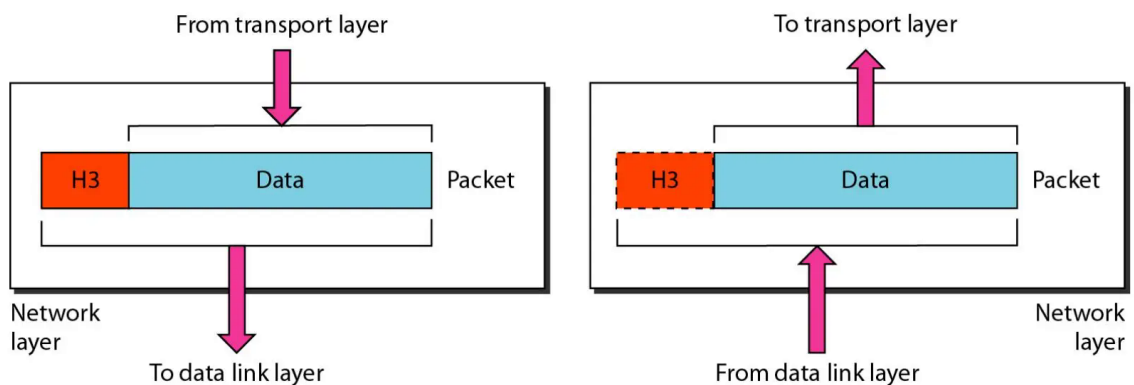
Il **livello data link** è responsabile del movimento dei frame da un hop al successivo (segmento). Lavora su frame, porzioni di dati tipicamente di poche centinaia di byte, e si occupa di controllo di flusso ed errori (tramite sequenze di controllo del frame) e di controllo di accesso.



]

3. Livello di rete

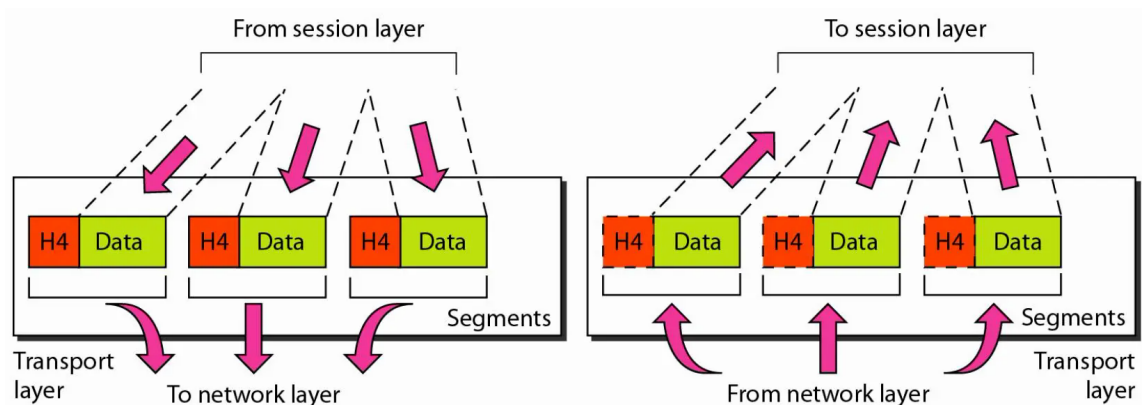
Il **livello di rete** è responsabile della consegna dei singoli pacchetti dall'host sorgente all'host destinazione (cammino). Lavora su pacchetti, tipicamente più grandi e complessi dei frame, e gestisce la consegna da sorgente a destinazione, l'indirizzamento logico e il routing tramite tabelle di routing.



]

4. Livello di trasporto

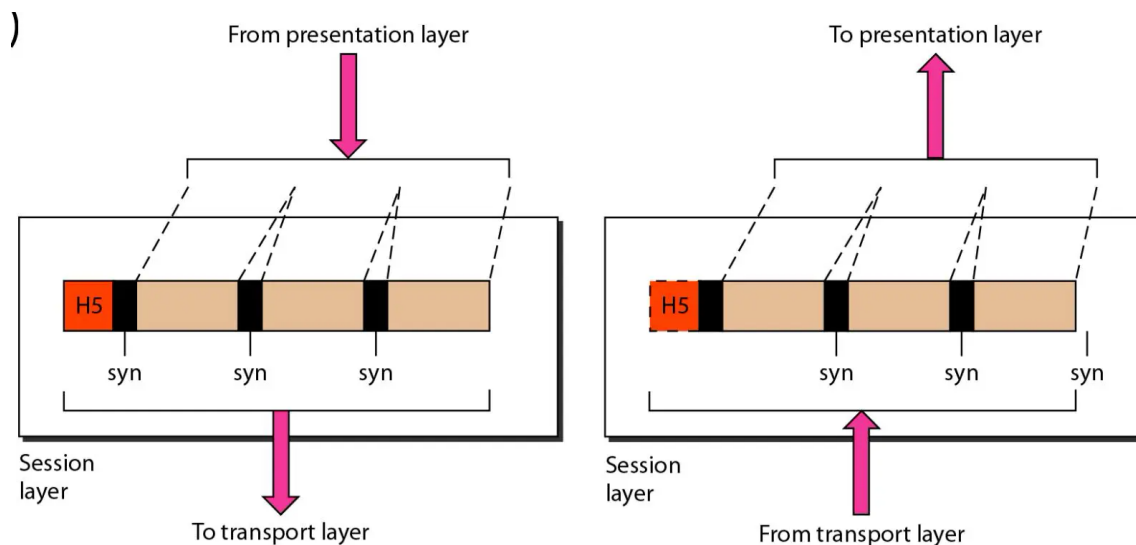
Il **livello di trasporto** è responsabile della consegna di un messaggio da un estremo all'altro, così che il messaggio arrivi al programma giusto. Si occupa di consegna end-to-end, controllo della connessione (orientato o senza connessione), segmentazione/riassemblaggio da e verso i pacchetti del livello 3, indirizzamento delle porte e controllo di flusso ed errori.



]

5. Livello di sessione

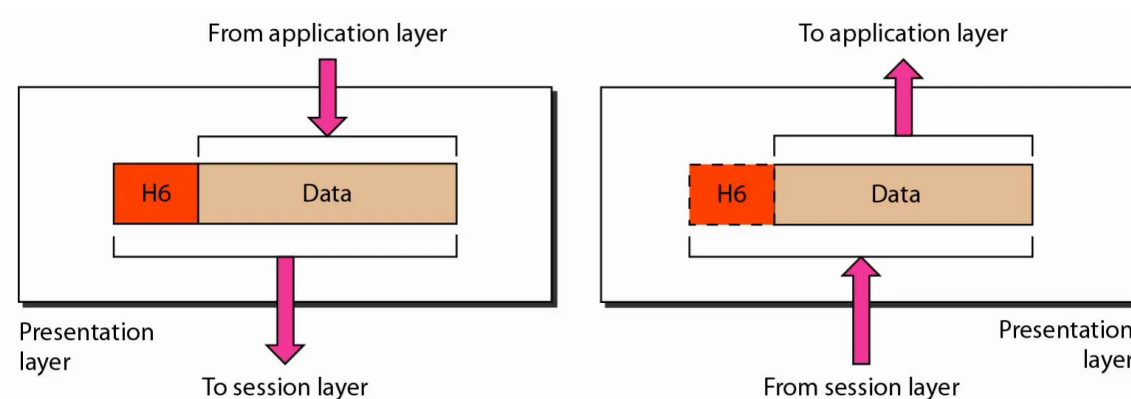
Il **livello di sessione** è responsabile del controllo del dialogo e della sincronizzazione richiesta/risposta: stabilisce, mantiene e sincronizza l'interazione tra i sistemi che comunicano (sessione di comunicazione), gestisce il dialogo e prevede punti di controllo (*checkpoint*) per la sincronizzazione.



]

6. Livello di presentazione

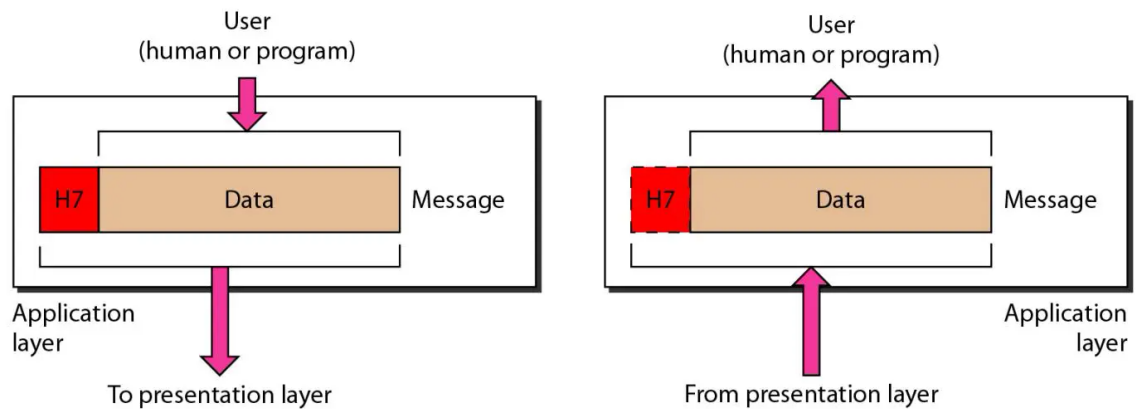
Il **livello di presentazione** è responsabile della rappresentazione dei dati: si occupa di traduzione (ad esempio da codifica EBCDIC o ASCII a testo), cifratura e decifratura, e compressione.



]

7. Livello applicazione

Il **livello applicazione** è responsabile di fornire servizi all'utente: terminale virtuale di rete (accesso remoto), trasferimento e accesso a file, servizi di posta e accesso al World Wide Web.



]

Critiche ai modelli

Critiche al modello OSI

Il modello OSI fu criticato su più fronti: il **tempismo** era sbagliato, poiché i protocolli concorrenti TCP/IP erano già in uso diffuso; il **progetto** era scadente, con difetti sia nel modello sia nei protocolli; le prime **implementazioni** erano enormi, ingombranti e lente; e la **politica** lo vedeva diffusamente come creatura dei ministeri europei delle telecomunicazioni e dei governi. Per evitare il “tempismo sbagliato”, è essenziale che gli standard vengano scritti nell’incavo tra le creste delle due “onde degli elefanti”.

Critiche al modello TCP/IP

Anche il modello TCP/IP ha difetti: non distingue chiaramente i concetti di **servizi, interfacce e protocolli**; non è affatto generale, quindi è poco adatto a descrivere qualunque altro stack di protocolli; il **livello link** non è davvero un livello nel senso normale del termine; e il modello non distingue tra livello fisico e data link. A ciò si aggiunge il fatto che altre implementazioni di protocolli erano distribuite gratuitamente.

TCP/IP vs OSI

Il modello ISO/OSI è lo stack standard *de iure* per le reti di calcolatori, ma è piuttosto complesso e dettagliato. TCP/IP e UDP/IP (detti *Internet Protocol Suite*) sono invece l’insieme di protocolli di comunicazione effettivamente usati su Internet e nelle reti locali, TCP (*Transmission Control Protocol*), UDP (*User Datagram Protocol*) e IP (*Internet Protocol*), e costituiscono quindi lo stack standard *de facto* per la comunicazione su Internet. Esistono infine diversi altri protocolli per la comunicazione tra dispositivi, usabili a seconda della situazione, che si basano comunque sul modello a livelli.

Lezione 3: Il livello applicazione

Applicazioni di rete

Un'**applicazione di rete** è composta da più programmi che girano su sistemi finali (host o endpoint) diversi e comunicano tra loro attraverso la rete. Un esempio è un'applicazione Web, che include due programmi distinti: il **programma browser** in esecuzione sull'host dell'utente e il **programma Web server** in esecuzione sull'host server. Le **applicazioni** sono i programmi che girano sui dispositivi e permettono agli utenti di accedere ai **servizi**: social network, Web, messaggistica, e-mail, giochi multiutente, video in streaming, file-sharing P2P, voice over IP, videoconferenze in tempo reale, ricerca su Internet, accesso remoto e altri.

Creare un'applicazione di rete significa scrivere programmi che **girano su sistemi finali diversi** (potenzialmente con linguaggi o sistemi operativi diversi) e **comunicano attraverso la rete** tramite un protocollo specifico, ad esempio via socket. Non serve invece scrivere software per i dispositivi del nucleo della rete, poiché i dispositivi di rete **non eseguono applicazioni utente** e le funzionalità di rete sono implementate da **librerie dedicate** (come i socket). La comunicazione tra processi avviene dunque tramite socket, e le applicazioni usano i servizi del livello di trasporto (flusso di dati affidabile).

Architetture delle applicazioni

L'architettura di un'applicazione è progettata dallo sviluppatore e determina come l'applicazione è strutturata sui vari sistemi finali. Le due architetture fondamentali sono **client-server**, dove i nodi sono eterogenei ed esiste un host sempre attivo (il server) che fornisce servizi su richiesta di molti altri host (i client), e **peer-to-peer (P2P)**, dove i nodi sono idealmente omogenei e l'applicazione sfrutta la comunicazione diretta tra coppie di host connessi intermittentemente (i peer).

Applicazioni client-server

In un'architettura client-server i **client non comunicano direttamente tra loro**. Poiché il server ha un **indirizzo fisso e noto** (hostname o indirizzo IP), un client può sempre contattarlo inviandogli un pacchetto di richiesta. Per reggere il volume di richieste sono spesso coinvolti **più server**, ma il client tipicamente non se ne accorge e li percepisce come un unico server. I server multipli possono essere organizzati in modi diversi:

- Raggruppati in **data center**, con un numero enorme di server in una sede specifica (da alimentare, mantenere e ben connettere).
- Sparsi come **server distribuiti** in tutto il mondo, da interconnettere e coordinare.
- Organizzati in **data center distribuiti**.

La **web application** è l'esempio tipico: un Web server sempre attivo riceve richieste dai browser sugli host client e, quando riceve una **richiesta** di un oggetto, **risponde** inviando l'oggetto richiesto.

Applicazioni peer-to-peer

Le architetture P2P sono spesso usate per applicazioni a **traffico intenso** e non prevedono un server unico con indirizzo fisso: ogni client ha il proprio indirizzo. Le applicazioni P2P pure sono rare, poiché la maggior parte adotta **architetture ibride** che combinano client-server e P2P: ad esempio, in molte app di messaggistica istantanea i **server tracciano gli indirizzi degli utenti**, ma i messaggi utente-utente partono direttamente dagli host. Le architetture P2P sono **scalabili e distribuite**: nel file-sharing ogni peer genera carico richiedendo file, ma aggiunge anche **capacità di servizio** distribuendo file agli altri peer. Sono anche **economiche** (niente infrastrutture o banda costosi), ma presentano **problemi di sicurezza, prestazioni e affidabilità**. Applicazioni P2P tipiche sono la telefonia e la videoconferenza su Internet e il file-sharing (BitTorrent, Napster); nel file-sharing il server può anche **tracciare i file disponibili** per velocizzare la ricerca.

Comunicazione tra processi

In un'applicazione di rete ci sono **processi** su macchine diverse, potenzialmente con sistemi operativi diversi, che comunicano tramite la rete: nelle applicazioni web il processo del browser scambia messaggi con il processo del server, mentre nel file-sharing P2P un file è trasferito da un processo di un peer a un processo di un altro peer. In una coppia di processi che comunicano ci sono tipicamente un **processo client** e un **processo server**: nelle app web il browser è client e il server è server; nel file-sharing P2P il peer che scarica è visto come client e quello che carica come server. In un'applicazione P2P i processi possono “cambiare ruolo”, e la maggior parte delle applicazioni di rete consiste proprio in **coppie di processi comunicanti** che si scambiano messaggi tramite la rete sottostante.

Socket

Seguendo il modello a livelli, un **socket** è l'**interfaccia software** tra livello applicazione e livello di trasporto, e permette ai processi di inviare e ricevere messaggi sulla rete. L'analogia tipica è quella delle **cassette della posta**: un processo che vuole inviare un messaggio a un processo su un altro host lo **mette nella propria cassetta**, presumendo che esista un'**infrastruttura di trasporto** che porterà il messaggio alla cassetta del destinatario; all'arrivo sull'host di destinazione, il messaggio passa per la cassetta (socket) del processo ricevente. I socket sono usati come **Application Programming Interface** (API) tra applicazione e rete. Lo sviluppatore controlla tutto sul lato applicazione del socket, ma sul lato trasporto ha poco controllo, limitato a:

1. La **scelta** del protocollo di trasporto (TCP/UDP).
2. Eventualmente l'impostazione di alcuni **parametri di trasporto** (buffer massimo, dimensione massima del segmento, ecc.).

Scelto il protocollo di trasporto, l'applicazione è costruita assumendo come dati i servizi forniti da quel protocollo.

Indirizzamento dei processi

Proseguendo l'analogia postale, i processi hanno bisogno di un **indirizzo** per inviare messaggi. Poiché su un singolo host possono girare più applicazioni di rete, per identificare il processo ricevente servono due informazioni: l'**indirizzo** dell'host, che lavora a livello di **rete** (routing), e un **identificatore** del processo ricevente, che lavora a livello di **trasporto**. Su Internet l'host è identificato da un **indirizzo IP** (quantità a 32 bit) e il processo da un **numero di porta**. Alle applicazioni popolari sono state assegnate convenzionalmente porte specifiche: ad esempio un Web server usa la porta 80, un server di posta (SMTP) la porta 25.

Qualità del servizio (QoS) dal livello di trasporto

Il **livello di trasporto** offre protocolli per implementare su richiesta alcune proprietà, dette **Quality of Service (QoS)**:

- **Affidabilità** del trasferimento: i dati inviati da un capo sono consegnati correttamente e completamente all'altro capo.
- **Throughput**: la velocità con cui il processo mittente consegna bit al processo ricevente (bit/s).
- **Temporizzazione** (*timing*): ogni bit immesso nel socket del mittente arriva al socket del ricevente entro un intervallo di tempo.
- **Sicurezza**: cifratura e decifratura dei messaggi. Teoricamente è un problema del livello di trasporto, ma i protocolli sicuri (es. TLS, *Transport-Layer Security*) sono spesso costruiti sopra TCP, quindi in pratica sta a livello applicazione.

I due protocolli di trasporto principali sono **TCP** (*Transmission Control Protocol*), che prevede un **handshake** tra i processi e un servizio di trasferimento dati affidabile (detto **orientato alla connessione**), e **UDP** (*User Datagram Protocol*), **leggero** e con servizi minimi, senza handshake e senza garanzia di ricezione dei messaggi (detto **senza connessione**).

Protocolli applicativi

Un **protocollo di livello applicazione** definisce come i processi di un'applicazione di rete, in esecuzione su sistemi finali diversi, si scambiano messaggi. In particolare definisce:

1. I **tipi** di messaggi scambiati (es. messaggi di richiesta e di risposta).
2. La **sintassi** dei vari tipi di messaggio (campi e loro delimitazione).
3. La **semantica** dei campi, cioè il significato dell'informazione.
4. Le **regole** che determinano quando e come un processo invia messaggi e risponde.

Alcuni protocolli applicativi

Applicazione	Descrizione
DHCP	Dynamic Host Configuration Protocol, assegna indirizzi IP
DNS	Domain Name System, traduce i nomi dei siti in indirizzi IP
HTTP/HTTPS	HyperText Transfer Protocol (Secure), trasferisce pagine web
SMTP/SMTSP	Simple Mail Transfer Protocol (Secure), invia messaggi e-mail
SNMP	Simple Network Management Protocol, gestisce dispositivi di rete
Telnet/SSH	Teletype Network (Secure SHell), interfaccia a riga di comando con host remoti
FTP/FTPS	File Transfer Protocol (Secure), usato per trasferire file

Poiché alcune applicazioni scambiano **informazioni sensibili** (credenziali, dati personali, ecc.), sulle reti pubbliche i messaggi vanno protetti.

FTP e FTPS

FTP (*File Transfer Protocol*) è uno dei protocolli **più antichi** di Internet (prima versione nel 1971) e serve a trasferire file tra host sulla rete. Nella versione base il trasferimento avviene **in chiaro** (username, password e file), quindi è adatto ad usi locali o privati; la versione sicura **FTPS** (*FTP Secure*) protegge username e password e cifra i contenuti. Sia FTP che FTPS hanno due componenti: il **protocollo**, che specifica i comandi (mostrare, scaricare, cancellare file, ecc.), e un'**applicazione software** che lo implementa lato client e lato server.

Esempio FTP

Su Linux si possono usare ftp e vsftpd (*very secure FTP daemon*) rispettivamente come client e server.

Sul server:

```
$ sudo apt-get install vsftpd
$ service vsftpd status # verifica che il demone sia in esecuzione
```

Sul client:

```
$ sudo apt-get install ftp # spesso già disponibile in Ubuntu
$ ftp ADDRESS # chiederà utente e password
$ exit # chiude la connessione
```

Comandi FTP comuni

Comando	Funzione
help	Elenca tutti i comandi FTP disponibili
cd	Cambia directory sulla macchina remota

Comando	Funzione
lcd	Cambia directory sulla macchina locale
ls	Mostra file e directory della directory remota corrente
mkdir	Crea una nuova directory remota
pwd	Stampa la directory di lavoro corrente sulla macchina remota
delete	Elimina un file nella directory remota corrente
rmdir	Rimuove una directory remota
get	Copia un file dal server remoto alla macchina locale
put	Copia un file dalla macchina locale al server remoto

Lezione 4: Il livello applicazione: HTTP

Web e HTTP

Fino ai primi anni '90 Internet era usato principalmente da ricercatori, accademici e studenti universitari, per accedere a host remoti (Telnet), trasferire file (FTP), ricevere e inviare notizie ed e-mail. Queste applicazioni erano (e restano) utilissime, ma Internet restava essenzialmente sconosciuto fuori dalla comunità accademica. Nei primi anni '90 il **World Wide Web** fu la **prima applicazione di Internet** a conquistare il grande pubblico.

Il **World Wide Web** (WWW, o semplicemente Web) è una **raccolta di informazioni** — documenti, immagini, video, audio, ecc. — accessibile su Internet tramite un protocollo specifico: l'**HyperText Transfer Protocol** (HTTP). La comunicazione basata su HTTP è tipicamente **client-server**: un **programma client** traduce le richieste dell'utente in messaggi HTTP (è implementato nei **browser**, come Chrome, Firefox, Edge e Safari), mentre un **programma server** esegue la richiesta HTTP e restituisce una risposta HTTP. HTTP definisce principalmente la **struttura dei messaggi** e come client e server li scambiano; il Web e i suoi protocolli **fanno da piattaforma** per le applicazioni web: YouTube, webmail, gran parte delle app mobile e i social network.

URL

Le informazioni sul Web si chiamano **risorse** (o oggetti) e sono identificate da un **Uniform Resource Locator** (URL), una stringa così composta:

```
[protocollo]://[usrinfo@][host][:porta][/percorso][?query][#frammento]
```

Dove [protocollo] è il protocollo da usare per accedere alla risorsa (HTTP, HTTPS, FTP, ecc.); [usrinfo] (opzionale) sono le credenziali utente (username e password seguiti da @), **largamente deprecato** per motivi di sicurezza; [host] è il nome o l'indirizzo IP del server; [porta] (opzionale) è la porta da usare, spesso dedotta dal protocollo; [percorso] è il percorso della risorsa sul server; [query], preceduta da ?, specifica eventuali richieste; [frammento], preceduto da #, identifica un elemento nella risorsa (es. una sezione).

Esempi

La maggior parte delle pagine Web è costituita da un file **HTML base** e riferimenti ad altri **oggetti** (immagini, video, ecc.): una pagina con testo HTML e cinque immagini JPEG, ad esempio, ha sei oggetti — il file HTML base più le cinque immagini.

```
http://www.someSchool.edu/someDepartment/picture1.gif
```

Qui http è il **protocollo**, www.someSchool.edu l'**hostname** e /someDepartment/picture1.gif il **percorso** dell'oggetto. Un esempio reale di query:

```
https://en.wikipedia.org/w/index.php?title=SSC_Napoli
```

dove https è il protocollo, en.wikipedia.org l'hostname, /w/index.php il percorso della pagina e ?title=SSC_Napoli la query; in questo caso equivale a chiedere la pagina https://en.wikipedia.org/wiki/SSC_Napoli.

HTTP

HTTP definisce come i client Web chiedono oggetti (es. pagine) ai server Web e come i server li trasferiscono ai client. Quando l'utente chiede un oggetto (es. cliccando su un hyperlink), il **browser** invia messaggi di richiesta HTTP per gli oggetti della pagina al server, che riceve le richieste e risponde con messaggi HTTP

contenenti gli oggetti. HTTP **evolve continuamente** per soddisfare i requisiti (affidabilità vs prestazioni) del networking moderno: HTTP/1.0 (1996), HTTP/1.1 (1997), HTTP/2 (2015), HTTP/3 (2022).

Protocollo di trasporto

HTTP usa principalmente **TCP** come trasporto, ma dal 2022 (HTTP/3) può usare anche UDP: circa il 30% del traffico HTTP complessivo è oggi su UDP. Il client HTTP **per prima cosa apre una connessione** con il server; stabilita la connessione, browser e server comunicano tramite le proprie **interfacce socket**: il client invia richieste HTTP nel suo socket e riceve risposte dal suo socket, mentre il server fa viceversa.

Storicamente HTTP usava TCP per i suoi servizi utili, in particolare il **trasferimento affidabile dei dati**: TCP garantisce che richiesta e risposta arrivino integre a destinazione (se possibile). Tuttavia i servizi TCP possono rallentare la comunicazione, quindi le versioni recenti usano anche UDP, con una stima di circa il 30% di velocità in più rispetto alla sola comunicazione TCP; questo si basa sul protocollo **QUIC** (*Quick UDP Internet Connection*), che **implementa l'affidabilità sopra UDP**. Qui si vede uno dei **grandi vantaggi dell'architettura a livelli**: le applicazioni HTTP non devono preoccuparsi di raggiungibilità o perdita di dati, quindi possono essere **più semplici** (logicamente e computazionalmente), delegando gran parte del lavoro allo stack inferiore.

Assenza di stato (statelessness)

La semplicità è fondamentale per server HTTP che gestiscono migliaia di richieste al secondo: in media un server esegue circa 1000 richieste HTTP al secondo, e i grandi motori di ricerca ricevono centinaia di migliaia di query al secondo. Le applicazioni HTTP sono tipicamente **stateless**: il server non mantiene informazioni sull'interazione (sequenza di richieste/risposte) con un client specifico. Se un client chiede due volte di seguito lo stesso oggetto, il server non considera la richiesta ridondante: **reinvia semplicemente l'oggetto**, avendo completamente dimenticato il passato. Non tutte le applicazioni sono però senza stato: diverse sono **stateful**.

Connessioni persistenti e non persistenti

In molte applicazioni client e server comunicano per un periodo esteso, scambiando coppie richiesta-risposta multiple (una dietro l'altra, a intervalli regolari o intermittenti). Con un protocollo orientato alla connessione (TCP o QUIC) si possono usare **connessioni persistenti**, in cui tutte le richieste e le risposte corrispondenti passano sulla stessa connessione, oppure **connessioni non persistenti**, in cui per ogni coppia richiesta-risposta si stabilisce una nuova connessione. Oggi le applicazioni HTTP usano di default connessioni persistenti, ma client e server possono essere configurati diversamente; le prime versioni (HTTP 1.0) usavano non-persistenti di default.

Esempio di connessione non persistente

Nella connessione non persistente la connessione si chiude ogni volta che una richiesta è servita (es. un oggetto inviato). Si consideri, su TCP, una pagina composta da un file HTML base e 10 immagini JPEG, tutti e 11 gli oggetti sullo stesso server, all'URL `http://www.someschool.edu/someDepartment/home.index`. La connessione avviene così:

1. Il **processo client HTTP apre una connessione TCP** verso il server sulla porta 80 (porta di default di HTTP); si avranno due socket, uno al client e uno al server.
2. Il **client invia un messaggio di richiesta HTTP** tramite il proprio socket, includendo il percorso `/someDepartment/home.index`.
3. Il **processo server riceve la richiesta, recupera l'oggetto** (da RAM o disco), lo **incapsula** in un messaggio di risposta HTTP e lo **rimanda** al client tramite il proprio socket.
4. Il **server chiede a TCP di chiudere** la connessione; TCP la terminerà (più tardi) solo quando sarà certo che il client ha ricevuto la risposta integra (affidabilità).
5. Il **client riceve la risposta; la connessione TCP termina**. Il messaggio indica che l'oggetto è un file HTML: il client lo estrae, esamina il file e **trova i riferimenti alle 10 immagini JPEG**.
6. I primi quattro passi **si ripetono** per ciascuno degli oggetti JPEG referenziati.

Poiché la connessione non persiste tra oggetti diversi, servono **11 connessioni TCP** (e 11 coppie di socket) per l'intera pagina, e gli elementi possono arrivare in momenti diversi. Si noti che HTTP **definisce solo il protocollo di comunicazione** tra programma client e programma server, non come i contenuti sono visualizzati: la visualizzazione **dipende dal browser**, e due browser possono interpretare (e mostrare) la stessa pagina in modo diverso, ad esempio attendendo il caricamento completo o mostrandola progressivamente.

Le connessioni multiple possono stabilirsi in **sequenza** o in **parallelo**: nell'esempio, noto che servono 10 JPEG, possiamo scaricarle tutte in parallelo. I browser moderni permettono di **configurare** il grado di parallelismo: la maggior parte apre 5-10 connessioni TCP parallele, ognuna per una transazione richiesta-risposta. Il parallelismo **riduce il tempo di risposta ma aumenta il costo computazionale**; nonostante il parallelismo, stabilire più connessioni TCP induce comunque un significativo **overhead di tempo**.

RTT (Round-Trip Time)

È difficile stimare i tempi con precisione su Internet; possiamo però stimare l'overhead per completare una richiesta HTML in termini di **round-trip time** (RTT), cioè il tempo che un piccolo pacchetto impiega per andare dal client al server e tornare al client. Quando l'utente clicca su un hyperlink, il browser apre una connessione con il server. Con una richiesta TCP (orientata alla connessione), per stabilire la connessione si esegue un **three-way handshake**: il client invia un piccolo segmento TCP al server per richiedere la connessione, il server conferma e risponde con un piccolo segmento TCP, e il client conferma a sua volta.

Le prime due parti dell'handshake richiedono **un RTT**; in HTTP il client invia il messaggio di richiesta insieme alla terza parte dell'handshake (l'acknowledgment). Arrivata la richiesta, il server invia il file HTML: questa richiesta/risposta consuma **un altro RTT**. In grossolanità, il tempo di risposta totale è quindi **2 RTT più il tempo di trasmissione** del file HTML da parte del server. Un handshake simile avviene anche alla **chiusura** della connessione.

Connessioni persistenti

Nelle connessioni persistenti il server **lascia aperta** la connessione TCP dopo aver inviato una risposta: richieste e risposte tra lo stesso client e server passano sulla stessa connessione, quindi un'intera pagina Web (file HTML + 10 immagini dell'esempio) può essere inviata su **una sola connessione TCP persistente**. Anche **più pagine Web** sullo stesso server possono essere inviate allo stesso client su una sola connessione persistente. Così si risparmiano grossolanamente **2-4 RTT per oggetto**.

Pipelining

Con il **pipelining** le richieste di oggetti possono essere fatte una dietro l'altra, senza attendere le risposte alle richieste pendenti. Dagli ultimi anni (da HTTP/2) richieste e risposte multiple possono essere intervallate nella stessa connessione, ed esiste anche un meccanismo per dare priorità a richieste e risposte nella connessione. Le connessioni persistenti **con pipelining sono la modalità di default** di HTTP.

Confronto: persistenti vs non persistenti

Le connessioni **persistenti** sono più veloci (soprattutto con il pipelining) e richiedono meno risorse (soprattutto memoria), ma sono più complesse da implementare; inoltre la connessione può restare aperta inutilizzata, quindi tipicamente il server HTTP la chiude dopo un certo tempo di inattività (*timeout*). Le connessioni **non persistenti** sono facili da implementare e coerenti con la statelessness di HTTP, ma richiedono più risorse — per ogni connessione vanno allocati buffer TCP e mantenute variabili TCP sia nel client che nel server — e sono più lente, con 1-2 RTT aggiuntivi per richiesta; le connessioni, infine, non possono restare aperte.

Lezione 5: Formato dei messaggi HTTP, cookie, caching

Ripasso di HTTP

L'**HyperText Transfer Protocol (HTTP)** è uno dei protocolli più usati su Internet (applicazioni web-based). È un protocollo **client-server** che definisce la **struttura dei messaggi** (formato) e come **client e server li scambiano**; come implementare le applicazioni client/server non è invece deciso da HTTP. Usa messaggi di tipo **richiesta/risposta**, è **stateless** (client e server non ricordano le interazioni passate) e presuppone un trasporto **affidabile** e orientato alla connessione (per lo più TCP, di recente anche UDP), potendo usare connessioni **persistenti** e **non persistenti**.

Formato dei messaggi HTTP

In HTTP esistono due formati diversi per richiesta e risposta: la **richiesta** specifica il comando (metodo) che il server deve eseguire (es. dammi una pagina web, compila un form), mentre la **risposta** riporta l'esito del comando (successo, fallimento, ecc.) ed eventuali dati (es. il file richiesto). Entrambi i messaggi sono scritti in semplice testo ASCII, quindi leggibili anche dall'uomo.

Formato del messaggio di richiesta

Il messaggio di richiesta include i seguenti campi:

- Il **metodo**: il comando richiesto che il server deve eseguire.
- L'**URL**: identifica l'oggetto su cui operare.
- La **versione**: la versione di HTTP (es. HTTP/1.1).
- Le **righe di intestazione (header)**: contengono i parametri della richiesta; il loro numero e tipo non sono fissi, e ogni riga include **nome** e **valore** del parametro. Ad esempio si può specificare se si vuole una connessione persistente o non persistente, e si possono usare anche header personalizzati.
- Il **body**: specifico del metodo, contiene dati potenzialmente associati al comando (es. testo di un form).

I campi sono separati da caratteri speciali: **sp** (spazio), **cr** (*carriage return*, \r), **lf** (*line feed*, \n).

Metodi

- **GET**: recupera oggetti (risorse) dal server. È tra i comandi più usati, poiché ogni volta che navighiamo su una nuova pagina il file associato deve essere recuperato; nel metodo GET il **body è vuoto**.
- **POST**: imposta informazioni dentro oggetti (risorse) del server, e il body contiene le informazioni da inviare. Una richiesta generata da un form non necessariamente usa POST: i form HTML usano spesso GET includendo i dati nell'URL richiesto.
- **HEAD**: simile a GET, ma il server risponde con un messaggio HTTP **senza l'oggetto**; spesso usato dagli sviluppatori per il debugging.
- **PUT**: spesso usato con strumenti di pubblicazione Web; permette di caricare un oggetto in un percorso specifico di un server Web, ed è usato anche dalle applicazioni che devono caricare oggetti su server Web.
- **DELETE**: permette a un utente o a un'applicazione di eliminare un oggetto su un server Web.

Esempio di richiesta

Un **browser** (Mozilla/5.0) che implementa "HTTP/1.1" **richiede** l'oggetto "/somedir/page.html" al server "www.someschool.edu". Il messaggio ha cinque righe: la riga di richiesta GET (risorsa e versione), il body vuoto, e quattro header (host, connection, user-agent, accept-language):

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
Connection: close
User-agent: Mozilla/5.0
```

```
Accept-language: fr
```

Host: `www.someschool.edu` specifica l'host su cui risiede l'oggetto: la destinazione non è sempre il prossimo host, perché un messaggio può essere **inoltrato da un altro host** (es. proxy), quindi l'indirizzo TCP può differire dall'host di destinazione reale. **Connection:** `close` chiede una connessione non persistente (chiudi alla fine). **User-agent:** `Mozilla/5.0` specifica il tipo di browser: ciò è utile perché il **server può inviare versioni diverse** dello stesso oggetto (stesso URL) a seconda del tipo. **Accept-language: fr** indica che l'utente preferisce la versione francese dell'oggetto (se esiste).

Formato del messaggio di risposta

Il formato generale è simile alla richiesta, ma invece della riga di richiesta c'è una **riga di stato** (*status line*) che riporta l'esito del comando, con: la **versione** (la versione HTTP della risposta del server), il **codice di stato** (un numero che specifica l'esito del comando) e la **frase** (contiene il risultato della richiesta).

Codici di stato

I codici di stato sono divisi in classi:

- 100-199 **informativi**: informazioni sulla richiesta.
- 200-299 **successo**: la richiesta è stata eseguita con successo.
- 300-399 **redirezione**: sono necessarie azioni aggiuntive da parte del client.
- 400-499 **errore client**: la richiesta non può essere eseguita per un problema del client.
- 500-599 **errore server**: la richiesta non può essere eseguita per un problema del server.

Esempi tipici: **200 OK** (richiesta riuscita, l'informazione è restituita nella risposta), **301 Moved Permanently** (l'oggetto richiesto è stato spostato permanentemente; il nuovo URL è nell'header "Location" della risposta), **400 Bad Request** (errore generico, la richiesta non è stata compresa dal server), **404 Not Found** (il documento richiesto non esiste su questo server), **505 HTTP Version Not Supported** (la versione di HTTP richiesta non è supportata dal server).

Esempio di risposta

La status line indica che il server usa HTTP/1.1 e che tutto è OK (l'oggetto è stato trovato e viene inviato); il **body contiene l'oggetto richiesto**:

```
HTTP/1.1 200 OK
Connection: close
Date: Tue, 18 Aug 2015 15:44:04 GMT
Server: Apache/2.2.3 (CentOS)
Last-Modified: Tue, 18 Aug 2015 15:11:03 GMT
Content-Length: 6821
Content-Type: text/html

(data data data data data ...)
```

Connection: close comunica che la connessione verrà chiusa dopo questo messaggio (non persistente); **Date** riporta data e ora in cui la risposta è stata creata e inviata dal server; **Server** indica che il messaggio è stato generato da un server Apache (analogo a user-agent); **Last-Modified** riporta data e ora di creazione/ultima modifica dell'oggetto, utile per il caching perché i file in cache possono non essere aggiornati; **Content-Length** è il numero di byte dell'oggetto; **Content-Type** indica che l'oggetto è testo HTML (il tipo è indicato ufficialmente da questo header, non dall'estensione del file).

Cookie

Un server HTTP è tipicamente **stateless**: questo semplifica la progettazione, riduce l'uso di risorse e permette a un server di gestire migliaia di connessioni TCP simultanee. La statelessness pura è però un limite forte, perché diverse funzioni web sono specifiche del client: il carrello di Amazon dipende dal client, Netflix suggerisce contenuti in base alle preferenze, e così via. Per questi scopi HTTP usa i **cookie**: un cookie è un token digitale (ID alfanumerico) usato dai server per identificare un client specifico, **creato dal server** e consegnato al client. I cookie permettono ai siti di tracciare gli utenti (la maggior parte dei grandi siti commerciali li usa) e possono avere attributi, come la data di scadenza.

La tecnologia dei cookie ha quattro componenti principali:

1. Un header “**Set-cookie**” nel messaggio di risposta HTTP.
2. Un header “**Cookie**” nel messaggio di richiesta HTTP.
3. Un **file** sul sistema client (gestito dal browser).
4. Un **database di back-end** sul server.

Esempio

Si consideri un host client che usa regolarmente eBay e **contatta Amazon.com per la prima volta**: ha già un cookie per eBay ma non per Amazon. Quando la richiesta arriva al server Amazon, **il server crea un cookie** (numero ID) e una voce associata nel suo database di back-end, poi risponde al browser **includendo nella risposta un header “Set-cookie”** con l’ID (Set-cookie: 1678). Ricevuta la risposta, **il browser aggiunge una riga a un file cookie speciale** con l’hostname del server e il numero ID del cookie. Da quel momento le richieste del client ad Amazon saranno associate al nuovo cookie, includendo l’header **Cookie: 1678** nei messaggi HTTP. Il server Amazon può così **tracciare l’attività del client nel database**: sa esattamente quali pagine l’utente 1678 ha visitato, in che ordine e in quali momenti. Se il client torna sul sito una settimana dopo, il browser continuerà a inserire **Cookie: 1678** nelle richieste.

Usi dei cookie

Amazon (e altri siti) può usare i cookie per servizi diversi: il **carrello** (il server mantiene la lista degli acquisti intenzionati durante la navigazione), la **registrazione utente** (associando le informazioni dell’utente al cookie nel database — carta di credito, nome, e-mail, indirizzo — così da non reinserirle ogni volta) e i **consigli sui prodotti**, basati sulle pagine visitate. Sebbene i cookie semplifichino l’esperienza d’acquisto, sono **controversi** perché possono essere considerati un’invasione della privacy: combinando cookie e informazioni di account fornite dall’utente, un sito può imparare molto su di lui e potenzialmente vendere queste informazioni a terzi.

Web caching

Una **Web cache** è un’entità di rete che soddisfa richieste HTTP per conto di un server Web di origine; queste entità intermedie che lavorano per conto di altri si chiamano **proxy server**. La Web cache ha un suo spazio su disco e **conserva copie degli oggetti richiesti di recente**; un browser può essere configurato così che tutte le **richieste HTTP vadano prima alla Web cache**, per verificare se è disponibile una copia dell’oggetto.

Funzionamento

Si assume che un browser richieda `http://www.someschool.edu/campus.gif` passando per una Web cache:

1. Il **browser apre una connessione TCP** con la Web cache e le invia la richiesta HTTP dell’oggetto.
2. La Web cache **controlla se ha una copia locale** dell’oggetto; in tal caso, la restituisce al browser in un messaggio di risposta HTTP.
3. Se la **cache non ha l’oggetto**, apre una connessione TCP al server di origine (`www.someschool.edu`) e gli invia la richiesta.
4. Ricevuto l’oggetto, la **cache ne memorizza una copia** in locale e ne invia una copia al browser in un messaggio di risposta.

Si noti che la cache è **sia server** (quando fornisce oggetti) **sia client** (quando li richiede). Il caching Web è stato adottato su Internet per due ragioni: una cache può **ridurre sensibilmente il tempo di risposta** per il client, soprattutto se tra client e cache c’è una connessione ad alta velocità, e può **ridurre sensibilmente il traffico** verso Internet di un’azienda o istituzione, riducendo i costi di banda.

Hit rate

Le cache Web sono spesso installate nella rete locale di aziende e istituzioni per accelerare e ridurre il traffico. Un **hit** avviene quando una cache fornisce un oggetto senza contattare il server originale; il tasso di hit (frazione di richieste soddisfatte dalla cache) tipicamente **va da 0,2 a 0,7** e cresce al crescere dei client che usano la cache. Significa che fino al 70% delle richieste può essere servito localmente.

Il problema dell'aggiornamento: conditional GET

Il caching introduce un problema: la copia nella cache può essere **obsoleta**. Per evitarlo, HTTP ha un meccanismo che permette alla cache di verificare l'oggetto memorizzato: il **conditional GET**, un messaggio di richiesta HTTP con metodo GET e header “If-Modified-Since”. La cache **controlla se l'oggetto è aggiornato** e, in tal caso, la versione memorizzata è rispedita all'host, senza necessità di altre comunicazioni.

Caching nel browser

Il caching avviene anche localmente nei browser. Il principio è lo stesso: il browser memorizza gli oggetti in locale, così non devono essere riscaricati dal server; è una tecnica comune nei browser moderni perché **migliora drasticamente le prestazioni**. La versione locale però può non essere aggiornata, generando errori (piuttosto frequenti).

Oltre il caching

Oltre alle prestazioni, i **proxy server possono dare accesso a servizi istituzionali**: ad esempio la rete UNINA fornisce un proxy web istituzionale (proxy.unina.it). Poiché le richieste passano dal proxy istituzionale, dal punto di vista del server di origine **tutte le richieste provengono dall'istituzione**. Su Internet esistono anche diversi proxy commerciali o gratuiti.

Lezione 6: E-mail, P2P, BitTorrent

Posta elettronica e SMTP

La posta elettronica è una delle applicazioni Internet più antiche e importanti, ed è tipicamente implementata in modalità client-server con due tipi di host: lo **user agent**, l'applicazione per la gestione della posta (es. Pine, K-9 Mail), e il **mail server**, il server che archivia le mail e gestisce le cassette postali (*mailbox*) degli utenti. Diversamente da HTTP, qui la comunicazione avviene anche **tra server**, quindi servono due applicazioni/protocolli: uno tra user agent e mail server, e uno tra mail server e mail server. Gli utenti non si scambiano le mail direttamente (come nella messaggistica istantanea), perché è preferibile usare mail server più affidabili e specializzati.

Il **Simple Mail Transfer Protocol** (SMTP) è il principale protocollo applicativo tra mail server: permette ai server di scambiarsi le mail, mentre **gli user agent non usano SMTP**. Arrivata sul server, una mail è archiviata nella **mailbox dell'utente**, in attesa di essere scaricata dallo user agent. In SMTP esistono un lato **client** (mittente) e un lato **server** (ricevente), entrambi su mail server, che usano l'affidabile **TCP** per trasferire la posta.

Scenario tipico

Quando l'host A (Alice) invia un'e-mail all'host B (Bob) sono coinvolti quattro elementi: lo user agent di Alice, il mail server di Alice, lo user agent di Bob e il mail server di Bob. Il processo funziona così:

1. Alice **invoca il suo user agent**, fornisce l'indirizzo e-mail di Bob (es. bob@someschool.edu), compone il messaggio e chiede all'user agent di inviarlo.
2. Lo **user agent di Alice invia il messaggio al suo mail server**, dove viene messo in una coda di messaggi.
3. Il lato client SMTP del server di Alice vede il messaggio in coda e **apre una connessione TCP** verso un server SMTP sul mail server di Bob.
4. Dopo un handshaking SMTP iniziale, il client SMTP invia il messaggio di Alice **nella connessione TCP**.
5. Sul server di Bob, **il lato server SMTP riceve il messaggio** e lo colloca nella **mailbox di Bob**.
6. Bob **invoca il suo user agent** per leggere il messaggio quando vuole.

Dettagli della connessione

Il client SMTP (sul server mittente) **stabilisce una connessione TCP alla porta 25** del server SMTP (sul server ricevente); se il server è giù, il client riprova più tardi. Stabilita la connessione, **server e client eseguono un handshaking a livello applicazione**, presentandosi prima di trasferire informazioni: durante l'handshaking il client SMTP indica l'indirizzo e-mail del mittente e quello del destinatario. SMTP può contare sul **servizio di trasferimento affidabile di TCP** per consegnare il messaggio al server senza errori.

Accesso alla posta

Il deployment di server SMTP è **più affidabile** rispetto alla posta diretta utente-utente: i server sono, per definizione, sempre accesi e visibili a tutti i dispositivi della rete, sono certificati, e possono ritentare continuamente l'invio in caso di fallimento (un processo computazionalmente costoso). D'altro canto, per accedere alle mail sul server serve **un'applicazione client-server aggiuntiva** con un protocollo diverso; i più diffusi sono il **Post Office Protocol - Versione 3** (POP3), l'**Internet Mail Access Protocol** (IMAP) e **HTTP**.

POP3

POP3 è un protocollo di accesso alla posta estremamente semplice. Lo user agent (client) apre una **connessione TCP** al mail server sulla porta 110; stabilita la connessione, POP3 procede in tre fasi:

1. **Autorizzazione:** lo user agent invia username e password per autenticare l'utente.
2. **Transazione:** lo user agent può recuperare i messaggi, marcarli per la cancellazione, rimuovere i marcatori, ottenere statistiche sulla posta.
3. **Aggiornamento:** dopo il comando *quit* che chiude la sessione, il mail server **elimina i messaggi** marcati per la cancellazione.

IMAP

Con POP3 i messaggi possono solo essere scaricati o cancellati: azioni come cercare o organizzare la posta in cartelle non sono previste e vanno fatte in locale. L'**Internet Mail Access Protocol** (IMAP) permette invece ai server di offrire funzionalità aggiuntive:

- Gestione e **creazione di cartelle**.
- **Ricerca** nelle cartelle remote di messaggi che corrispondono a criteri specifici.
- Ottenere solo parti dei messaggi (es. solo header, solo allegati).
- Connettere **più client** allo stesso server.

Accesso via HTTP

Sempre più utenti inviano e leggono e-mail tramite il **browser**: l'accesso Web via HTTP fu introdotto da Hotmail a metà anni '90 ed è ora l'approccio dominante (Google, Yahoo!, Virgilio, ecc.), usato da quasi tutte le università e le grandi aziende. Con questo servizio **lo user agent è un normale browser**, e l'utente comunica con la propria mailbox remota via HTTP anziché via POP3 o IMAP; per lo scambio dei messaggi, invece, **i mail server restano sullo standard SMTP**.

Peer-to-peer

Diversamente dal client-server, l'architettura P2P fa un uso minimo (o nullo) di server: ci sono **coppie di host connessi intermittentemente** (peer) che comunicano direttamente tra loro. I peer **non sono di proprietà di un provider**, ma sono desktop e laptop controllati dagli utenti. Un'applicazione naturale del P2P è la **condivisione di file**: nel client-server il server deve condividere i file con tutti i client (un grave carico, con esigenze di banda elevate), mentre nel P2P i peer che ricevono il file possono a loro volta **condividerlo con altri peer**, essendo in un certo senso client e server allo stesso tempo. Nel file sharing l'approccio P2P tipicamente **scala meglio** di quello client-server; d'altro canto può essere piuttosto **complesso da implementare**, e possono esserci **problemi di sicurezza** nell'avere tutti questi client in comunicazione diretta.

BitTorrent

BitTorrent è un protocollo P2P popolare per la condivisione e distribuzione di file, con 150-170 milioni di utenti stimati (nel 2023). Un **torrent** è l'insieme di tutti i peer che partecipano alla distribuzione di un particolare file; i peer in un torrent scaricano l'uno dall'altro **blocchi (chunk) di pari dimensione** (dimensione tipica: 256 kbyte). Quando un peer entra in un torrent (senza chunk), inizia **accumulando chunk** e, mentre li scarica, **li carica anche verso altri peer**; una volta **ottenuto l'intero file**, può egoisticamente **uscire dal torrent**, oppure altruisticamente **restare** e continuare a caricare chunk verso altri peer. I peer possono **uscire dal torrent in qualunque momento** con un sottoinsieme di chunk e rientrarvi più tardi.

Il tracker

Ogni torrent ha un nodo di infrastruttura chiamato **tracker** che tiene traccia dei peer partecipanti (i tracker sono in pratica server). Quando un **nuovo peer** entra nel torrent, si **registra** presso il tracker e **periodicamente lo informa** del proprio stato; il tracker **fornisce gli indirizzi IP** di un sottoinsieme casuale di peer del torrent. Ottenuta la lista, il nuovo host cerca di aprire **connessioni TCP simultanee con tutti i peer della lista** (i vicini); durante l'esecuzione, alcuni peer connessi possono uscire, mentre altri (fuori dalla lista iniziale) possono cercare di aprire nuove connessioni. In ogni istante ciascun peer ha **un sottoinsieme dei**

chunk, con peer diversi che hanno sottoinsiemi diversi; periodicamente un host chiede a ciascun peer connesso (via TCP) **la lista dei chunk che possiede**.

Il client BitTorrent

Il **client in download** decide quali chunk richiedere (e a chi) seguendo il principio *rarest-first*: vengono prioritizzati i chunk con **meno copie** tra i vicini, così i **chunk più rari vengono ridistribuiti più velocemente**, in modo da grossolanamente pareggiare il numero di copie nel torrent. Il **client in upload** decide invece quali richieste servire seguendo due principi intrecciati:

- **Trading**: l'host dà priorità ai migliori 4 vicini che in quel momento forniscono dati alla velocità più alta; la verifica è periodica (10 secondi) e la lista dei 4 migliori viene aggiornata.
- **Selezione casuale**: ogni 30 secondi l'host sceglie anche **un vicino aggiuntivo a caso** e gli invia chunk. Se questo scambio casuale va bene (i due host sono buoni partner), entrano nelle rispettive liste dei migliori; questo processo permette anche a peer con velocità di upload compatibili di trovarsi.

Lezione 7: Il DNS

Introduzione al DNS

Un host si può identificare in due modi: per **hostname** e per **indirizzo IP**. Le persone preferiscono il più mnemonico hostname, mentre i dispositivi di rete preferiscono gli IP, a lunghezza fissa e strutturati gerarchicamente (esempio: `www.unina.it` → `143.225.15.50`). Il **Domain Name System** (DNS) è un protocollo di livello applicazione che gestisce la traduzione da hostname a indirizzo IP. È un protocollo **client-server**: un client DNS chiede a un server DNS una specifica traduzione nome→indirizzo. I server DNS sono spesso macchine UNIX con il software *Berkeley Internet Name Domain* (BIND), **tipicamente via UDP** sulla porta 53 (in rari casi anche TCP).

Un repository (tabella) centralizzato sarebbe impossibile da gestire, a causa dell'elevato numero di host e della distanza geografica (con i relativi ritardi). Il DNS è quindi un sistema **distribuito e decentralizzato**: gerarchico, basato su domini e implementato tramite un database distribuito.

Il resolver

Il **resolver** è il programma che restituisce l'indirizzo IP associato a un nome: interroga un server DNS locale, e tutti i messaggi scambiati sono pacchetti UDP.

Lo spazio dei nomi DNS

Lo spazio dei nomi è una gerarchia organizzata da ICANN (*Internet Corp. for Assigned Names and Numbers*), con circa 250 domini di primo livello; ogni dominio è partizionato in sotto-domini, e così via. I domini di primo livello sono di due tipi: **generici** (aero, com, edu, ...) e **di paese** (uk, it, tv, ...), tutti gestiti da registrar nominati da ICANN. Lo spazio dei nomi è gerarchico **dalla radice verso il basso**, con parti delegate a organizzazioni diverse.

Domini generici di primo livello

Controllati da ICANN, che nomina i registrar che li gestiscono:

Dominio	Uso previsto	Attivo dal	Ristretto?
com	Commerciale	1985	No
edu	Istituzioni educative	1985	Sì
gov	Governo	1985	Sì
int	Organizzazioni internazionali	1988	Sì
mil	Militare	1985	Sì
net	Provider di rete	1985	No
org	Organizzazioni no-profit	1985	No
aero	Trasporto aereo	2001	Sì
biz	Aziende	2001	No
coop	Cooperative	2001	Sì
info	Informativo	2002	No
museum	Musei	2002	Sì
name	Persone	2002	No
pro	Professionisti	2002	Sì
cat	Catalano	2005	Sì

Dominio	Uso previsto	Attivo dal	Ristretto?
jobs	Lavoro	2005	Sì
mobi	Dispositivi mobili	2005	Sì
tel	Dati di contatto	2005	Sì
travel	Industria del viaggio	2005	Sì
xxx	Industria del sesso	2010	No

Nomi di dominio

I nomi sono cammini dal dominio verso l'alto fino alla radice (esempio: `eng.mit.edu`). I cammini **assoluti** terminano con un punto, mentre quelli **relativi** vanno interpretati. È possibile registrarsi sotto più domini di primo livello (es. `ibm.com`, `ibm.us`); il **cyber-squatting** è l'acquisto di domini solo per rivenderli poi ad aziende interessate. Ogni dominio controlla l'allocatione dei domini sottostanti, quindi creare un nuovo dominio richiede il permesso del dominio padre. Va infine notato che la nomenclatura non segue i confini fisici o geografici.

Record di risorsa (resource records)

I campi di un record di risorsa sono:

- **Nome di dominio (NAM)**: indica il dominio cui il record si applica; è la chiave di ricerca principale per le query DNS. Normalmente esistono più record per dominio, su server diversi.
- **Time to live (TTL)**: vita del record; lungo per host stabili (es. $86400 = 1$ giorno), corto per quelli volatili (es. $60 = 1$ minuto). Valore massimo $2^{31} - 1$, circa 68 anni.
- **Classe (CLASS)**: sempre **IN** (Internet) per i record Internet; altri codici sono raramente usati.
- **Tipo (TYPE)**: vedi sotto.
- **Valore (RDATA)**: dipende dal tipo di record; può contenere un dominio, un valore o una stringa ASCII.

Tipi di record

- **SOA** (*Start of Authority*): informazioni sulla zona del server, contatto dell'amministratore, ecc.
- **A**: indirizzo IPv4 a 32 bit di un'interfaccia dell'host (alcuni host hanno più interfacce); equivalente **AAAA** per IPv6.
- **MX** (*Mail eXchange*): nome dell'host che accetta la posta nel dominio.
- **NS** (*Name Server*): name server del dominio, usato nella risoluzione dei nomi.
- **CNAME** (*Canonical Name*): permette alias; la lookup prosegue con il nuovo nome.
- **PTR** (*Pointer*): alias per la lookup inversa; la lookup non prosegue, restituisce il nome.

Altri tipi:

Tipo	Significato	Valore
SOA	Inizio di autorità	Parametri della zona
A	Indirizzo IPv4 di un host	Intero a 32 bit
AAAA	Indirizzo IPv6 di un host	Intero a 128 bit
MX	Mail exchange	Priorità, dominio che accetta la posta
NS	Name server	Nome di un server per il dominio
CNAME	Nome canonico	Nome di dominio
PTR	Puntatore	Alias di un indirizzo IP
SPF	Sender policy framework	Codifica testuale della policy di invio mail
SRV	Servizio	Host che lo fornisce
TXT	Testo	Testo ASCII descrittivo

Esempio (dati autorevoli per cs.vu.nl)

```
cs.vu.nl.      86400  IN  SOA  star boss (9527,7200,7200,241920,86400)
cs.vu.nl.      86400  IN  MX   1 zephyr
cs.vu.nl.      86400  IN  MX   2 top
cs.vu.nl.      86400  IN  NS   star

star           86400  IN  A    130.37.56.205      ; name server
zephyr         86400  IN  A    130.37.20.10      ; mail gateway
top            86400  IN  A    130.37.20.11
www            86400  IN  CNAME star.cs.vu.nl
ftp            86400  IN  CNAME zephyr.cs.vu.nl

flits          86400  IN  A    130.37.16.112
flits          86400  IN  A    192.31.231.165
flits          86400  IN  MX   1 flits
flits          86400  IN  MX   2 zephyr
flits          86400  IN  MX   3 top

rowboat        IN  A    130.37.56.201
               IN  MX  1 rowboat
               IN  MX  2 zephyr

little-sister  IN  A    130.37.62.23
laserjet       IN  A    192.31.231.216
```

Name server

Un server DNS unico e centralizzato non avrebbe senso. Lo spazio dei nomi DNS è diviso in **zone non sovrapposte**, e il partizionamento spetta all'amministratore della zona; ogni zona contiene uno o più name server, uno **primario** e uno o più **secondari**. I name server contengono i dati delle porzioni di spazio dei nomi dette **zone**.

Risoluzione dei nomi

1. Un host interroga un name server locale.
2. Se il dominio ricade nella giurisdizione (zona) del server locale, viene restituito un record **autoritativo**.
3. Altrimenti il server locale gestisce la query **ricorsivamente**, emettendo query remote: se il server DNS ha l'indirizzo, lo restituisce; altrimenti restituisce il name server di una zona più in basso; il processo prosegue **iterativamente** fino a trovare l'indirizzo.

I **root server** sono il punto di partenza quando non si ha alcuna informazione: sono 13, da **a.root-servers.net** a **m.root-servers.net**, altamente replicati. Poiché sono molto occupati, di solito restituiscono informazioni su zone inferiori (non sui singoli record).

Esempio: nslookup

Possiamo eseguire manualmente le query iterative fatte da un server DNS locale; è possibile anche la **lookup inversa** (nome dall'indirizzo). Il server di default (in questo caso il router) è **127.0.0.53** (porta 53). I record restituiti sono **non autoritativi** (in cache), cioè non provengono dal server DNS che gestisce la zona. Il server locale gestisce l'intera risoluzione via query iterative, mentre l'host emette una **query ricorsiva**: gli viene restituita la risposta completa oppure un errore.

```
andrea@turgia:~$ nslookup www.vatican.va
Server:  127.0.0.53
Address: 127.0.0.53#53

Non-authoritative answer:
www.vatican.va canonical name = wcm-disp.spc.va.
Name:     wcm-disp.spc.va
Address:  185.152.70.33
```

Tipi di query e record

I server DNS locali gestiscono query ricorsive: è un servizio verso i propri host. Molti server DNS (i più occupati, come quelli vicini alla radice) invece non gestiscono query ricorsive. I record sono **autoritativi** o **in cache**: i record in cache scadono (si ricordi il TTL), ma il caching aiuta le prestazioni perché tutte le risposte vengono messe in cache, così l'indirizzo può essere restituito subito dal server locale. Se arriva una richiesta per `cs.mit.edu` e il server ha l'indirizzo del DNS server per `mit.edu`, può interrogare direttamente quel server; i root server sono interrogati solo in assenza di qualunque informazione su un dominio.

Il resolver locale

C'è un altro tipo importante di server DNS: il **local DNS server** (o **DNS resolver**). Non appartiene strettamente alla gerarchia di server ed è **tipicamente gestito dagli ISP**; anche Google fornisce server simili in tutto il mondo (Google Public DNS) agli indirizzi 8.8.8.8 e 8.8.4.4. Quando un host si connette a un ISP, l'ISP fornisce gli IP di uno o più dei suoi server DNS locali (tipicamente “vicini” all'host). Quando l'host fa una query DNS, questa va al server locale, che **agisce da proxy**, inoltrandola alla gerarchia DNS.

Alias

Il DNS fornisce un servizio di **aliasing** per server, in cui nomi complicati o lunghi (detti **canonici**) sono associati a nomi più semplici e corti (detti **alias**). Con l'**host aliasing**, host con hostname complicati possono essere associati ad alias più mnemonici (esempio: `relay1.west-coast.enterprise.com` → `www.enterprise.com`). Il DNS può essere invocato da un'applicazione **per ottenere l'hostname canonico (originale)** a partire da un alias, insieme all'indirizzo IP associato.

Distribuzione del carico

Il DNS può essere usato per la **distribuzione del carico** tra server replicati (detto **round-robin DNS**). I siti molto trafficati (es. Google, Amazon, CNN) sono tipicamente **replicati su più server**, ognuno su un sistema finale diverso con un IP diverso (più IP associati a un solo hostname canonico). Il database DNS contiene questo insieme di IP: quando i client fanno una query per un nome mappato su un insieme di indirizzi, il server può **ruotare l'ordine** in cui gli indirizzi vengono restituiti.

Lezione 8: Programmazione di rete (socket)

Creare applicazioni di rete

La maggior parte delle applicazioni di rete include un programma **lato client** e uno **lato server** che comunicano attraverso la rete. Eseguendo questi due programmi si creano **un processo client** e **un processo server**, che comunicano leggendo e scrivendo su **socket**.

Scelta dei protocolli

I protocolli si dividono in **aperti**, le cui regole sono pubbliche, e **proprietary**, protocolli “custom” non pubblicati apertamente in un RFC o altrove. Per il trasporto ci sono due protocolli utilizzabili: **TCP** (*Transmission Control Protocol*), **orientato alla connessione**, che fornisce un canale affidabile a flusso di byte tra i due sistemi finali, e **UDP** (*User Datagram Protocol*), **senza connessione**, che invia pacchetti indipendenti da un sistema finale all'altro senza garanzie di consegna.

Socket

I **socket** sono un elemento centrale della creazione di applicazioni di rete: implementano (in modo “black-box”) le funzionalità dei livelli di trasporto (UDP/TCP) e di rete (IP). Esistono applicazioni (middleware) che possono **incapsulare i socket**, semplificandone la creazione e fornendo funzionalità più complesse; tuttavia **le API di base restano molto usate**. I socket UDP e TCP sono disponibili in quasi **tutti i linguaggi e sistemi operativi** (C, C++, C#, Java, Perl, Python, Matlab, ecc.), e l'implementazione sottostante e l'uso dipendono dalla configurazione delle macchine. Nel dominio Unix si usano i **Berkeley socket** (detti anche BSD o POSIX socket) per TCP e UDP: API C native.

Definizioni (strutture in C)

```
#include <netinet/in.h>
#include <arpa/inet.h>    // per inet_addr()
#include <sys/types.h>    // alcuni tipi custom

struct sockaddr_in {
    short sin_family;      // famiglia, tipicamente AF_INET (IPv4)
    unsigned short sin_port; // numero di porta, es. htons(3490)
    struct in_addr sin_addr;
    char sin_zero[8];      // tipicamente zeri: la struttura ha la stessa
                          // dimensione di sockaddr e può esserle "castata"
};

struct in_addr {
    unsigned long s_addr;  // indirizzo IP: inet_addr() o INADDR_ANY per localhost
};
```

Creazione: socket()

```
#include <sys/socket.h>

int sockfd = socket(int domain, int type, int protocol);
```

I parametri sono: **domain**, la famiglia di indirizzi (es. **AF_INET**); **type**, il tipo di socket (**SOCK_STREAM** per TCP, **SOCK_DGRAM** per UDP); **protocol**, il protocollo specifico da usare nel socket (spesso 0, nessun protocollo particolare). Il valore restituito **sockfd** è un file descriptor che identifica il socket appena creato: i socket

seguono la filosofia Unix “tutto è un file”, per cui le sorgenti di I/O sono trattate come file, con file descriptor associato.

Associazione: bind()

```
#include <sys/socket.h>

int val = bind(int socket, const struct sockaddr *address, socklen_t address_len);
```

socket è il file descriptor del socket da associare; address è un puntatore a una struttura sockaddr che specifica porta e indirizzo (address.sin_addr è spesso impostato a INADDR_ANY, così da ascoltare connessioni da tutti gli indirizzi, IP multipli); address_len è la lunghezza della struttura sockaddr (si può usare sizeof()); val vale 0 in caso di successo, -1 altrimenti. Si noti che bind **si usa sui server** (il socket va legato a una porta specifica) ed è **opzionale sui client**, poiché il sistema operativo assegna automaticamente una porta libera.

Invio e ricezione (UDP)

```
#include <sys/socket.h>

int ob = sendto(int osock, const void *obuf, size_t olen, int oflags,
               const struct sockaddr *oaddr, socklen_t oaddr_len);
int ib = recvfrom(int isock, void *ibuf, size_t ilen, int iflags,
                 struct sockaddr *iaddr, socklen_t *iaddr_len);
```

- osock/isock: file descriptor su cui inviare/ricevere il messaggio.
- obuf/ibuf: puntatori ai buffer con il messaggio da inviare/ricevere.
- olen/ilen: lunghezze dei messaggi in byte.
- oflags/iflags: flag (tipicamente 0 o MSG_WAITALL per attendere tutti i byte).
- oaddr/iaddr: puntatori a sockaddr con l'indirizzo del ricevente/mittente.
- oaddr_len/iaddr_len: lunghezza in byte della struttura sockaddr.
- ob/ib: numero di byte effettivamente inviati/ricevuti.

Chiusura: close()

```
#include <unistd.h>

int val = close(int socket);
```

socket è il file descriptor del socket da chiudere; val vale 0 in caso di successo, -1 altrimenti.

Esempio UDP in C/C++

Client

```
int main() {
    int sockfd;
    char buffer[1024];
    const char *hello = "Hello from client";
    struct sockaddr_in servaddr;

    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("socket creation failed");
        exit(EXIT_FAILURE);
    }

    memset(&servaddr, 0, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_port = htons(8080);
    servaddr.sin_addr.s_addr = INADDR_ANY;
```

```

int n;
socklen_t len;

sendto(sockfd, (const char *)hello, strlen(hello),
        0, (const struct sockaddr *)&servaddr, sizeof(servaddr));
std::cout << "Hello message sent." << std::endl;

n = recvfrom(sockfd, (char *)buffer, 1024,
              MSG_WAITALL, (struct sockaddr *)&servaddr, &len);
buffer[n] = '\0';
std::cout << "Received \"" << buffer << "\"" << std::endl;

close(sockfd);
return 0;
}

```

Nota: per comunicare con un altro computer usare `inet_addr()` al posto di `INADDR_ANY` (es. `inet_addr("192.168.1.10")`).

Server

```

int main() {
    int sockfd;
    char buffer[1024];
    const char *hello = "Hello from server";
    struct sockaddr_in servaddr, cliaddr;

    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("socket creation failed");
        exit(EXIT_FAILURE);
    }

    memset(&servaddr, 0, sizeof(servaddr));
    memset(&cliaddr, 0, sizeof(cliaddr));

    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = INADDR_ANY;
    servaddr.sin_port = htons(8080);

    if (bind(sockfd, (const struct sockaddr *)&servaddr, sizeof(servaddr)) < 0) {
        perror("bind failed");
        exit(EXIT_FAILURE);
    }

    socklen_t len = sizeof(cliaddr);
    int n;

    n = recvfrom(sockfd, (char *)buffer, 1024,
                  MSG_WAITALL, (struct sockaddr *)&cliaddr, &len);
    buffer[n] = '\0';
    std::cout << "Received \"" << buffer << "\"" << std::endl;

    sendto(sockfd, (const char *)hello, strlen(hello),
            0, (const struct sockaddr *)&cliaddr, len);
    std::cout << "Hello message sent." << std::endl;

    close(sockfd);
    return 0;
}

```

Da UDP a TCP

Diversamente da UDP, in TCP client e server devono fare **handshake** prima di trasmettere messaggi. Sul lato server si usano **due socket**: un **welcoming socket**, sempre attivo, con cui i client fanno l'handshake col

server, e un **socket specifico del client**, creato dopo l'handshake e usato da client e server per comunicare. Il three-way handshake, che avviene nel livello di trasporto, è completamente **invisibile ai programmi client e server**; accettata la connessione (handshake riuscito), **la comunicazione passa al nuovo socket creato**.

Connessione lato client: connect()

```
#include <sys/socket.h>

int val = connect(int socket, const struct sockaddr *address, socklen_t address_len);
```

socket è il file descriptor del socket con cui connettersi; address è un puntatore a sockaddr con l'indirizzo del server; address_len è la lunghezza della struttura (si può usare sizeof()); val vale 0 in caso di successo, -1 altrimenti.

Attesa di connessione lato server: listen() e accept()

Queste funzioni funzionano in combinazione con la connect() del client: listen() apre una coda dove sono messe le connessioni in arrivo, mentre accept() apre il socket specifico del client.

```
#include <sys/socket.h>

int val = listen(int socket, int backlog);
int new_sockfd = accept(int socket, struct sockaddr *address, socklen_t *address_len);
```

- socket: file descriptor del socket su cui attendere connessioni (deve essere in bind su indirizzo/porta).
- backlog: lunghezza massima della coda di connessioni pendenti.
- address: può essere NULL, o un puntatore a sockaddr dove verrà restituito l'indirizzo del socket che si connette.
- address_len: lunghezza dell'indirizzo.
- new_sockfd: nuovo descrittore di socket su cui client e server comunicheranno.
- val: 0 in caso di successo, -1 altrimenti.

Invio e ricezione (TCP)

```
#include <sys/socket.h>
int ob = send(int osock, const void *obuf, size_t olen, int flags);

#include <unistd.h>
int ib = read(int isock, void *ibuf, size_t ilen);
```

osock/isock sono i file descriptor su cui inviare/ricevere, obuf/ibuf i puntatori ai buffer, olen/ilen le lunghezze in byte, flags il tipo di trasmissione (dipende dal protocollo, tipicamente 0), e ob/ib i byte effettivamente inviati/ricevuti. Poiché gli indirizzi di client e server sono già stati specificati in fase di connessione, non serve farlo qui come in sendto/recvfrom.

Esempio TCP in C/C++

Client

```
int main() {
    int sockfd, status;
    char buffer[1024];
    const char *hello = "Hello from client";
    struct sockaddr_in servaddr;

    if ((sockfd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        perror("socket creation failed");
        exit(EXIT_FAILURE);
    }
}
```

```

memset(&servaddr, 0, sizeof(servaddr));
servaddr.sin_family = AF_INET;
servaddr.sin_port = htons(8080);
servaddr.sin_addr.s_addr = INADDR_ANY;

int n;

if ((status = connect(sockfd, (struct sockaddr *)&servaddr, sizeof(servaddr))) < 0) {
    printf("\nConnection Failed \n");
    return -1;
}

send(sockfd, hello, strlen(hello), 0);
std::cout << "Hello message sent." << std::endl;

n = read(sockfd, buffer, 1024);
std::cout << "Received \"" << buffer << "\"" << std::endl;

close(sockfd);
return 0;
}

```

Server

```

int main() {
    int sockfd, new_socket;
    char buffer[1024];
    const char *hello = "Hello from server";
    struct sockaddr_in servaddr, cliaddr;

    if ((sockfd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        perror("socket creation failed");
        exit(EXIT_FAILURE);
    }

    memset(&servaddr, 0, sizeof(servaddr));
    memset(&cliaddr, 0, sizeof(cliaddr));

    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = INADDR_ANY;
    servaddr.sin_port = htons(8080);

    if (bind(sockfd, (const struct sockaddr *)&servaddr, sizeof(servaddr)) < 0) {
        perror("bind failed");
        exit(EXIT_FAILURE);
    }

    if (listen(sockfd, 3) < 0) {
        perror("listen");
        exit(EXIT_FAILURE);
    }

    int addrlen = sizeof(cliaddr);
    if ((new_socket = accept(sockfd, (struct sockaddr *)&cliaddr,
                            (socklen_t *)&addrlen)) < 0) {
        perror("accept");
        exit(EXIT_FAILURE);
    }

    int n = read(new_socket, buffer, 1024);
    std::cout << "Received \"" << buffer << "\"" << std::endl;

    send(new_socket, hello, strlen(hello), 0);
    std::cout << "Hello message sent." << std::endl;

    close(new_socket);
}

```

```
    close(sockfd);  
    return 0;  
}
```

Lezione 9: Programmazione di rete e DNS

Ripasso: socket

I **socket** sono un elemento centrale della creazione di applicazioni di rete e gestiscono (in modo “black-box”) le funzionalità dei livelli di trasporto (UDP/TCP) e di rete (IP). Esistono applicazioni (middleware) che **incapsulano i socket**, semplificandone la creazione e fornendo funzionalità più complesse; tuttavia **le API di base restano molto usate**.

Comandi socket UDP

Per una connessione UDP client e server devono entrambi aprire un socket; il server deve fare il bind del socket su una porta specifica. Poi si scambiano messaggi con le funzioni `sendto` e `recvfrom`: IP e porta del client si ottengono tramite `recvfrom`, e si può ciclare su `sendto/recvfrom` per mantenere viva la comunicazione. A comunicazione finita, entrambi gli host chiudono il socket; se solo il client chiude il socket, il server può accettare altri client.

Comandi socket TCP

Per una connessione TCP client e server devono entrambi aprire un socket. Il server deve fare il bind su una porta specifica, `listen` e infine `accept` le nuove connessioni; il client deve fare `connect`. Poi si scambiano messaggi con `send` e `read`: IP e porta del client si ricavano dal nuovo socket creato, e si può ciclare su `send/read` per mantenere viva la comunicazione. A comunicazione finita, entrambi gli host chiudono il socket. Il server dovrebbe fare una `read` aggiuntiva prima di chiudere il socket specifico del client (rilascio della connessione), altrimenti bisogna aspettare circa 4 minuti per riusare la porta; il server può anche tornare alla funzione `accept` e accogliere un altro client.

Traduzione DNS in C/C++

Possiamo creare socket tra due host, ma **serve conoscere l'indirizzo (IP) e la porta del server**. Su Internet gli host sono tipicamente identificati per **hostname** anziché per IP, e i Berkeley socket (C/C++) funzionano solo con indirizzi IP: non sorprende, poiché **i socket lavorano al livello di trasporto** mentre **gli hostname al livello applicazione**. Per connettersi a un host via hostname serve quindi la **traduzione DNS** per ottenere l'IP da passare al socket.

C/C++ offre funzioni e strutture che fanno questa traduzione, usate per contattare i server DNS e recuperare gli indirizzi: in particolare si usa `gethostbyname()`. La funzione **non restituisce solo l'IP**, ma una struttura `hostent` con informazioni sull'host: il nome canonico, gli eventuali alias e uno o più indirizzi.

La struttura hostent

```
#include <netdb.h>

struct hostent {
    char *h_name;           // nome originale dell'host (canonico)
    char **h_aliases;       // lista di alias (terminata da NULL)
    int h_addrtype;         // famiglia dell'indirizzo (tipicamente AF_INET)
    int h_length;           // lunghezza dell'indirizzo (tipicamente 4 byte)
    char **h_addr_list;     // lista di indirizzi (terminata da NULL)
};

// per ragioni di compatibilità
#define h_addr h_addr_list[0]
```

Un DNS può fornire una lista di indirizzi (che può essere in ordine casuale se la query va a server round-robin DNS): si può prendere il primo elemento dell'array (`h_addr_list[0]` o `h_addr`). Gli indirizzi restituiti si possono castare alla struttura `in_addr` usata dai socket.

gethostbyname()

```
#include <netdb.h>

struct hostent *host_info = gethostbyname(const char *name);
```

`name` è la stringa con l'hostname da tradurre, mentre `host_info` è un puntatore alla struttura `hostent` con le informazioni sull'host (NULL in caso di errore). Esiste anche `gethostbyaddr()`, che restituisce una struttura `hostent` per un dato indirizzo IP.

Esempio completo

```
#include <iostream>
#include <cstring>
#include <string>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <unistd.h>
#include <netdb.h>
#include <stdio.h>

int main(int argc, char **argv) {
    struct hostent *server_info;
    char *server_name;

    if (argc < 2) {
        std::cout << "No hostname specified" << std::endl;
        return 0;
    } else {
        server_name = argv[1];
    }

    // invoca il DNS e controlla la risposta
    server_info = gethostbyname(server_name);
    if (server_info == NULL) {
        std::cout << "Could not resolve hostname " << server_name << " :(" << std::endl;
        return 0;
    }

    // nome canonico
    std::cout << "Canonical name:" << std::endl;
    std::cout << "\t" << server_info->h_name << std::endl;

    // lista di alias
    std::cout << "Aliases:" << std::endl;
    int i = 0;
    while (server_info->h_aliases[i] != NULL) {
        std::cout << "\t" << server_info->h_aliases[i] << std::endl;
        i++;
    }

    // lista di IP
    std::cout << "IPs:" << std::endl;
    i = 0;
    while (server_info->h_addr_list[i] != NULL) {
        std::cout << "\t"
                  << inet_ntoa(*(struct in_addr *)server_info->h_addr_list[i])
                  << std::endl;
        i++;
    }
}
```

```
    return 0;
}
```

Esempio: richiesta HTTP via socket

Creiamo un codice C++ che implementa da zero una richiesta HTTP (HEAD) verso una pagina web: la pagina “cenni storici” del sito UNINA, <http://www.unina.it/chi-siamo/cenni-storici>. Per una richiesta HTTP si specificano tre elementi: l’hostname del server, il numero di porta e la risorsa da recuperare (percorso dell’oggetto).

Creazione del socket TCP

```
int main() {
    int socket_desc;
    struct sockaddr_in serv_addr;
    struct hostent *server;
    char buffer[4096];

    socket_desc = socket(AF_INET, SOCK_STREAM, 0);
    if (socket_desc < 0) {
        std::cout << "failed to create socket" << std::endl;
        return 0;
    }
    ...
}
```

Connessione al server web

```
server = gethostname("www.unina.it");
if (server == NULL) {
    std::cout << "could not resolve hostname :(" << std::endl;
    close(socket_desc);
    return 0;
}

bzero((char *)&serv_addr, sizeof(serv_addr));
serv_addr.sin_family = AF_INET;
serv_addr.sin_port = htons(80);
bcopy((char *)server->h_addr,
      (char *)&serv_addr.sin_addr.s_addr, server->h_length);

if (connect(socket_desc, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
    std::cout << "connection failed :(" << std::endl;
    close(socket_desc);
    return 0;
}
```

Qui `gethostname()` invoca il DNS e ricava l’IP del server dall’hostname (dentro la struttura `hostent`); l’IP della struttura restituita è copiato nella struttura `serv_addr` per la successiva `connect`.

Richiesta e risposta

```
const char *request =
    "HEAD /chi-siamo/cenni-storici HTTP/1.1\r\n"
    "Host: www.unina.it\r\n"
    "Connection: close\r\n\r\n";

if (send(socket_desc, request, strlen(request), 0) < 0) {
    std::cout << "failed to send request..." << std::endl;
    close(socket_desc);
}
```

```
    return 0;
}
std::cout << "message sent:" << std::endl;
std::cout << request << std::endl;

int n = recv(socket_desc, buffer, sizeof(buffer), 0);
std::cout << "received " << n << " bytes:" << std::endl;
std::cout << buffer;

close(socket_desc);
return 0;
```

La richiesta HTTP è definita nella stringa `request`. Creiamo una **connessione non persistente**, dato che vogliamo controllare una sola pagina; infine si chiude il socket e si stampa la risposta ricevuta.

Lezione 10: Il livello di trasporto

Dal livello applicazione al livello di trasporto

Un protocollo di trasporto fornisce soprattutto **comunicazione logica tra processi applicativi** su host diversi: dal punto di vista dell'applicazione, gli host che eseguono i processi **sembrano direttamente connessi**, come sulla stessa macchina, anche se sono ai lati opposti del pianeta. Il **livello di trasporto converte i messaggi applicativi** in uno o più pacchetti di trasporto, detti **segmenti** o **datagram** (quest'ultimo termine soprattutto per i pacchetti UDP): se i messaggi sono spezzati in blocchi più piccoli, ogni blocco riceve un header di trasporto, e i segmenti sono passati uno a uno al livello di rete per la trasmissione.

Responsabilità del livello di trasporto

I protocolli di trasporto (UDP e TCP) hanno quattro responsabilità principali:

1. **Consegna processo-a-processo**: i messaggi sono consegnati indipendentemente da dove sono i processi; è diverso dalla consegna **host-a-host** (tra macchine), che è responsabilità del protocollo IP di livello inferiore.
2. **Controllo di integrità**: tramite campi di rilevamento errori negli header dei segmenti.
3. **Trasferimento affidabile dei dati**: garantire che i dati arrivino dal processo mittente a quello ricevente, corretti e in ordine.
4. **Controllo di congestione/flusso**: evita che una connessione saturi i dispositivi di rete (link o router) con troppo traffico; è utile per le prestazioni e **molto benefico per l'intera rete Internet**.

UDP (più veloce ma semplice) fornisce **solo i primi due servizi**; TCP fornisce tutti e quattro.

Consegna processo-a-processo

A livello applicazione **più processi possono accedere alla rete contemporaneamente** (ascoltare musica, scaricare file, navigare, tutti insieme), e un singolo processo può essere collegato a più processi contemporaneamente (es. proxy server, mail server). Per risolvere questi problemi si usano i socket per la consegna processo-a-processo: invece di consegnare il messaggio direttamente a un processo, **i socket sono gli endpoint da cui i processi prendono i messaggi**. Questo approccio **disaccoppia** il numero di processi dal numero di connessioni.

Poiché nell'host ricevente ci sono più socket, servono un **identificatore univoco** per ogni socket (la **porta**) da allegare ai segmenti ricevuti, e un processo di **multiplazione/demultiplazione** (*multiplexing/demultiplexing*) su mittente e ricevente. La **demultiplazione** reindirizza un segmento al socket giusto in base all'identificatore associato, mentre la **multiplazione** crea segmenti da processi diversi, assegnando l'identificatore giusto. Multiplazione e demultiplazione sono presenti anche in altri livelli, perché il problema di reindirizzare messaggi a più sorgenti è comune nelle reti.

Porte

Gli identificatori dei socket sono dette porte sorgente e destinazione: una **porta è un numero a 16 bit** da 0 a 65535, e le porte **da 0 a 1023 sono dette "well-known"** perché riservate ai protocolli applicativi noti. La **lista delle porte note** è aggiornata da **IANA** (*Internet Assigned Numbers Authority*); sviluppando una nuova applicazione di rete, occorre assegnare le porte ai socket (e quindi alle applicazioni) di conseguenza.

Porta	Uso
20	File Transfer Protocol (FTP) — trasferimento dati
21	FTP — controllo comandi
22	Secure Shell (SSH)
23	Telnet — accesso remoto, messaggi in chiaro
25	Simple Mail Transfer Protocol (SMTP) — instradamento e-mail
53	Domain Name System (DNS)
80	Hypertext Transfer Protocol (HTTP), Web
110	Post Office Protocol (POP3) — recupero e-mail dal server
119	Network News Transfer Protocol (NNTP)
123	Network Time Protocol (NTP)
143	Internet Mail Access Protocol (IMAP) — gestione posta
161	Simple Network Management Protocol (SNMP)
443	HTTP Secure (HTTPS), HTTP su TLS/SSL

Nmap

Per **controllare l'uso delle porte** (e altro) su Linux si può usare `nmap`: il *Network Mapper* è una **utility di scansione di rete** creata per mappare (scoprire) host e servizi su una rete. Può essere usata per sondare le porte degli host **per individuare quelle aperte** (su cui un'applicazione è in ascolto) ed eventualmente i servizi associati. Uso tipico:

- Port-scan di un bersaglio: `sudo nmap [indirizzo]`
- Sondare le porte per determinare i servizi: `sudo nmap -sV [indirizzo]`
- Controllare le N porte più usate: `sudo nmap --top-port N [indirizzo]`

Esempio:

```
$ nmap --top-port 10 www.google.com
PORT      STATE SERVICE
21/tcp    filtered ftp
22/tcp    filtered ssh
23/tcp    filtered telnet
25/tcp    filtered smtp
80/tcp    open  http
110/tcp   filtered pop3
139/tcp   filtered netbios-ssn
443/tcp   open  https
445/tcp   filtered microsoft-ds
3389/tcp  filtered ms-wbt-server
```

Lo stato può essere open o closed, filtered o unfiltered (non scansionabile).

Multiplazione e demultiplazione

Si consideri un processo sull'host A (porta 19157) che vuole inviare un messaggio a un processo (porta 46428) sull'host B. Nella **multiplazione**, il trasporto dell'host A crea un segmento/datagram che include i dati applicativi, la porta sorgente (19157) e la porta destinazione (46428), poi **passa il segmento al livello di rete**, che lo incapsula in un datagram IP e fa un tentativo best-effort di consegnarlo all'host ricevente. Nella **demultiplazione**, se il segmento arriva all'host B, **il trasporto lo riceve e lo decapsula**, quindi **passa il segmento al socket appropriato** esaminando la porta di destinazione.

In UDP (C/C++)

Client:

```

sockfd = socket(AF_INET, SOCK_DGRAM, 0);
// il socket tipicamente non è bound: il SO sceglie una porta libera (es. 46428),
// ma si può sempre fare bind su una porta specifica

servaddr.sin_port = htons(19157);
servaddr.sin_addr.s_addr = inet_addr(address_of_B);

sendto(sockfd, (const char *)msg, strlen(msg), 0,
        (const struct sockaddr *)&destaddr, sizeof(destaddr));

```

Server:

```

sockfd = socket(AF_INET, SOCK_DGRAM, 0);

servaddr.sin_addr.s_addr = INADDR_ANY;
servaddr.sin_port = htons(19157);

bind(sockfd, (const struct sockaddr *)&servaddr, sizeof(servaddr));

// porta/indirizzo di destinazione recuperati dalla struttura riempita da recvfrom
recvfrom(sockfd, (char *)msg, 1024, 0,
        (struct sockaddr *)&cliaddr, &len);

sendto(sockfd, (const char *)msg, strlen(msg), 0,
        (const struct sockaddr *)&cliaddr, sizeof(cliaddr));

```

La risposta viene rispedita **a qualunque client** sia. Un socket UDP si identifica con **due elementi**: indirizzo (IP) di destinazione e porta di destinazione. Si può quindi usare lo stesso socket per inviare messaggi di ritorno **a più porte/indirizzi sorgente diversi**: la struttura `cliaddr`, riempita da `recvfrom`, può essere usata come destinazione del `sendto`, indipendentemente da chi sia l'host.

In TCP (C/C++)

Il server ha un **welcoming socket**, in attesa di richieste di connessione su una porta specifica (es. 19157). Il client crea un socket e **invia un segmento di richiesta di connessione** all'host indicato nella struttura `sockaddr_in`:

```

sockfd = socket(AF_INET, SOCK_STREAM, 0);

servaddr.sin_port = htons(19157);
servaddr.sin_addr.s_addr = inet_addr(address_of_B);

connect(sockfd, (struct sockaddr *)&servaddr, sizeof(servaddr));

```

Una **richiesta di connessione** è solo un **segmento TCP** con porta di destinazione (19157) e il bit speciale di instaurazione della connessione attivo nell'header TCP (`SYN = 1`). Ricevuta la richiesta, il server **individua il processo server in attesa** di accettare la connessione e crea un nuovo socket:

```

new_socket = accept(sockfd, (struct sockaddr *)&cliaddr, (socklen_t *)&addrlen);

```

Il trasporto lato server riempie la struttura `cliaddr` **con porte e indirizzi del segmento in arrivo** e crea il nuovo socket; tutti i segmenti futuri con quella specifica porta sorgente, IP sorgente, porta destinazione e IP destinazione **saranno demultiplati verso new_socket**. Un socket TCP si identifica quindi con **quattro elementi**: IP sorgente, porta sorgente, IP destinazione, porta destinazione. Per via dell'handshake iniziale, la connessione TCP **lega ("intreccia") i processi** di entrambi i lati del socket: un solo mittente e un solo destinatario; se uno dei due lati si interrompe, il socket si chiude da entrambi i lati (cosa che non avviene in UDP).

Esempio HTTP

Si consideri un host C che apre **due sessioni HTTP verso il server B** e un host A che ne apre una verso B: A, C e B hanno ciascuno il proprio IP, e C assegna due porte sorgente diverse (26145 e 7532) alle sue due connessioni. A può assegnare la porta sorgente 26145 alla propria connessione senza sapere di C, e non è un problema: **le due connessioni hanno IP sorgente diversi**. Il server apre un nuovo processo (o thread) per

ogni connessione in arrivo, assegnandogli lo specifico socket (`new_socket`); è un approccio tipico, ma aprire troppi processi/thread può danneggiare le prestazioni del server.

UDP

UDP è utile per connessioni semplici e rapide e svolge principalmente due compiti: consegna processo-a-processo (porte, moltiplicazione/demultiplicazione) e controllo degli errori. UDP è senza connessione: **nessun handshake** tra le entità di trasporto prima di inviare un segmento, **nessuna garanzia di consegna** (nessun trasferimento affidabile) e **nessun controllo di congestione/flusso**. Grossolanamente, UDP è un protocollo **minimalista**: aggiunge solo “l’essenziale” rispetto al protocollo IP sottostante.

Perché usare UDP invece di TCP?

- **Controllo a livello applicazione:** UDP è più diretto, l’applicazione ha più controllo sulla trasmissione; utile nelle applicazioni real-time, dove velocità di invio e ritardi sono cruciali e la perdita di dati è tollerabile.
- **Stabilimento rapido della connessione:** niente handshake, meno ritardo; bene per il DNS, per non aggiungere ritardi.
- **Nessuno stato di connessione:** niente parametri di controllo di congestione, niente numeri di sequenza o acknowledgment; un server può sostenere molti più client attivi su UDP che su TCP.
- **Overhead di pacchetto minimo:** l’header TCP aggiunge 20 byte per segmento, UDP solo 8.

Esempio: DNS su UDP

Il client DNS funziona così: a **livello applicazione** l’host costruisce il messaggio di query DNS e lo passa a UDP; a **livello di trasporto**, senza handshake, UDP aggiunge gli header e passa il segmento al livello di rete; a **livello di rete** il segmento UDP è incapsulato in un datagram IP e inviato al server DNS (attraverso il livello di accesso). Il client attende la risposta e, se **non arriva** (la rete ha perso query o risposta), può reinviare la query, inviarla a un altro name server, o informare l’applicazione che la risposta non è arrivata.

Uso di UDP

Le applicazioni che necessitano **trasferimento affidabile** (e-mail, terminale remoto, Web) girano su TCP (o su protocolli affidabili basati su UDP come QUIC), perché non ci si può permettere perdite: le mail, ad esempio, vanno consegnate per intero. **SNMP** usa UDP per la gestione dei dispositivi, poiché le applicazioni di network management **devono spesso girare quando la rete è sotto stress**, proprio quando il trasferimento affidabile è difficile. Le **applicazioni multimediali** (telefonia Internet, videoconferenza, streaming) usano spesso sia UDP che TCP, perché si può tollerare una piccola perdita di pacchetti; in generale le applicazioni real-time reagiscono male al controllo di congestione di TCP.

Servizio	Protocollo applicativo	Protocollo di trasporto
E-mail	SMTP	TCP
Terminale remoto	Telnet	TCP
Web	HTTP	TCP o UDP
Trasferimento file	FTP	TCP
Traduzione nomi	DNS	UDP
Gestione dispositivi	SNMP	UDP
Streaming multimediale	Proprietari	UDP o TCP
Telefonia Internet	Proprietari	UDP o TCP

Controversia

Far girare applicazioni multimediali su UDP è **controverso**, perché non c’è controllo di congestione. Se tutti (egoisticamente) iniziassero a trasmettere video ad alto bit-rate senza controllo di congestione, **i dispositivi di rete andrebbero in overflow** con più pacchetti UDP persi; gli alti tassi di perdita indotti dai sender UDP fuori controllo farebbero anche **calare drasticamente le velocità dei sender TCP**.

Formato del segmento UDP

Il **segmento UDP** (più precisamente, datagram) è composto da due parti: un **header UDP** (64 bit), con **4 campi da 16 bit**, e i **dati** (N bit), cioè il messaggio applicativo. I campi dell'header sono:

- **Porta sorgente**: numero di porta del processo mittente.
- **Porta destinazione**: numero di porta del processo ricevente.
- **Lunghezza**: lunghezza dell'intero datagram (header+dati, cioè $N + 64$ bit).
- **Checksum**: usato dal ricevente per verificare l'integrità del messaggio.

Checksum UDP

Il controllo di integrità usa il **checksum**, campo a 16 bit per il rilevamento degli errori. Il lato mittente calcola il **complemento a 1 della somma di tutte le parole a 16 bit** del datagram, con eventuali bit di overflow sommati a loro volta, e il **risultato è messo nel campo checksum**. Quando **il ricevente somma tutti i bit del messaggio (checksum incluso)**, la somma deve dare 1111111111111111 (16 uni); se anche un solo bit è 0, si sa che **sono stati introdotti errori** nel pacchetto.

Lezione 11: Trasporto affidabile dei dati

Il problema del trasferimento affidabile

Si pensi all'annuncio di una stazione: “Il treno 6 arriverà al binario 9”. Se la comunicazione non è affidabile, le parole possono essere **perse** (“Il treno %&! arriverà al binario 9”), **alterate** (“Il treno 7 arriverà al binario 9”), **scambiate** (“Il treno 9 arriverà al binario 6”) o **duplicate** (“Il treno 66 arriverà al binario 9”). Potreste accorgervi che il messaggio è sbagliato, ma potreste anche prendere il treno sbagliato o perdere quello giusto.

Il controllo degli errori (es. via checksum) permette al ricevente almeno di capire **se il messaggio è corrotto**, ma non basta: il trasferimento affidabile **resta uno dei problemi principali del networking**, affrontato non solo al livello di trasporto, ma anche al livello di collegamento e (spesso) applicazione. In un canale affidabile si devono garantire tre cose:

1. Nessun bit trasmesso è **corrotto** (invertito da 0 a 1 o vice versa).
2. Nessun bit trasmesso è **perso o ripetuto**.
3. Tutti i bit sono consegnati **nell'ordine esatto** di invio.

In generale si deve assumere che il livello di rete sottostante **non è affidabile**: un'assunzione realistica, dato che ad esempio TCP è un protocollo affidabile **implementato sopra un livello di rete (IP) end-to-end non affidabile**.

Corruzione dei pacchetti e Stop-and-wait

Si assuma un **canale in cui i bit possono solo corrompersi** (non perdersi): la corruzione è un problema tipico, dovuto principalmente ai componenti fisici della rete, poiché un pacchetto viene trasmesso, propagato e bufferizzato più volte durante la comunicazione. Un primo approccio è lo **stop-and-wait**, che richiede che ogni messaggio sia confermato prima di inviarne uno nuovo, tramite **acknowledgment positivo (ACK)** — il messaggio è arrivato intatto — o **acknowledgment negativo (NCK)** — c'è stato un errore, il messaggio va ripetuto. I protocolli basati su ritrasmissione sono detti anche **ARQ** (*Automatic Repeat reQuest*).

Problemi di stop-and-wait

Restano due domande: cosa succede se **il messaggio ACK/NCK stesso è corrotto**, e come può l'host B essere sicuro che **un messaggio è una ripetizione** anziché uno nuovo? La soluzione è il **numero di sequenza**: aggiungere agli header un campo che specifica l'ordine di ricezione dei pacchetti. Nello stop-and-wait c'è sempre **un solo messaggio** in linea, quindi basta **distinguere il pacchetto attuale dal precedente**: basta un solo bit ($s = 0/1$).

Perdita di pacchetti e timeout

Si assuma ora un **canale in cui i pacchetti possono anche perdersi** — un'assunzione ragionevole, dato che i dispositivi di rete possono andare in **buffer overflow** con traffico intenso. Il problema è evidente: **se un messaggio si perde, gli host non invieranno mai quello nuovo**, creando un loop. Vale sia perdendo il messaggio che l'acknowledgment: in entrambi i casi l'host A non viene “svegliato” per inviare.

La soluzione è semplice ed efficace: **aggiungere un timeout lato mittente** che, scaduto, gli permette di riprovare. Un approccio simile è usato nel DNS basato su UDP; il timeout **funziona sia per perdita del pacchetto che dell'acknowledgment**, e in caso di duplicato il ricevente si limita a scartare il pacchetto. La sfida è **come stimare un timeout adeguato** (dipende dall'RTT): troppo lungo la comunicazione rallenta, troppo corto i pacchetti si sovrappongono; un timeout ragionevole dev'essere in qualche modo più lungo dell'RTT. Il timeout salva lo stop-and-wait dal loop, ma **le prestazioni restano piuttosto scarse**.

Analogia con l'acqua

Il trasferimento dati si può vedere come un flusso d'acqua: c'è un tempo t per versare l'acqua dal serbatoio al tubo, dopo il quale tutta l'acqua è nel tubo (serbatoio di partenza e destinazione vuoti). L'acqua impiega un tempo $(RTT/2)$ a scorrere nel tubo, infine arriva al serbatoio di destinazione e — con tempi di upload/download uguali — impiega lo stesso tempo t per uscire $(RTT/2 + t)$ per ricevere l'ultima goccia).

Prestazioni dello stop-and-wait

Si consideri il caso idealizzato di **due host sulle coste opposte degli USA**: RTT alla velocità della luce di circa 30 ms (0,03 s), canale a velocità $R = 1$ Gbps (10^9 bit/s), pacchetto di $L = 1000$ byte (8000 bit). Il tempo t per trasmettere il pacchetto è $t = L/R = 8000/10^9 = 8$ **microsecondi**.

In **stop-and-wait**: il mittente **inizia a inviare** al tempo t_0 ; l'**ultimo bit entra nel canale** a $t_0 + 0,000008$ s; il pacchetto impiega **0,015 s per arrivare** al ricevente; l'**ultimo bit è ricevuto** a $t = RTT/2 + L/R = 0,015008$ s; assumendo gli ACK molto piccoli (tempo di trasmissione trascurabile), l'ACK torna al mittente a $t = RTT + L/R = 0,030008$ s. Su 0,030008 s totali, il mittente ha atteso quasi tutto il tempo (**99,973%**).

Pipelining

Con il **pipelining**, invece dello stop-and-wait, al mittente è **consentito inviare più pacchetti** senza attendere gli acknowledgment. Con 3 pacchetti invece di 1: il mittente inizia a t_0 ; l'**ultimo bit dell'ultimo pacchetto entra** a $t_0 + 0,000024$ s; l'**ultimo bit è ricevuto** a $t = RTT/2 + L/R = 0,015024$ s; l'ACK torna a $t = RTT + L/R = 0,030024$ s. In 0,030024 s il mittente ha atteso un po' meno (**99,920%** invece di 99,973%).

Nel **pipelining** l'**intervallo dei numeri di sequenza va aumentato** — ogni pacchetto in volo (escluse le ritrasmissioni) deve avere un numero di sequenza univoco e possono esserci più pacchetti in volo non confermati — e servono **buffer per memorizzare i pacchetti in arrivo**, perché non si sa se sono corretti o se ci sono “buchi” nella trasmissione. I due protocolli base con **pipelining** sono **Go-Back-N** e **Selective Repeat**.

Go-Back-N (GBN)

Il protocollo **Go-Back-N** (detto anche *sliding window*) permette al mittente di trasmettere più pacchetti **senza attendere acknowledgment**, ma con il vincolo di **non più di N pacchetti non confermati**. Le variabili chiave sono **base**, il numero di sequenza del **pacchetto più vecchio non confermato**, e **nextseqnum**, il più piccolo numero di sequenza **non ancora usato** (prossimo da inviare). Allora:

- I pacchetti in $[0, \text{base} - 1]$ sono stati **trasmessi e confermati**.
- I pacchetti in $[\text{base}, \text{nextseqnum} - 1]$ sono **inviati ma non confermati**.
- I pacchetti in $[\text{nextseqnum}, \text{base} + N - 1]$ **possono essere inviati**.
- I pacchetti in $[\text{base} + N, +\infty)$ **non possono essere usati** finché non arriva un nuovo acknowledgment.

Il ricevente in GBN

In GBN il **ricevente è molto semplice**: deve **semplicemente scartare i pacchetti fuori ordine** (che siano corrotti o no) e consegnare al livello superiore solo i pacchetti in ordine; i pacchetti scartati (non confermati) saranno prima o poi ritrasmessi dal mittente. Con questo approccio anche pacchetti buoni vengono scartati, ma il processo resta semplice: il **mittente mantiene gli indici della finestra**, mentre il **ricevente deve mantenere solo il numero di sequenza del prossimo pacchetto in ordine**.

Lo svantaggio è che un pacchetto ricevuto correttamente va **rispedito comunque**: è una **reazione a catena** — se si perdono pacchetti per difficoltà di rete, molti pacchetti corretti ma fuori ordine vengono scartati, forzando il mittente a ritrasmetterli, e la ritrasmissione stessa può perdersi o corrompersi, richiedendo altre ritrasmissioni. Ad esempio, se si perde pkt2, anche pkt3, pkt4 e pkt5 vanno scartati e ritrasmessi, anche se corretti.

Selective Repeat (SR)

GBN è più efficace dello stop-and-wait, ma può **soffrire di problemi di prestazioni**: con finestre grandi, **un solo errore può causare la ritrasmissione di molti pacchetti**, molti inutilmente. Nei protocolli **selective-repeat (SR)** il mittente ritrasmette **solo i pacchetti presumibilmente ricevuti con errore** (persi o corrotti): come in GBN, i singoli pacchetti sono confermati e si usa di nuovo **una finestra di N** per limitare i pacchetti non confermati; diversamente da GBN, però, **i pacchetti fuori ordine vengono bufferizzati** (memorizzati) e confermati dal ricevente.

Ad esempio, con pacchetti persi: il ricevente inizialmente bufferizza pkt3, pkt4 e pkt5 in attesa che pkt2 (perso) sia ritrasmesso, evitando così di rispedire pkt3, pkt4 e pkt5, ed evitando ulteriori perdite o errori di trasmissione.

Limite della finestra in SR

Resta un problema: la dimensione della finestra è legata al numero di sequenza, **che è finito**. Si considerino 4 pacchetti, **numero di sequenza massimo 3**, finestra di dimensione 3; i pacchetti da 0 a 2 sono ricevuti correttamente e gli ACK inviati (ma non ancora ricevuti). La **finestra del ricevente si sposta sul 6° pacchetto** (cioè su [3, 0, 1]) anche se gli ACK non sono stati ricevuti. Nel **caso a** gli ACK dei primi tre pacchetti si perdono e il mittente li ritrasmette: il pacchetto 0 è nuovo o vecchio? Nel **caso b** gli ACK arrivano, si inviano i pacchetti 3 e 0 ma il 3 si perde: il pacchetto 0 è nuovo o vecchio? Per evitare il problema, **lo spazio dei numeri di sequenza dev'essere almeno il doppio della dimensione della finestra**.

Lezione 12: Il livello di trasporto — TCP

TCP rispetto a UDP

Rispetto a UDP, TCP è **orientato alla connessione** (c'è una fase di “handshake” prima della trasmissione, che garantisce che i due processi siano “connessi”), **affidabile** (implementa rilevamento degli errori, ritrasmissioni, acknowledgment, timer e numeri di sequenza) e **sensibile a congestione e flusso** (c'è una regolazione della velocità di trasmissione in base alle prestazioni del ricevente — flusso — e della rete — congestione).

L'orientamento alla connessione rende TCP **full-duplex** (se A è connesso con B, allora B è connesso con A) e **punto-a-punto**: un solo mittente e un solo ricevente, quindi il trasferimento da un mittente a più riceventi non è possibile (UDP invece permette il multicasting). TCP è inoltre orientato all'invio/ricezione di **flussi di dati** (*data stream*): più segmenti TCP possono far parte di un flusso più grande (sequenza ordinata di dati), mentre i singoli datagram UDP sono considerati per lo più disaccoppiati.

Buffer TCP

Nella connessione TCP, invio e ricezione si basano fortemente sui **buffer**. I buffer permettono di **disaccoppiare parzialmente** i tempi di trasmissione dai **ritardi applicativi**, dai **ritardi del SO** nella moltiplicazione/demoltiplicazione dei pacchetti e dai **ritardi di rete** (oscillazioni delle prestazioni). C'è anche il limite della velocità di trasmissione: messaggi lunghi possono **essere spezzati in segmenti più piccoli**, raccolti nei buffer prima di essere disassemblati (lato mittente) o riassemblati (lato ricevente).

Maximum Segment Size (MSS)

La quantità di **dati dentro un segmento** è limitata dalla **Maximum Segment Size (MSS)**:

$$MSS + 40 = MTU$$

dove la **Maximum Transmission Unit (MTU)** è la lunghezza massima di un frame accettabile dal livello di collegamento (es. in Ethernet 1500 byte) e **40 byte** è la dimensione combinata degli header TCP e IP (tipicamente 20+20 byte). **In invio**, i segmenti sono creati dai dati applicativi e passati al livello di rete, dove sono incapsulati separatamente in datagram IP; **in ricezione**, i dati sono messi nel buffer di ricezione e l'applicazione preleva i dati quando è pronta.

Il segmento TCP

Il segmento TCP consiste di **campi di header** e un **campo dati** (al più MSS byte). L'**header** contiene:

- **Numero di sequenza** (32 bit): per il trasferimento affidabile.
- **Numero di acknowledgment** (32 bit): per il trasferimento affidabile.
- **Finestra di ricezione** (16 bit): indica quanti byte il ricevente è disposto ad accettare (per il controllo di flusso).
- **Lunghezza dell'header** (4 bit): numero di parole da 32 bit nell'header; l'header TCP è **variabile a causa del campo opzioni**.
- **Campo opzioni** (K bit): processi opzionali come la negoziazione dell'MSS, il timestamping, ecc.
- **Flag** (6/12 bit):
 - **ACK**: il segmento è un pacchetto ACK (il campo acknowledgment è in uso).
 - **RST, SYN, FIN**: usati per instaurazione e chiusura della connessione.
 - **CWR, ECE**: usati nella notifica esplicita di congestione (opzionale).
 - **PSH**: dice al ricevente di passare subito i dati all'applicazione (raro).

- **URG**: il segmento contiene dati “urgenti” (raro).
- **Checksum** (16 bit): controllo di integrità.
- **Puntatore ai dati urgenti** (16 bit): posizione dell’ultimo byte della parte urgente dei dati (raro).

Numeri di sequenza e acknowledgment

I numeri di sequenza servono non solo per il trasferimento affidabile, **ma anche per gestire la segmentazione**: un flusso grande va **spezzato in “pezzi”** in base all’MSS, e ogni pezzo va in un segmento TCP. Sequenza e acknowledgment sono **strettamente collegati**: il **numero di sequenza** di un segmento è convenzionalmente **la posizione che il primo byte dei dati del segmento occupa nel flusso**, mentre il **numero di acknowledgment** è convenzionalmente **il prossimo pezzo del flusso atteso** (calcolato dal numero di sequenza del segmento precedente).

TCP è un protocollo general-purpose e non sa nulla dei dati trasmessi: dal punto di vista di TCP i dati sono **solo un flusso ordinato di byte**, quindi il numero di sequenza è il **numero, nel flusso di byte, del primo byte dei dati** del segmento.

Esempio

Si consideri un flusso di dati di **500.000 byte** (~500 KB) con **MSS di 1.000 byte**: TCP costruisce **500 segmenti** — il primo con numero di sequenza 0, il secondo 1000, il terzo 2000, e così via. Quando il flusso è passato **dall’host A all’host B** (partendo dalla sequenza 0), il ricevente riceve il primo segmento di 1.000 byte, cioè **dalla sequenza 0 alla 999**, e conferma impostando **acknowledgment = 1.000** (prossimo byte atteso nel flusso); riceve poi il segmento successivo (sequenze 1.000–1.999), e così via.

Dati in entrambe le direzioni

E se il ricevente a sua volta invia dati? TCP è **full-duplex**: se A e B comunicano ci sono **due flussi** da considerare, **A→B** e **B→A**, e si usano **sequenza e acknowledgment contemporaneamente**. I segmenti di A hanno numeri di **sequenza relativi al flusso A→B** e di **acknowledgment relativi al flusso B→A**; i segmenti di B hanno sequenza relativa al flusso **B→A** e acknowledgment relativo al flusso **A→B**.

Esempio semplificato (echo)

Si considerino due messaggi da **1 solo byte** (1 carattere) da A a B e viceversa: A trasmette una ‘c’ che B **rimanda indietro (echo)**, e i segmenti forniscono **ACK e DATA insieme** (flag ACK = 1).

1. **Da A a B**: ack number = seq del prossimo pacchetto atteso nel flusso B→A (byte 79); seq number = posizione dell’unico byte trasmesso (byte 42).
2. **Da B ad A**: ack number = seq del prossimo pacchetto atteso nel flusso A→B (cioè seq + 1 byte del carattere ‘c’); seq number = posizione del byte trasmesso (byte 79).
3. **Da A a B**: segmento di **puro ACK**, con seq 43 (come atteso da B) e ack 80 (prossimo atteso).

Retransmission Timeout

Il meccanismo di trasferimento affidabile di TCP **fa un uso massiccio dei timeout**. Nella trasmissione TCP basata su pipelining, l’approccio di default è **ritrasmettere i segmenti solo se gli ACK associati non arrivano prima del timeout**, quindi la **stima del timeout è critica**: un timeout troppo lungo rallenta la comunicazione, uno troppo corto fa sovrapporre i pacchetti (e pacchetti inutili vengono ritrasmessi, generando traffico inutile).

Stima dell’RTT

Per impostare i timeout serve una **stima del round-trip time (RTT)**: il timeout dev’essere **maggiore dell’RTT**, altrimenti si invierebbero ritrasmissioni inutili. TCP stima l’RTT **campionando (SampleRTT) i segmenti confermati con successo e non ritrasmessi (one-shot)**. Poiché il SampleRTT **può oscillare** (congestione, carico del ricevente, ecc.), la stima (EstimatedRTT) è una **media mobile esponenziale (EWMA)**:

$$\text{EstimatedRTT} = (1 - \alpha) \cdot \text{EstimatedRTT} + \alpha \cdot \text{SampleRTT}$$

con α solitamente 0,125 (cioè 1/8). Si può considerare anche la **variabilità dell'RTT** (DevRTT), data dalla differenza tra campione e stima:

$$\text{DevRTT} = (1 - \beta) \cdot \text{DevRTT} + \beta \cdot |\text{SampleRTT} - \text{EstimatedRTT}|$$

con β solitamente 0,25 (cioè 1/4).

Timeout di ritrasmissione

Con queste stime si definisce un timeout né troppo basso né troppo alto rispetto all'RTT stimato:

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \cdot \text{DevRTT}$$

La deviazione dell'RTT dà un **marginе ragionevole e adattivo**: cresce quando l'RTT oscilla, quindi la finestra di timeout è più larga **quando siamo incerti** sull'RTT corrente. Di default, il **valore iniziale** dell'RTT (a t=0) è 1 secondo.

Fast Retransmit

La ritrasmissione su timeout è efficace, ma **il periodo di timeout può essere relativamente lungo**. Il **fast retransmit** è un approccio in cui il ricevente segnala al mittente che un pacchetto potrebbe essere perso inviando **ACK duplicati**: quando il ricevente riceve un segmento con **numero di sequenza maggiore di quello atteso**, ragionevolmente un segmento è andato perso, quindi **reinvia il vecchio ACK** (duplicato), con numero di acknowledgment pari a quello del segmento atteso. Se **il mittente riceve N duplicati** (tipicamente 3), assume che il segmento precedente sia perso e lo ritrasmette (fast) **ben prima della scadenza**.

Esempio

Si consideri un segmento perso con sequenza 100: ogni volta che B riceve un segmento fuori ordine, **rimanda un ACK duplicato** (ack=100); quando A **riceve 3 duplicati**, assume perso il segmento 100 e **lo rispedisce** (fast retransmit); quando B riceve il segmento perso, **invia l'ACK del prossimo segmento atteso**, secondo la sua politica (go-back-N o selective repeat).

Perché non ritrasmettere al primo ACK duplicato? Perché **possiamo ricevere ACK duplicati per motivi vari**, non solo perdite: ad esempio, se **2 segmenti arrivano in ordine invertito** (scambiati in trasmissione), il ricevente invia **1 ACK duplicato** (per il primo segmento) prima di accorgersi dello scambio.

Lezione 13: TCP — gestione della connessione

Richiamo: responsabilità del livello di trasporto

I protocolli di trasporto (UDP e TCP) hanno quattro responsabilità principali: **consegna processo-a-processo** (i messaggi sono consegnati indipendentemente da dove sono i processi, diversamente dalla consegna host-a-host, responsabilità del protocollo IP di livello inferiore), **controllo di integrità** (campi di rilevamento errori negli header dei segmenti), **trasferimento affidabile** (dati consegnati correttamente e in ordine) e **controllo di congestione/flusso** (evita di saturare i dispositivi di rete; migliora le prestazioni ed è molto benefico per l'intera rete Internet). UDP fornisce solo i primi due servizi, TCP tutti e quattro.

Richiamo: il segmento TCP

Il segmento TCP consiste di **campi di header** e un **campo dati** (al più MSS byte). L'**header** contiene:

- **Numero di sequenza** (32 bit) e **numero di acknowledgment** (32 bit), per il trasferimento affidabile.
- **Finestra di ricezione** (16 bit): byte che il ricevente è disposto ad accettare (controllo di flusso).
- **Lunghezza dell'header** (4 bit): numero di parole da 32 bit; l'header è variabile a causa delle opzioni.
- **Campo opzioni** (K bit): negoziazione dell'MSS, timestamping, ecc.
- **Flag** (6/12 bit): **ACK** (il segmento è un ACK), **RST**, **SYN**, **FIN** (instaurazione e chiusura della connessione), **CWR**, **ECE** (notifica esplicita di congestione, opzionale), **PSH** (passare subito i dati all'applicazione), **URG** (dati "urgenti").
- **Checksum** (16 bit): integrità.
- **Puntatore ai dati urgenti** (16 bit).

Problemi di gestione della connessione TCP

Essendo orientato alla connessione, **due host devono accordarsi** sia nell'apertura sia nella chiusura. Poiché i messaggi possono **perdersi o danneggiarsi**, trovare questo accordo è piuttosto difficile: se i messaggi si perdono, **un capo della comunicazione può essere aperto/chiuso mentre l'altro no**. Per mitigare il problema TCP implementa il **three-way handshake**: le richieste di apertura/chiusura **devono essere confermate** prima che la connessione sia stabilita o rilasciata.

Il problema dei due eserciti (two-army problem)

Si immagini un **esercito bianco accampato in una valle** e, sulle due colline, **due eserciti blu nemici**. L'esercito bianco è più grande di ciascun esercito blu, ma insieme i blu sono più forti: **vinceranno solo attaccando contemporaneamente**. Per sincronizzarsi, i blu devono **inviare messaggeri attraverso la valle**, dove possono essere catturati (comunicazione non affidabile).

Esiste un protocollo che permetta ai blu di vincere? Il comandante dell'esercito blu #1 manda: "Propongo di attaccare oggi, va bene?". Il messaggio arriva, il comandante #2 è d'accordo e la **risposta torna sana** al #1. L'attacco avverrà? Probabilmente no, perché **il comandante #2 non sa se la sua risposta è arrivata**: se non è arrivata, il #1 non attaccherà. Con un **three-way handshake** il primo comandante deve **confermare** la risposta, ma allora sarà il #1 a esitare (non sa se la conferma è arrivata); un four-way handshake non aiuterebbe. Si può dimostrare che **non esiste un protocollo che funzioni**: il **three-way handshake non è perfetto, ma di solito è adeguato**.

Instaurazione della connessione TCP

In TCP si esegue il **three-way handshake** per stabilire la connessione: il client invia una **richiesta di connessione** (segmento SYN) al server, che risponde con uno **speciale acknowledgment** (segmento SYN-ACK); infine il client rimanda un **acknowledgment finale** (segmento ACK). L'instaurazione è una **procedura delicata** che può aggiungere ritardi significativi: c'è tipicamente un **timeout** (30-60 s) per completare l'handshake, dopo il quale la procedura è abortita, ed è proprio qui che avvengono alcuni **attacchi** (es. SYN flood).

Dettagli del three-way handshake

1. Il client invia un **segmento SYN** con nessun dato applicativo (payload), bit SYN = 1 e un numero di sequenza iniziale casuale (`client_isn`).
2. Arrivato il SYN, il **server alloca buffer e variabili TCP** e invia un **segmento SYNACK** con nessun dato applicativo, bit SYN = 1 e ACK = 1, acknowledgment number = `client_isn + 1` e numero di sequenza iniziale casuale (`server_isn`).
3. Arrivato il SYNACK, anche il **client alloca buffer e variabili** e invia il **segmento ACK** finale, con eventuali dati applicativi, SYN = 0 (connessione stabilita) e ACK = 1, acknowledgment number = `server_isn + 1`.

Rilascio della connessione TCP

Il rilascio (detto **teardown**) può essere avviato da ciascuno dei due host. Se chiude il client, l'handshake è:

1. Il client invia un **segmento speciale di chiusura** (segmento FIN) con bit FIN = 1.
2. Il server risponde con un segmento di **acknowledgment/chiusura**, con ACK = 1 e FIN = 1; ACK e FIN possono essere inviati nello stesso segmento o in due separati.
3. Infine il client **conferma** il segmento di chiusura del server.

Gestione delle perdite nel rilascio

Poiché il three-way handshake non è perfetto, TCP si affida ai **timer** per chiudere prima o poi le connessioni o reinviare le richieste (ad esempio nella fase di rilascio della connessione).

Lezione 14: TCP — controllo di flusso e di congestione

Il problema della congestione e del controllo di flusso

Il controllo di congestione e di flusso sono funzionalità aggiuntive di TCP rispetto a UDP. La perdita di pacchetti è **spesso il risultato dell'overflow dei buffer**: dal buffer del ricevente, se l'**applicazione ricevente non è abbastanza veloce** a leggere i dati, o dai dispositivi di rete (es. router), se **i nodi sono congestionati**. Nell'analogia dell'acqua, **i buffer sono secchi intermedi** che traboccano se l'acqua eccede. Se i pacchetti si perdono, **gli host devono ricorrere a ritrasmissioni e timeout**, che danneggiano drasticamente le prestazioni della rete.

Controllo di flusso

Quando la connessione TCP riceve byte corretti e in sequenza, li mette nel buffer di ricezione; l'applicazione **leggerà i dati da questo buffer, ma non necessariamente nell'istante in cui arrivano** (può essere occupata). Se l'applicazione è **lenta nel leggere**, il mittente può facilmente **far traboccare il buffer del ricevente** inviando troppi dati troppo in fretta. Il **controllo di flusso TCP è un servizio di adattamento della velocità**: cerca di far combaciare (ridurre) la velocità di invio del mittente con quella di lettura dell'applicazione ricevente, e per farlo si affida a una **finestra di ricezione** che informa il mittente di quanti segmenti possiamo ricevere senza overflow: il campo **receive window** del segmento TCP dice al mittente **quanto buffer libero** rimane.

Meccanica

Per semplicità si assume che il ricevente **scarti i segmenti fuori ordine** (i segmenti nel buffer sono ordinati). Lato **ricevente** (host B) si considerano `RcvBuffer` (dimensione del buffer di ricezione, in byte), `LastByteRead` (ultimo byte del flusso letto dall'applicazione) e `LastByteRcvd` (ultimo byte del flusso ricevuto); la **finestra di ricezione** vale:

$$\text{rwnd} = \text{RcvBuffer} - (\text{LastByteRcvd} - \text{LastByteRead})$$

Lato **mittente** (host A) si considerano l'ultimo byte del flusso inviato (`LastByteSent`) e l'ultimo byte confermato dal ricevente (`LastByteAcked`). Conoscendo la finestra di ricezione, **il mittente garantisce in ogni istante**:

$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{rwnd}$$

Il problema della congestione

Il **controllo di congestione** è simile al controllo di flusso, ma riguarda l'infrastruttura di rete. Si considerino due host (A e B) con una connessione che **condivide un singolo router** e un singolo link in uscita di capacità **R** tra sorgente e destinazione; il **router ha buffer** per memorizzare i pacchetti in arrivo quando la velocità di arrivo supera la capacità del link in uscita. Si assuma che entrambe le applicazioni inviino dati a velocità media pari di λ_{in} byte/s.

Vista semplificata

Se $\lambda_{in} < R/2$, tutto ciò che è inviato arriva al destinatario **con un ritardo finito**; se $\lambda_{in} = R/2$, **il link raggiunge la piena capacità (R)** e i pacchetti in eccesso finiscono nel buffer. Oltre la capacità massima **i pacchetti si accumulano nel buffer del router** in attesa del proprio turno ma, poiché **il buffer è finito**, i pacchetti accumulati **prima o poi vengono scartati**, e gli host devono ritrasmetterli (generando ancora più traffico). Si noti che **un buffer infinito non risolve**: i pacchetti non si perderebbero, ma **il ritardo**

crescerebbe costantemente — eccedendo per sempre la capacità, il ritardo tende all'infinito e i pacchetti sono come persi.

Controllo di congestione

Esistono due approcci principali:

1. **Congestion control end-to-end** (approccio standard di TCP): la congestione è **dedotta dai sistemi finali** basandosi solo sul comportamento osservato della rete (perdita di pacchetti, ritardi); la perdita di segmenti (per timeout o per la ricezione di 3 ACK duplicati) è interpretata come segnale di congestione, e TCP riduce la velocità di invio.
2. **Congestion control assistito dalla rete** (approccio recente e opzionale): il trasporto lavora in sinergia col livello di rete, e **i router forniscono feedback espliciti** al mittente/ricevente sullo stato di congestione, usando i flag **CWR** ed **ECE**; il feedback può essere semplice come **un singolo bit che indica congestione** su un link, ma è possibile anche un feedback più sofisticato.

Il controllo di congestione in TCP

TCP si basa sul controllo **end-to-end**: **ciascun mittente adatta la propria velocità di invio alla congestione percepita della rete**. I tre problemi principali sono la **regolazione della velocità** (come un mittente TCP regola la velocità con cui invia traffico nella connessione), il **rilevamento della congestione** (come un mittente TCP rileva la congestione sul cammino verso la destinazione) e l'**adeguamento della velocità** (quale algoritmo usare per cambiare la velocità in funzione della congestione end-to-end percepita).

Regolazione della velocità

Nel controllo di flusso la velocità è regolata considerando **lo spazio libero nel buffer del ricevente** (finestra di ricezione `rwnd`), con $\text{LastByteSent} - \text{LastByteAcked} \leq \text{rwnd}$. Nel controllo di congestione il mittente tiene traccia anche di una **finestra di congestione** (`cwnd`), e la velocità è regolata aumentando/diminuendo `cwnd`:

$$\text{LastByteSent} - \text{LastByteAcked} \leq \min\{\text{cwnd}, \text{rwnd}\}$$

Rilevamento della congestione

Un mittente TCP percepisce la congestione controllando i **loss event** (eventi di perdita): un loss event avviene **quando scade un timeout o arrivano 3 ACK duplicati/errati**, e in entrambi i casi significa che un pacchetto precedente **non è arrivato a destinazione**, probabilmente per overflow dei dispositivi di rete. Se **non avvengono loss event** (gli ACK arrivano come attesi), TCP assume che la rete non sia congestionata e **la finestra di congestione può aumentare**; se **avvengono loss event**, **la finestra di congestione va diminuita**.

Adeguamento della velocità

L'adeguamento avviene tramite **bandwidth probing**: la velocità cresce finché gli ACK arrivano correttamente (si “sonda” la rete), e rilevata la congestione (loss event) la velocità si riduce; il processo si ripete continuamente. TCP usa l'**algoritmo di congestione di Jacobson** [Jacobson 1988], con tre fasi:

1. **Slow start**: si parte da 1 MSS/RTT e la velocità cresce **esponenzialmente**.
2. **Additive Increase** (congestion avoidance): la velocità cresce **linearmente**.
3. **Fast Recovery** (opzionale): **dimezza** la velocità invece di ripartire in slow start, e prosegue con incrementi additivi.

Il comportamento tipico dell'algoritmo di Jacobson è a **“dente di sega”** (*sawtooth*).

Lezione 15: Il livello di rete

Dal trasporto alla rete

Il compito del livello di rete è permettere a host remoti di comunicare: l'**host mittente** prende i segmenti dal livello di trasporto, li **incapsula in datagram** e li invia in rete, mentre l'**host ricevente** riceve i datagram, **ne estrae i segmenti di trasporto** e li passa al livello superiore. A questo livello lavorano alcuni dei protocolli più importanti (**IP, DHCP, NAT**, ecc.) e dispositivi dedicati, i **router**. Dentro la rete ci sono **nodi che inoltrano i datagram ai nodi adiacenti** (router) fino all'host di destinazione; i router hanno uno **stack di protocolli troncato** e potenzialmente non eseguono né applicazioni né protocolli di trasporto. Il reindirizzamento di un datagram da parte di un router si chiama **hop**.

Il livello di rete in Internet

Il livello di rete è responsabile della **consegna host-a-host**. I servizi che si potrebbero offrire insieme alla consegna sono: *consegna garantita* (un pacchetto inviato prima o poi arriva a destinazione), *consegna garantita con ritardo limitato* (entro un limite, es. 100 ms), *consegna in ordine*, *banda minima garantita* (possibilità di specificare un bit rate minimo tale che, se la velocità del mittente resta entro quel limite, tutti i pacchetti sono consegnati) e *sicurezza* (cifratura/decifratura dei datagram alla sorgente/destinazione).

In realtà **nessuno di questi servizi** è generalmente offerto dalle reti: il livello di rete di Internet fornisce **un solo servizio**, il **best-effort**, con cui la rete **fa del suo meglio** per consegnare un pacchetto dalla sorgente alla destinazione, senza garanzia di ordine né di consegna, e senza garanzie su ritardi o banda minima. Nonostante la semplicità, il best-effort **combinato con buona banda si è dimostrato adeguato**: Netflix, voice/video-over-IP, videoconferenze in tempo reale e Matrix funzionano tutti così.

I router

Il ruolo primario del livello di rete è la **consegna host-a-host**, svolta dai **router**: nodi speciali con **più link in ingresso/uscita**, che forniscono due funzioni. Il **forwarding** consiste nello spostare, quando un pacchetto arriva sul link di input, il pacchetto sul link di output appropriato; è anche possibile **bloccare** un pacchetto (es. da/verso host malevoli o vietati), **scartare** un pacchetto scaduto, **duplicare** un pacchetto e inviarlo su più link, o **modificarlo** (es. per notificare congestione). Il **routing** consiste invece nel **decidere il cammino** seguito dai pacchetti da mittente a ricevente; gli algoritmi che calcolano i cammini si chiamano **algoritmi di routing**.

Tipi di router

I router sono molto **diversi** per uso e tecnologia: domestici o aziendali, wireless o cablati. Una distinzione importante è quella tra **edge router**, che distribuiscono pacchetti tra reti diverse (es. collegano una rete all'ISP) e sono la tipologia più comune, e **core router**, che distribuiscono pacchetti **dentro la stessa rete** e sono usati **sul backbone di Internet** per trasferimenti di dati pesanti.

Piano dati e piano di controllo

Il lavoro dei router è tipicamente un processo **a due piani**. Il **forwarding** (piano dati) è un'azione **locale al router**, che trasferisce un pacchetto da un'interfaccia di input all'output appropriata: è un'operazione **veloce** (pochi nanosecondi), tipicamente implementata in hardware. Il **routing** (piano di controllo) è invece un processo **a livello di rete** che determina i cammini end-to-end: un processo **più lento** (secondi), spesso implementato via software.

Nel piano dati c'è la **forwarding table**: una tabella che associa link di output a possibili indirizzi di destinazione. Il router inoltra un pacchetto **esaminando il valore di uno o più campi dell'header**, e il

valore nella forwarding table indica **l'interfaccia in uscita**. Poiché si attraversano più router, il contenuto della forwarding table è determinato **raccogliendo informazioni da router diversi**, tramite le **tabelle di routing** (piano di controllo) che portano alla creazione della forwarding table (piano dati). La creazione delle tabelle avviene in due modi:

- **Decentralizzato (distribuito)**: ogni router ha un componente di routing che comunica con i componenti degli altri router (approccio dominante storicamente).
- **Centralizzato**: un controller remoto, fisicamente separato dai router, calcola e distribuisce le forwarding table.

Nel caso centralizzato il controller può stare in un **data center remoto** (con alta affidabilità e ridondanza), gestito dall'ISP o da terzi. Questo approccio è alla base del **software-defined networking** (SDN): la rete è “definita via software” perché il controller è un software centralizzato (a volte open source).

Componenti principali di un router

- **Porte di input** (diverse dalle “porte” di trasporto): le **interfacce fisiche di input**, che operano con il livello di collegamento del link connesso. Devono consultare la forwarding table (**lookup**) e preparare lo switching fabric per la porta di output, e **inoltrare i pacchetti di controllo** (es. con informazioni di routing) al processore di routing. Le porte vanno da decine a centinaia (es. Juniper MX2020 supporta fino a 960 porte Ethernet da 10 Gbps).
- **Switching fabric**: collega le porte di input alle porte di output.
- **Porte di output**: memorizzano i pacchetti ricevuti dallo switching fabric e li trasmettono sul link in uscita; le porte possono essere **bidirezionali** (input/output accoppiati).
- **Processore di routing**: esegue i protocolli di routing, mantiene informazioni di stato sui link, calcola/aggiorna la forwarding table.

Forwarding

La **forwarding table** associa indirizzi IP a porte di output, così i pacchetti sono inoltrati sul link giusto verso il nodo successivo. Un **indirizzo IP** è un **numero di 4 byte (32 bit)**, di solito in notazione decimale (ma si può vedere anche in binario). Ricevuto un pacchetto su una porta di input si esegue una **lookup**: **ogni porta ha una copia della forwarding table** (dal processore di routing, via bus dedicato, es. PCI), per evitare il collo di bottiglia di invocare il processore centralizzato per ogni pacchetto, e **più indirizzi possono essere associati alla stessa porta**.

Esempio di tabella a 4 porte:

Prefisso di indirizzi IP	Interfaccia
11001000 00010111 00010*** **	0
11001000 00010111 00011000 ***	1
11001000 00010111 00011*** **	2
altrimenti	3

Le voci **non sono mutuamente esclusive** (es. 11001000 00010111 00011000 10101010 corrisponde sia al link 1 che al link 2), quindi il router sceglie la **voce con il prefisso più lungo** (*longest prefix matching*): il match col Link 1 vale 24 bit e il match col Link 2 vale 21 bit, quindi **vince il Link 1**. La lookup è tipicamente eseguita **in hardware**, per essere più veloce possibile (nanosecondi per trasmissioni in Gigabit), e coinvolge memorie incorporate e algoritmi di ricerca avanzati.

Determinata la porta di output, **il pacchetto entra nello switching fabric**; in alcuni dispositivi può essere messo temporaneamente in coda se altre porte stanno usando la fabric. L'operazione in due passi — **cercare l'indirizzo di destinazione (match)** e **inoltrare (action)** — si chiama **match-plus-action**, ed è svolta in molti dispositivi: negli **switch** l'azione è simile ai router, nei **firewall** l'azione è **filtrare** specifici pacchetti in arrivo, nel **NAT** l'azione è **riscrivere il numero di porta** prima dell'inoltro.

Switching fabric

Lo switching si può fare in tre modi:

1. **Switching via memoria:** le porte sono viste come dispositivi di I/O che scrivono i pacchetti in celle di memoria; il processo di routing copia il messaggio sulla porta di output come indicato dalla forwarding table. È un approccio un po' lento (accesso alla memoria), comune nei primi router (calcolatori normali).
2. **Switching via bus:** la porta di input trasferisce il pacchetto direttamente alla porta di output su un bus condiviso, senza intervento del processore di routing. Solo una porta per volta è servita, ma spesso è sufficiente per router in piccole reti locali.
3. **Switching via interconnection network:** le porte di input/output sono collegate da una rete con punti d'incrocio (*crossbar*) che si aprono/chiudono per reindirizzare i pacchetti. **Più pacchetti possono essere inoltrati in parallelo;** è usato in molti router moderni.

Code sulle porte

Poiché lo **switching richiede tempo**, le porte di input e output hanno **code per memorizzare temporaneamente i pacchetti** (come auto ai semafori). L'**entità delle code non è fissa**: dipende dal carico di traffico, dalla velocità della fabric e dalla velocità di linea; i pacchetti possono arrivare o partire più velocemente o lentamente dello switching, accumulandosi nei buffer (che possono **traboccare**).

Scenario worst-case

Si consideri un router con **N porte di input e N di output**, tutte le porte che ricevono pacchetti contemporaneamente, tutte le linee alla stessa velocità R_{line} pacchetti/s, e **tutti i pacchetti che devono andare sulla stessa porta di output**. Con velocità della fabric R_{switch} :

- Se $R_{switch} \cong NR_{line}$: **code trascurabili sulle porte di input** (la fabric smista tutto in tempo), ma **code significative sulla porta di output** (i pacchetti in arrivo sono N volte la velocità del link in uscita).
- Se $R_{line} < R_{switch} < NR_{line}$: **code su entrambe** (l'input aspetta la fabric, l'output aspetta il link).
- Se $R_{switch} \cong R_{line}$: **code trascurabili sull'output** ma **significative sulle porte di input**.

Scheduling dei pacchetti

È ragionevole che **più pacchetti** (anche da più porte) debbano uscire **dalla stessa porta di output**: l'accesso dei pacchetti in coda al link di output va quindi **pianificato (scheduled)**. Tre approcci celebri sono il **First-come-first-served (FCFS / FIFO)**, basato sul tempo, il **priority queuing**, basato sull'importanza dei pacchetti, e il **round-robin queuing**, in cui i pacchetti sono divisi in classi e ogni classe è servita a turno.

FIFO

Se il **link di output è occupato**, i pacchetti in arrivo vanno **bufferizzati**; se lo **spazio di buffer è insufficiente**, serve una **politica di scarto**: tipicamente si scartano i pacchetti appena arrivati (*drop-tail*), mentre in approcci più sofisticati si possono rimuovere anche pacchetti già in coda per far spazio ai nuovi. Un pacchetto esce dalla coda **solo quando è stato trasmesso completamente** sul link in uscita, e nello **scheduling FIFO** i pacchetti sono trasmessi **nello stesso ordine di arrivo**.

Priority queuing

I pacchetti in arrivo sono **classificati** (es. per porte TCP/UDP) in classi di priorità al loro arrivo in coda. Un **operatore di rete può configurare la coda** così che specifici pacchetti (es. management di rete, voice-over-IP in tempo reale) abbiano priorità sul traffico utente o sui pacchetti non real-time. **Ogni classe di priorità può avere la propria coda**: si trasmettono prima i pacchetti della classe a priorità più alta non vuota, e dentro la stessa classe la scelta è tipicamente **FIFO**.

Round-robin

Anche nel round-robin i pacchetti sono ordinati in classi, ma **le classi si alternano** anziché essere scelte per priorità. Un'implementazione comune è la **weighted fair queuing (WFQ)**: i pacchetti sono classificati e messi in coda nell'area di attesa appropriata, e ogni classe ha un **peso** che detta la **frequenza con cui la classe è selezionata** rispetto alle altre.

Lezione 17: Livello di rete — gestione degli indirizzi

Ripasso

Il livello di rete svolge soprattutto la **consegna host-a-host**: incapsulare i segmenti in datagram e decapsularli, oltre a **identificare gli host (indirizzi)** e **inoltrare** i messaggi verso l'host desiderato. Dentro la rete ci sono **nodi che inoltrano i datagram ai nodi adiacenti** (router), con più **porte in ingresso/uscita**, un **modulo di switching** che collega le porte e un **processore di controllo** che crea le **forwarding table**.

Internet Protocol (IP)

L'**Internet Protocol (IP)** è il protocollo di livello rete che garantisce la consegna host-a-host; ne esistono due versioni in uso: **IPv4**, la più usata e comune, e **IPv6**, versione più recente proposta per sostituirla. La funzionalità più importante di IP è **identificare gli host** sulla rete (IP addressing): l'**IP addressing** è il processo di assegnare indirizzi IP ai dispositivi, che è **cruciale** e in Internet è piuttosto **complesso** (milioni o miliardi di host).

Indirizzi IPv4

Gli indirizzi IP si scrivono in **notazione decimale puntata** (*dotted-decimal*): ogni byte in forma decimale, separato da punti.

	Decimale puntata	Binario
Indirizzo IP	193.32.216.9	11000001 00100000 11011000 00001001

Ogni dispositivo nella rete globale deve avere un **IP globalmente univoco**. Un IP è lungo **32 bit (4 byte)**, quindi in totale esistono 2^{32} (circa 4 miliardi) di indirizzi possibili. Host e dispositivi sono connessi tramite **link** (cablati o wireless): un host ha tipicamente un solo link, mentre un dispositivo di rete (es. router) **può averne più di uno**. Il confine tra host e link fisico si chiama **interfaccia**; poiché ogni host e router invia e riceve datagram, IP richiede che **ogni interfaccia abbia il proprio IP**: un **indirizzo IP è tecnicamente associato all'interfaccia**, non all'host o al router che la contiene.

Subnetting

Assegnare indirizzi IP **non è banale**: sarebbe imprudente assegnarli a caso, perché gli IP non dicono nulla sulla posizione (non sapremmo dove trovare gli host) e servirebbero **forwarding table enormi** nei router. L'indirizzamento assomiglia a quello *telefonico*: **le reti sono divise gerarchicamente in sotto-reti (subnet)** con prefissi diversi. Un IP si divide in **due parti**: la prima (a sinistra) identifica la **subnet** a cui il nodo è connesso, la seconda (a destra) identifica la **singola interfaccia**; il numero di bit di ciascuna parte **non è fisso**.

Subnet mask

Per distinguere la parte di subnet da quella di interfaccia si usa una **subnet mask** che specifica quali bit appartengono alla subnet:

	Decimale puntata	Binario
Indirizzo IP	193.32.216.9	11000001 00100000 11011000 00001001
Subnet mask	255.255.255.0	11111111 11111111 11111111 00000000

Un'altra notazione comune è quella **con slash**: 193.32.216.0/24 indica l'IP della subnet, dove /24 dice che i 24 bit più a sinistra sono dedicati alla subnet. La mask individua un prefisso, quindi gli "1" devono essere **i più a sinistra e contigui**:

Decimale puntata	Binario	
255.255.255.0	11111111 11111111 11111111 00000000	SUBNET MASK (/24)
255.255.128.0	11111111 11111111 10000000 00000000	SUBNET MASK (/17)
255.224.0.0	11111111 11100000 00000000 00000000	SUBNET MASK (/11)
255.255.10.0	11111111 11111111 00001010 00000000	NON È UNA MASK
63.255.255.0	00111111 11111111 11111111 00000000	NON È UNA MASK

Stessa subnet o no?

Due host sono sulla stessa subnet se i loro **prefissi di subnet coincidono**. Esempi:

IP 1	Subnet mask	IP 2	
231.23.11.117	255.255.255.0	231.23.11.9	STESSA SUBNET
10.54.32.1	255.255.128.0	10.54.60.203	STESSA SUBNET
110.32.100.10	255.224.0.0	110.64.100.11	SUBNET DIVERSE
121.50.79.4	255.0.0.0	121.10.36.240	STESSA SUBNET
121.50.79.4	255.0.0.0	120.50.79.5	SUBNET DIVERSE

Host di subnet diverse **possono comunque comunicare**, ma gli amministratori possono trattare i messaggi in ingresso/uscita in modo diverso (es. bloccarli/filtrarli).

Router e subnet

Il dispositivo che interconnette subnet diverse è il **router**: ha **più porte fisiche**, legate a **interfacce di rete diverse** con IP diversi. I router sono spesso al confine tra più subnet, gestendo i messaggi in ingresso/uscita (**gateway**). Grazie a questa posizione privilegiata, **i router offrono spesso più servizi** oltre al routing: IP masquerading (NAT), assegnazione dinamica degli IP (DHCP) e protezione (firewall).

Esempi

Un router con tre interfacce interconnette 7 host, divisi in 3 subnet (sinistra, destra, sotto), ognuna collegata a un'interfaccia. La subnet di sinistra ha indirizzo 223.1.1.0/24, quindi tutte le interfacce hanno IP nella forma 223.1.1.XXX (223.1.1.1, 223.1.1.2, 223.1.1.3 per gli host, 223.1.1.4 per il router); le altre due subnet sono 223.1.2.0/24 e 223.1.3.0/24. Con **3 router** interconnessi da link punto-punto, ognuno con **3 interfacce** (una per link punto-punto, una per il link broadcast verso i suoi host), si ottengono **6 subnet**: 223.1.1.0/24 (R1-host), 223.1.2.0/24 (R2-host), 223.1.3.0/24 (R3-host), 223.1.9.0/24 (R1-R2), 223.1.8.0/24 (R2-R3) e 223.1.7.0/24 (R3-R1). Tipicamente le organizzazioni medie/grandi (aziende, atenei) hanno più subnet interconnesse.

ifconfig

Su Linux si controlla il proprio indirizzo con `ifconfig` (equivalente Windows: `ipconfig`): `ifconfig` (*interface configuration*) elenca tutte le interfacce di rete della macchina con la loro configurazione (indirizzi, mask, ecc.); esistono anche comandi moderni come `ip`. Uso: `$ ifconfig`

```
eth0  Link encap:Ethernet  Hwaddr 00:0F:20:CF:8B:42
      inet addr:217.149.127.10  Bcast:217.149.127.63  Mask:255.255.255.192
      UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
      RX packets:2472694671 errors:1 dropped:0 overruns:0 frame:0
      TX packets:44641779 errors:0 dropped:0 overruns:0 carrier:0
      ...
```

Indirizzamento in Internet

Spezzare reti grandi in reti più piccole è **particolarmente importante** su Internet, dove si collegano miliardi di dispositivi. Su Internet gli IP **vanno assegnati con cura** per evitare **forwarding table** dei router troppo grandi, interfacce diverse con **lo stesso indirizzo** e l'esaurimento degli indirizzi. L'approccio è dividere gli indirizzi **fornendo subnet alle organizzazioni** (ISP, aziende, istituzioni), in due modi: il **classful addressing** (vecchio, non più usato in pratica) e il **classless addressing** (attuale).

Classful addressing

Gli indirizzi erano divisi in classi:

	Formato	Esempio	IP per rete
Classe A	a.b.c.d/8	10.X.X.X	> 16 milioni
Classe B	a.b.c.d/16	10.10.X.X	65535
Classe C	a.b.c.d/24	10.10.10.X	254

Se servivano 300 IP si assegnava **una classe B intera** (es. 241.115.0.0) con tutti i suoi IP. Il problema è evidente: servono solo 300 IP e **gli altri 65335 sono sprecati!**

Classless addressing (CIDR)

L'approccio moderno, più flessibile, è il **Classless InterDomain Routing (CIDR)**: un'organizzazione può ricevere indirizzi di qualunque forma a.b.c.d/X. Se servono 300 IP si assegna a.b.c.d/23 (es. 241.115.2.0/23), che **fornisce 512 IP sprecandone solo 212**. Negli indirizzi "CIDRizzati" la parte di rete si chiama **prefisso** e l'insieme di IP riservati all'organizzazione si chiama **blocco**. I blocchi possono sovrapporsi, e in tal caso la lunghezza (X) del prefisso discrimina i blocchi: ad esempio l'IP 10.10.10.15/16 non è nello stesso blocco di 10.10.10.14/24. È inoltre possibile creare **subnet interne** a un blocco CIDR: il blocco 241.115.0.0/16 si può decomporre in 241.115.1.0/24, 241.115.2.0/24, 241.115.3.0/24, ecc.

Aggregazione degli indirizzi

L'indirizzamento a prefissi è utilissimo per i dispositivi che connettono prefissi diversi: un router può ricordare (nella forwarding table) **solo i prefissi**, e ricevuto un messaggio con un certo prefisso lo inoltra a un router più specifico, e così via. Questo approccio si chiama **address aggregation** (o *route summarization*), ed è più efficace finché i blocchi sono **raggruppati (clustered)**.

Ottenere un blocco

Per **ottenere un blocco di IP** per la subnet di un'organizzazione ci sono due modi: contattare un **ISP**, che fornisce indirizzi da un blocco più grande già allocatogli (es. l'ISP può aver ricevuto 200.23.16.0/20, suddiviso in sotto-blocchi di dimensione variabile secondo la sua politica), oppure chiedere alla **Internet Corporation for Assigned Names and Numbers (ICANN)**, autorità globale no-profit che gestisce lo spazio degli indirizzi IP allocando blocchi agli ISP; ICANN gestisce anche i root server DNS, assegnando nomi di dominio e risolvendo le dispute.

Il datagram IPv4

Il pacchetto di livello rete di Internet si chiama **datagram** (come in UDP). I campi chiave sono:

- **Version** (4 bit): la **versione di IP** (v4 o v6); il formato dipende dalla versione.
- **Header length** (4 bit): **dimensione dell'header** (non fissa, opzioni a dimensione variabile); serve a sapere dove inizia il payload (senza opzioni l'header è 20 byte).
- **Type of Service (TOS, 8 bit)**: **proprietà** specifiche del datagram (es. real-time/non-real-time); i tipi sono **definiti dagli amministratori di rete** dei router.
- **Datagram length** (16 bit): lunghezza in **byte del datagram** (header + dati); raramente oltre 1500 byte (su 65535 max).
- **Identifier** (16 bit): numero progressivo che identifica univocamente un datagram (usato nella frammentazione).
- **Flags + fragmentation offset** (3+13 bit): proprietà e offset del frammento (usati nella frammentazione).
- **Time-to-live (TTL, 8 bit)**: garantisce che i datagram **non girino per sempre** nella rete; è decrementato di 1 a ogni router e a 0 il datagram va scartato.
- **Upper-layer protocol** (8 bit): il **protocollo di trasporto** cui passare il payload (es. 6 per TCP, 17 per UDP).
- **Header checksum** (16 bit): rilevamento errori, calcolato come complemento a 1 della somma a 2 byte dell'header. **I router verificano** il valore e **i datagram errati sono tipicamente scartati**; il checksum va **ricalcolato e ri-memorizzato a ogni router**, perché TTL e opzioni possono cambiare.
- **Indirizzi IP sorgente e destinazione** (32+32 bit).
- **Options** (dimensione variabile): estendono l'header IP (**funzionalità aggiuntive**); aggiungono complessità (dimensione sconosciuta) e **non esistono in IPv6**.
- **Data**: il **messaggio effettivo (payload)**, tipicamente un segmento TCP/UDP.

Frammentazione IPv4

I datagram IP possono essere **frammentati a livello di link**, perché alcuni protocolli di collegamento trasportano datagram di **dimensioni diverse** (es. i frame Ethernet trasportano al massimo 1500 byte di dati). I datagram IP sono incapsulati in frame di livello link per essere trasportati di nodo in nodo, e la quantità massima di dati che un frame può trasportare è la **MTU** (*Maximum Transmission Unit*). Un router che interconnette due link con MTU diverse **può ricevere un datagram dall'input che non entra nel link di output**: in tal caso **il router deve dividere il payload** del datagram in due o più datagram IP più piccoli (**frammenti**), che sono **incapsulati in frame di livello link** e inoltrati.

Quando un router frammenta un datagram, copia in ogni frammento **lo stesso identificatore, indirizzo sorgente e destinazione**, imposta il campo **fragment offset** di tutti i frammenti a numeri progressivi, e imposta il flag dell'**ultimo frammento** a 1 (segnalando che i frammenti sono finiti). I frammenti vanno **riassemblati prima di arrivare al livello di trasporto** a destinazione, perché TCP e UDP si aspettano segmenti completi dal livello di rete. I progettisti di IPv4 ritenevano che riassemblare nei router avrebbe complicato il protocollo e rallentato i router, quindi in IPv4 il **riassemblaggio avviene nei sistemi finali**: se arrivano più datagram con **stessi indirizzi e identificatore**, il datagram originale era frammentato e l'host deve **ricostruire il datagram originale** dai frammenti.

Lezione 18: Livello di rete — DHCP, LAN, IPv6

Assegnazione degli IP: DHCP

Dentro un blocco di indirizzi, gli IP **vanno assegnati alle singole interfacce**. Un **amministratore di sistema** può configurarli in due modi: **manualmente**, assegnando gli IP uno a uno agli host, oppure **automaticamente**, lasciando che la rete assegni autonomamente IP liberi agli host in arrivo. Il secondo approccio (il più comune) usa il **Dynamic Host Configuration Protocol (DHCP)**: oltre all'indirizzo, DHCP fornisce all'host le informazioni per entrare nella rete — subnet mask, indirizzo del **gateway di default** (per uscire dalla rete) e indirizzo del **server DNS locale**. DHCP è **plug-and-play** ed è tipicamente usato nelle case.

Funzionamento

DHCP è client-server: un host che arriva (client) contatta il server DHCP per ricevere le informazioni di rete; il servizio può essere offerto da un computer o dal router stesso. Formalmente DHCP è un protocollo di **livello applicazione**. Nell'esempio delle 3 subnet collegate da un router, la rete a destra (233.1.2.0/24) ha un server DHCP con cui un nuovo host che entra “tratta” l'IP. Aggiungere un host è un processo in 4 passi:

1. **DHCP server discovery**: il nuovo host invia in broadcast un messaggio DHCP discovery (UDP porta 67) per trovare il server, con IP destinazione 255.255.255.255 (broadcast), IP sorgente 0.0.0.0 (*this host*) e un ID di transazione casuale.
2. **DHCP server offer**: potendoci essere più server, quando un server riceve la discovery risponde in broadcast (porta 68) offrendo una possibile configurazione di rete (campo *yiaddr* – *Your Internet ADDRESS*), con IP destinazione 255.255.255.255 e IP sorgente 223.1.2.5 (server DHCP).
3. **DHCP request**: il nuovo client sceglie l'offerta accettata, riecheggando il messaggio di offerta.
4. **DHCP ACK**: messaggio ACK finale che conferma la transazione.

Se il server è in **un'altra subnet** serve un **DHCP relay agent** che inoltra i messaggi (può farlo un router): configurando il router come relay agent, anche gli host delle subnet 223.1.1.0/24 e 223.1.3.0/24 saranno serviti dal nostro DHCP.

Comandi Linux

`ip addr` mostra la configurazione corrente (indirizzo IP, mask); `ip route` mostra il gateway di default.

Visibilità degli IP e NAT

Su Internet ci sono miliardi di dispositivi da associare a IP univoci **per essere raggiungibili da chiunque**. Ma è davvero necessario che **tutti i dispositivi siano visibili a tutti**? Esporre tutti i dispositivi presenta problemi: gli IP sono finiti; per dispositivi connessi localmente è spesso **irragionevole (o indesiderabile) esporli** su Internet; e gli **amministratori locali** dovrebbero conoscere a fondo la struttura del blocco IP per assegnare indirizzi liberi (spesso non dipende da loro, es. reti domestiche).

Network Address Translation (NAT)

Il servizio NAT permette di **rimappare gli indirizzi IP** dei pacchetti in una rete in indirizzi diversi (**network masquerading**). Si usa una **tabella di traduzione** che associa **più IP/porte locali (della LAN)** a un **IP/porta globale (della WAN)**; il servizio può essere offerto dai **router**. La rete locale mascherata si chiama anche **rete privata** (*realm with private addresses*): gli IP privati sono validi solo dentro la rete privata.

Si consideri un router NAT-enabled le cui quattro interfacce della LAN hanno subnet 10.0.0.0/24: il router si comporta da “passa-messaggi” che sovrascrive gli IP come da tabella. I pacchetti in **uscita** hanno IP/porta sorgente sovrascritti con un singolo IP WAN (138.76.29.7) e una porta nuova (es. 5001); i pacchetti in **ingresso** hanno IP/porta destinazione sovrascritti con l'IP LAN specifico dell'host (es. 10.0.0.1) e la porta iniziale

(3345). È importante notare che gli host non conoscono il traffico degli altri, quindi il NAT **non può usare la porta iniziale**, perché più host potrebbero scegliere la stessa porta contemporaneamente; essendo la porta a 16 bit, un NAT può gestire **oltre 60000 connessioni simultanee**.

Ping

Per verificare se un host è raggiungibile si usa `ping` (uguale su Linux/Windows). **Ping** (*Packet Internet Groper*) si basa su **ICMP** (*Internet Control Message Protocol*) e invia un pacchetto “echo request” a un host, che risponde automaticamente con “echo reply”; ping fornisce anche il **tempo trascorso** tra richiesta e risposta, misurando l’RTT. Uso: `$ ping [indirizzo]`

```
$ ping google.com
PING google.com (142.251.209.46) 56(84) bytes of data.
64 bytes from mil04s51-in-f14.1e100.net (142.251.209.46): icmp_seq=1 ttl=117 time=19.8 ms
64 bytes from mil04s51-in-f14.1e100.net (142.251.209.46): icmp_seq=2 ttl=117 time=20.3 ms
...
```

Nmap (parte 2)

Oltre alla scansione delle porte, `nmap` può **mappare/scansionare i dispositivi** della rete, usando ping (ICMP) per scoprire gli host. Uso: `sudo nmap -sn [indirizzo gateway]/[bit di subnet]`

```
$ sudo nmap -sn 100.103.0.1/16
Nmap scan report for _gateway (100.103.0.1)
Host is up (0.0024s latency).
MAC Address: 10:BD:18:E5:74:80 (Cisco Systems)
Nmap scan report for 100.103.0.3
...
```

Indirizzi IP riservati

Poiché gli amministratori locali devono poter **organizzare una rete privata senza interferenze**, esistono per convenzione **blocchi IP riservati**, che ISP/ICANN non possono assegnare:

- 10.0.0.0 – 10.255.255.255 (riservato alle **reti private**).
- 192.168.0.0 – 192.168.255.255 (riservato alle **reti private**).
- 127.0.0.0 – 127.255.255.255 (indica questa macchina, **loopback**; tipicamente si usa 127.0.0.1).
- 0.0.0.0 (indica la **rete corrente**; anche tutti gli indirizzi con 0 nella parte host).
- 255.255.255.255 (**broadcast**; anche tutti gli indirizzi con 255 nella parte host).

La necessità di riservare IP nasce dal fatto che **gli indirizzi privati possono coincidere con quelli pubblici**: il routing dentro la rete sarebbe **ambiguo**.

Esempio: setup di una LAN

Si vuole creare **una nuova sottorete locale (LAN) dentro una rete preesistente (WAN)**. L’amministratore del WAN fornisce: l’IP del WAN (150.100.0.0/16), l’IP del gateway verso la WAN esterna/Internet (150.100.50.1) e un IP libero per la nostra rete (150.100.50.10). Configuriamo router e dispositivi con un **router NAT-enabled**; l’IP della nuova LAN sarà 192.168.1.0/24 (riservato alle reti locali).

Configurazione lato WAN

Il router locale ha **2 interfacce** (WAN e LAN). Lato WAN si specificano: l’indirizzo IP del router nel WAN, la subnet del WAN, il gateway e uno o più DNS:

```
Internet Connection Type: Static IP
IP Address:      150.100.50.10
Subnet Mask:     255.255.0.0
Default Gateway: 150.100.50.1
Primary DNS:     8.8.8.8
Secondary DNS:   4.4.4.4 (opzionale)
```

Si può indicare un **IP dinamico** se il WAN ha un servizio DHCP.

Configurazione lato LAN

Si dà un IP all'interfaccia LAN del router: essendo la rete 192.168.1.0/24, si assegna 192.168.1.1/24 (è tipico dargli il .1). Il router fornisce anche **DHCP**, quindi si specificano l'**address pool** (per gli host DHCP), il **lease time** (durata dell'assegnazione, dopo la quale l'IP può essere riusato) e **gateway/DNS** da comunicare agli host:

```
MAC Address: 48-22-54-16-18-7D
IP Address: 192.168.1.1
Subnet Mask: 255.255.255.0

DHCP Server: [x] Enable
IP Address Pool: 192.168.1.100 - 192.168.1.250
Address Lease Time: 120 minutes
Default Gateway: 192.168.1.1
```

Configurazione di un host

L'host può essere configurato in modo **automatico (DHCP)**, usando DHCP per configurare la connessione (IP da 192.168.1.100 a 192.168.1.250), oppure **manuale**, impostando a mano (serve conoscere la configurazione e gli IP disponibili) l'indirizzo IP statico dell'host, la subnet mask, il gateway e i DNS. Gli host connessi **inviano i pacchetti al router locale** (ignari del WAN esterno), e il router locale **inoltra i pacchetti al router WAN** (e possibilmente a Internet). Un nuovo host può usare un IP manuale **fuori dal pool DHCP** o automatico **dentro il pool**.

IPv6

Nei primi anni '90 l'**IETF** iniziò a sviluppare un successore di IPv4, per far fronte alla **penuria di indirizzi a 32 bit**. IPv6 fu progettato per **garantire più indirizzi**, ma anche per **aggiornare altri aspetti di IPv4** sulla base dell'esperienza operativa maturata. Non è certo quando gli IPv4 si esauriranno (si speculava 2008 o 2018, ma ne restano ancora). Quanto ad **IPv5**, fu proposto nel 1979, basato sul protocollo sperimentale *Internet Stream Protocol* (ST); usava ancora indirizzi a 32 bit, quindi fu **abbandonato proprio per la scarsità di indirizzi**.

Il datagram IPv6

I vantaggi principali di IPv6 sono:

- **Indirizzamento ampliato**: da 32 a **128 bit** ($3,4028237 \times 10^{38}$ indirizzi): non si esauriranno mai — ogni granello di sabbia del pianeta può avere un IP.
- **Header a lunghezza fissa di 40 byte**: alcuni campi di IPv4 sono stati eliminati o resi opzionali.
- **Flow labeling**: i pacchetti di alcune applicazioni (es. streaming audio/video) possono essere raggruppati in un **flusso** con identificazione specifica; il ruolo preciso dei flow è ancora in discussione.

I campi del datagram IPv6 sono:

- **Version** (4 bit): numero di versione (in IPv6 vale 6).
- **Traffic class** (8 bit): come il TOS di IPv4, dà priorità ai datagram (es. VoIP su SMTP).
- **Flow label** (20 bit): identifica un flusso di datagram.
- **Payload length** (16 bit): byte del payload (header di 40 byte escluso).
- **Next header** (8 bit): protocollo di livello superiore cui consegnare i dati (es. TCP o UDP); simile al campo protocol di IPv4.
- **Hop limit** (8 bit): il time-to-live come in IPv4 (decrementato a ogni inoltro).
- **Indirizzi sorgente e destinazione** (128+128 bit).
- **Data** (variabile): il payload del datagram.

Cosa manca rispetto a IPv4

- **Campi di frammentazione**: IPv6 **non consente frammentazione/riassemblaggio sui router intermedi**, solo sugli host. Se un datagram è troppo grande per essere inoltrato, il router lo **scarta** e invia al mittente un errore "Packet Too Big"; il mittente può rispedire con un datagram più piccolo.

- **Header checksum:** costa tempo sui router ed è ridondante, perché i protocolli di livello link fanno tipicamente checksum sull'intero pacchetto.
- **Options:** non fanno più parte dell'header standard; alcune opzioni si specificano nel campo next header (insieme agli identificatori TCP/UDP).

Da IPv4 a IPv6

Il grande problema è che **IPv6 non è retrocompatibile**: i sistemi IPv4 non sanno gestire datagram IPv6. Come adotterà IPv6 Internet? Con l'approccio “**flag day**”, a una certa data **tutte le macchine di Internet verrebbero spente e aggiornate** da IPv4 a IPv6 — un approccio simile fu usato quando fu introdotto TCP, ma fu un disastro (anche per la piccola rete di allora), e con miliardi di dispositivi è impensabile. L'alternativa è l'approccio **tunneling**: router dedicati (**tunnel**) incaricati di **mappare datagram IPv4 in IPv6** e viceversa, in modo quasi trasparente; è probabilmente l'approccio più realistico ed è **già usato in pratica**. L'adozione di IPv6, inizialmente lenta, sta accelerando.

Lezione 20: Livello di rete — piano di controllo e routing

Introduzione al routing

Alla ricezione di un pacchetto, un **router può compiere tre azioni**: inoltrare il pacchetto tale e quale a un vicino adatto; inoltrare un pacchetto modificato o sostituito (ad esempio in caso di frammentazione o masquerading); oppure scartarlo, ad esempio perché scaduto. Esistono poi tipi diversi di router: alcuni molto semplici, usati dagli amministratori locali per connettere LAN e WAN, altri usati dagli ISP per inoltrare i pacchetti al dispositivo giusto in tutta Internet.

Ogni router possiede una **forwarding table** che specifica la strategia locale di inoltro nel proprio vicinato, e alla creazione delle tabelle si può procedere in modo centralizzato o decentralizzato. Poiché i router non hanno conoscenza del resto della rete, per decidere dove inoltrare i pacchetti si usano **algoritmi di routing**, che determinano buoni cammini (sequenze di rotte) da mittenti a riceventi. Tipicamente un “buon” cammino è quello a **costo minimo**, quindi più corto o più veloce; nel mondo reale, però, vanno considerati anche gli **aspetti di policy**, come le regole su quali organizzazioni possono comunicare tra loro. Il problema del routing è cruciale per le reti di calcolatori, ma trovare buoni cammini su un grafo è rilevante anche in molti altri ambiti (videogiochi, guida autonoma, robotica).

Flooding

L’approccio più semplice è il **flooding**: ogni router inoltra tutti i pacchetti in arrivo su tutti i link, tranne quello da cui li ha ricevuti. Poiché vengono esplorati tutti i cammini, incluso l’ottimo, il pacchetto sarà consegnato di sicuro se esiste un cammino; i pacchetti che “non vanno da nessuna parte” raggiungono l’**hop limit** e vengono scartati. Grazie alla ridondanza l’approccio è **estremamente robusto**: anche se alcuni router sono offline, i cammini alternativi vengono comunque considerati. Il flooding è quindi adatto a situazioni specifiche in cui serve ridondanza (broadcast di messaggi, scenari militari), ma risulta **impraticabile in reti grandi e ad alte prestazioni** come Internet, per le quali servono algoritmi più raffinati.

Formalizzazione del problema

La rete si formalizza come un **grafo** $G = (N, E)$, dove N è l’insieme dei nodi (i router della rete) ed E è l’insieme degli archi, cioè i link fisici, rappresentati come coppie di nodi. Ogni arco ha un valore di **costo**, che può riflettere la lunghezza fisica del link, la sua velocità o un costo monetario. Si definisce inoltre una funzione di costo c tale che, data una coppia di nodi (x,y) : se $(x,y) \in E$, allora $c(x,y)$ è il costo dell’arco; se $(x,y) \notin E$, allora $c(x,y) = \infty$; e poiché il grafo è non orientato, se $(x,y) \in E$ anche $(y,x) \in E$ e $c(x,y) = c(y,x)$.

Un **cammino** è una sequenza di nodi in cui tutti i nodi adiacenti sono collegati:

$$(x_1, x_2, \dots, x_p) \quad \text{con} \quad (x_1, x_2), \dots, (x_{p-1}, x_p) \in E$$

Il costo complessivo del cammino è la somma dei costi dei link. Detto P_{uz} l’insieme di tutti i cammini possibili tra i nodi u e z , il cammino migliore è quello a **costo minimo**, e gli algoritmi che lo trovano si chiamano **algoritmi a costo minimo** (o shortest-path, quando tutti gli archi hanno lo stesso costo). Nelle reti di calcolatori interessa trovare il cammino ottimo **per tutte le coppie di nodi**: un algoritmo di routing **converge** quando il cammino più corto è stato trovato per ogni coppia. Le due famiglie tipiche sono il **Distance Vector** e il **Link State**.

Algoritmo Distance-Vector (DV)

Il **Distance-Vector** è un approccio **decentralizzato** che sfrutta la conoscenza locale della rete, combinata con la comunicazione tra nodi, per trovare i cammini minimi; si basa sull'**algoritmo di Bellman-Ford**. Ogni nodo riceve informazioni da uno o più vicini direttamente collegati, esegue un calcolo e distribuisce il risultato ai vicini. Poiché non richiede che tutti i nodi siano coordinati tra loro, l'algoritmo è **asincrono**.

L'equazione di Bellman

Detto $d_x(y)$ il costo del miglior cammino dal nodo x al nodo y :

$$d_x(y) = \min_v \{c(x, v) + d_v(y)\}$$

L'intuizione è che deve esistere un nodo adiacente v a x che sta sul miglior cammino da x a y : il costo di quel cammino è il costo per raggiungere v , più il miglior cammino tra v e y . Trovato tale v , lo si inserisce nella **forwarding table** e gli si inoltrano tutti i pacchetti diretti a y . Per farlo serve una stima di $d_v(y)$ per tutti i vicini: il **vettore di distanza** (distance vector).

Esempio

Tra u e z il miglior cammino è:

$$u \rightarrow x \rightarrow y \rightarrow z$$

Lungo il cammino ottimo l'equazione di Bellman deve valere per tutti i nodi. Considerando u , che ha tre vicini w , v e x : per x valgono $c(u, x) = 1$ e $d_x(z) = 3$; per v valgono $c(u, v) = 2$ e $d_v(z) = 5$; per w valgono $c(u, w) = 5$ e $d_w(z) = 3$. Di conseguenza **x minimizza** $c(u, v) + d_v(z)$ ed è il prossimo nodo del cammino ottimo. Iterando sul nodo successivo, l'equazione continua a valere: y minimizza per x , e così via. Con il distance vector di tutti i nodi, l'equazione di Bellman permette quindi di calcolare il **next hop** per la forwarding table.

Pseudocodice

```
DistanceVector(x):
  per ogni nodo v:
    se v è vicino di x: D_x(v) = c(x, v)
    altrimenti:        D_x(v) = ∞
  per ogni vicino w e destinazione y: D_w(y) = ?
  invia D_x(·) iniziale a tutti i vicini

  repeat:
    se il costo verso un vicino si aggiorna:
      per ogni nodo y: D_x(y) = min_v { c(x, v) + D_v(y) }
      se D_x(y) è cambiato per qualche destinazione y:
        invia D_x(·) aggiornato a tutti i vicini
  until false // per sempre
```

La struttura dati $D_w(v)$ memorizza i vettori di distanza da tutti i nodi w verso le destinazioni v . In fase di **inizializzazione** sono impostate solo le distanze verso i vicini. Nella **fase online**, invece, si ricevono le novità dal vicinato, i vettori di distanza vengono aggiornati tramite l'equazione di Bellman e i cambiamenti locali vengono comunicati ai nodi adiacenti.

Esempio sul cammino u-z

1. All'inizio u conosce solo i vicini, quindi il costo verso z è infinito ($D_u(z) = \infty$).
2. y si "sveglia" e scopre un miglior cammino verso z , cioè il link diretto y - z di costo 2 ($D_y(z) = c(y, z) = 2$), e comunica la buona notizia a x .
3. x scopre che il miglior cammino verso z passa per y con costo 3 ($D_x(z) = c(x, y) + D_y(z) = 1 + 2$) e comunica la notizia a u .
4. u riceve la notizia: ora esiste un cammino verso z che costa meno di infinito, passando per x ($D_u(z) = c(u, x) + D_x(z) = 1 + 3$).

Lo stesso processo avviene per tutti i nodi e cammini della rete.

Complessità, pro e contro

A ogni aggiornamento del costo di un nodo, tutti i nodi rivalutano i propri archi, quindi la complessità è **quadratica nel caso peggiore**. Il vantaggio principale è che l'algoritmo è **asincrono**: i router possono adattarsi online, in qualunque fase, a costi aggiornati. I contro riguardano invece la **convergenza lenta**, poiché gli aggiornamenti si diffondono lentamente e tutti i nodi devono rilevare il cambiamento e comunicarlo agli adiacenti, e il **count-to-infinity**: i miglioramenti sono riconosciuti e adottati subito, ma i guasti vengono rilevati lentamente e possono produrre **loop**.

Il problema del count-to-infinity

Il problema nasce dalla natura **locale** dell'algoritmo DV, che è però anche la sua forza. Il vettore locale $D_x(y)$ è calcolato direttamente dai vettori degli altri nodi ($D_v(y)$), ma non c'è alcuna indicazione su quali nodi si trovino nei cammini migliori scelti dai nodi adiacenti. Ne deriva la domanda cruciale: posso essere sicuro di non essere nel "miglior cammino" dei miei vicini?

Esempio semplificato

Il link x-y subisce un improvviso **aumento di costo**, e **solo y se ne accorge**. Nella situazione iniziale, con $c(x,y) = 4$, i vettori sono:

- $D_y(x) = 4, D_y(z) = 1$;
- $D_x(y) = 4, D_x(z) = 4 + D_y(z) = 5$;
- $D_z(x) = 1 + D_y(x) = 5, D_z(y) = 1$.

I passi successivi mostrano il loop:

1. y rileva l'aumento e aggiorna il vettore verso x:

$$D_y(x) = \min\{60, 1 + D_z(x)\} = 1 + D_z(x) = 6$$

2. Poiché il vettore $D_y(x)$ è usato da z per calcolare il cammino verso x, anche $D_z(x)$ va aggiornato: $D_z(x) = 1 + D_y(x) = 7$.
3. A sua volta, $D_z(x)$ è usato da y per il proprio calcolo, quindi $D_y(x)$ va aggiornato di nuovo: $D_y(x) = 1 + D_z(x) = 8$.
4. Si instaura così un **loop**: i due vettori vengono aggiornati in continuazione, con distanze che crescono (5, 6, 7, 8, 9...) finché non si raggiunge una configurazione stabile alla convergenza.

Come evitarlo

Il problema si sarebbe evitato se anche **x avesse notato e comunicato** il cambio di costo del link x-y. Se x è malfunzionante o in ritardo — ad esempio, costi alti dei link di x possono indicare un suo malfunzionamento — la convergenza richiede tempo. Quando invece x rileva il cambiamento, la convergenza è corretta:

$$D_y(x) = \min\{c(y,x), c(y,z) + D_z(x)\} = 1 + 50 = 51$$

$$D_y(z) = \min\{c(y,z), c(y,x) + D_x(z)\} = 1$$

$$D_x(y) = \min\{c(x,y), c(x,z) + D_z(y)\} = 50 + 1 = 51$$

$$D_x(z) = \min\{c(x,z), c(x,y) + D_y(z)\} = 50$$

$$D_z(x) = \min\{c(z,x), c(z,y) + D_y(x)\} = 50$$

$$D_z(y) = \min\{c(z,y), c(z,x) + D_x(y)\} = 1$$

Lezione 21: Livello di rete — algoritmo Link-State

L'algoritmo Link-State (LS)

Il **Link-State** è un approccio **centralizzato** che sfrutta la conoscenza completa della rete per trovare il cammino migliore; si basa sull'**algoritmo di Dijkstra** e, poiché lo stato di tutti i link viene propagato, **non soffre del count-to-infinity**. Questo livello di conoscenza si ottiene in pratica facendo in modo che ogni nodo invii in broadcast **pacchetti link-state**, contenenti ID e costi dei propri link, a tutti gli altri nodi della rete. La versione base calcola i cammini minimi da un nodo di partenza verso tutte le possibili destinazioni, quindi va eseguito su tutti i nodi.

La fase di scambio dei messaggi

Prima di calcolare i cammini minimi, ogni router deve scoprire i nodi circostanti e condividere le informazioni. La fase si svolge in quattro passaggi rigorosi:

1. **Scoperta dei vicini:** il router invia un messaggio “hello” su tutti i link, e gli altri router rispondono con i propri ID.
2. **Impostazione dei costi:** il router imposta la distanza verso ciascun vicino, che può dipendere dai costi effettivi (tempo, banda) o essere impostata dall'amministratore.
3. **Costruzione del pacchetto:** il router crea un pacchetto che riassume gli ID dei nodi adiacenti e i relativi costi.
4. **Scambio dei pacchetti:** il router invia il proprio pacchetto in broadcast a tutti gli altri router e riceve pacchetti analoghi da tutti.

Completato il processo, ogni router possiede la conoscenza completa della topologia della rete e può quindi eseguire l'algoritmo dei cammini minimi (Dijkstra).

Pseudocodice

```

LinkState(x):
  N' = {x}
  per ogni nodo v:
    se v è vicino di x: D(v) = c(x,v)
    altrimenti:        D(v) = ∞
  repeat:
    trova w non in N' tale che D(w) è minimo
    aggiungi w a N'
    per ogni vicino v di w che non è in N':
      D(v) = min( D(v), D(w) + c(w,v) )
  until N' = N

```

La struttura dati **D(v)** memorizza la distanza dal nodo corrente verso tutte le destinazioni. In **inizializzazione** si assume che tutti i costi siano noti e vengono impostate solo le distanze verso i vicini. Nella **fase di costruzione** si seleziona il nodo più vicino w, si esplora il suo vicinato controllando se il cammino attraverso w è migliore di quello corrente, e si esce quando tutti i nodi sono stati esplorati.

Esempio (6 nodi, algoritmo invocato su u)

Si usa una funzione aggiuntiva **p(v)** per memorizzare il nodo che precede quello selezionato: per ricostruire il cammino basta risalire a ritroso tutti i predecessori fino al nodo corrente.

step	N'	D(v), p(v)	D(w), p(w)	D(x), p(x)	D(y), p(y)	D(z), p(z)
0	u	2, u	5, u	1, u	∞	∞
1	ux	2, u	4, x	—	2, x	∞

step	N'	D(v), p(v)	D(w), p(w)	D(x), p(x)	D(y), p(y)	D(z), p(z)
2	uxy	2, u	3, y	—	—	4, y
3	uxyv	—	3, y	—	—	4, y
4	uxyvw	—	—	—	—	4, y
5	uxyvwz	—	—	—	—	—

Al passo 0 si parte da u con i costi verso i vicini diretti; a ogni iterazione si aggiunge il nodo con distanza minima a N' e si aggiornano le distanze dei suoi vicini. Alla fine, risalendo i predecessori (ad esempio $z \leftarrow y \leftarrow x \leftarrow u$), si ottengono i cammini minimi.

Complessità, pro e contro

A ogni iterazione si controllano tutti i nodi non ancora in N'; poiché a ogni iterazione un nodo esce, le iterazioni totali sono $O(n^2)$ nel caso peggiore. Con strutture dati più sofisticate, come gli **heap**, le prestazioni migliorano fino a $O(n \log n)$. Il **pro** principale è la **convergenza più rapida**, poiché tutte le comunicazioni avvengono contemporaneamente, unito all'assenza di count-to-infinity. Il **contro** è che l'algoritmo è **sincrono**: i nodi devono ricevere le informazioni dall'intera rete prima di iniziare.

DV vs. LS

I due algoritmi adottano approcci **complementari**: DV sfrutta informazioni locali sui vicini, mentre LS richiede informazioni globali su tutti i nodi. Per quanto riguarda la **complessità dei messaggi**, LS è più complesso da implementare perché serve comunicazione sincrona tra tutti i nodi, mentre in DV la comunicazione avviene solo se cambia un miglior cammino. Per la **velocità di convergenza**, DV richiede tempo per convergere e nel frattempo si possono avere cammini subottimali; inoltre soffre del count-to-infinity. Per la **robustezza**, LS è considerato più robusto poiché le forwarding table sono calcolate separatamente: ogni nodo riceve le informazioni da tutti gli altri e crea la propria tabella. In DV, invece, tutti i calcoli sono a catena, quindi ogni nodo dipende dalle tabelle degli altri e se una tabella è sbagliata l'errore si propaga a tutte. Nel 1997, ad esempio, un router malfunzionante di un piccolo ISP causò una reazione a catena che inondò i router del backbone e disconnesse parte di Internet per diverse ore. Alla fine, entrambe le soluzioni sono usate in Internet.

Lezione 22: Livello di collegamento — Framing e gestione degli errori

Dal livello di rete ai livelli di collegamento e fisico

Il livello di rete fornisce fondamentalmente la comunicazione tra due host, ovunque si trovino, mentre i livelli di collegamento (*link*) e fisico forniscono la comunicazione tra **due host connessi direttamente**. Il **livello di collegamento** è la porzione dello stack dedicata alla trasmissione di pacchetti sui collegamenti (canali di trasmissione), da nodo a nodo della rete; il **livello fisico**, invece, regola la struttura dei collegamenti (mezzo trasmissivo, connettori, tipi di cavo) e il modo in cui i bit sono rappresentati e trasmessi lungo il collegamento. I due livelli sono **strettamente intrecciati** — spesso raggruppati in un unico livello detto *network access* — tanto che alcuni protocolli comuni, come Ethernet, coprono entrambi. A differenza dei livelli precedenti, questi due livelli sono **presenti in ogni componente** dell'infrastruttura di rete: nei nodi (host, router, switch, hub, access point WiFi) e nei collegamenti, sia cablati (rame, fibra ottica) sia wireless (radiofrequenze).

Per trasmettere un datagram IP bisogna trasferirlo dall'host sorgente all'host destinazione “saltando” di collegamento in collegamento lungo il percorso. Poiché ogni collegamento ha un proprio tipo, i **datagram vengono incapsulati in frame del livello di collegamento** specifici del collegamento su cui devono viaggiare. Questa è l'**ultima incapsulazione**: i frame vengono convertiti in segnali — impulsi elettrici, luce o onde radio — e trasferiti sul collegamento secondo la sua specifica. Ad esempio, per raggiungere Internet, i pacchetti di un host wireless devono attraversare 4/5 collegamenti (wireless e cablati), un access point, uno switch e 2/3 router.

Servizi del livello di collegamento

Il livello di collegamento può offrire fino a quattro servizi, tra loro indipendenti:

1. **Framing**: incapsulare i datagram in frame del livello di collegamento, con il datagram dei livelli superiori come payload e un header/trailer specifici del protocollo; la struttura del frame dipende dai protocolli dei livelli di collegamento e fisico.
2. **Accesso al collegamento** (*link access*): definire le regole che governano l'accesso al collegamento mediante un protocollo **Medium Access Control (MAC)**, in base all'architettura del collegamento. Nei **collegamenti punto-a-punto** (un solo mittente e un solo destinatario) il protocollo MAC è semplice o inesistente, poiché il mittente può inviare un frame quando il collegamento è inattivo; nei **collegamenti broadcast** esiste invece il *problema degli accessi multipli*, e il protocollo MAC serve a coordinare le trasmissioni dei frame dei molti nodi.
3. **Consegna affidabile** (*reliable delivery*): garantire che i frame trasmessi sul collegamento vengano ricevuti. Il servizio è spesso usato su collegamenti soggetti a un alto tasso di errore (es. wireless), con l'obiettivo di correggere l'errore localmente; poiché comporta un forte overhead e altri protocolli (es. TCP) sono già affidabili, non è sempre implementato.
4. **Rilevamento e correzione degli errori**: l'hardware del livello di collegamento può essere soggetto all'**inversione dei bit** (*bit flipping*), quindi molti protocolli implementano strategie di verifica e correzione incorporate nell'hardware, oltre a quelle dei livelli superiori (checksum di TCP e IP). Poiché non esistono ulteriori incapsulamenti, queste verifiche coprono praticamente l'intero messaggio e, essendo incorporate nell'hardware, possono essere molto più rapide.

Implementazione

Nei dispositivi di rete (sistemi finali, router, ecc.) le funzionalità del livello di collegamento sono implementate sia via software sia, soprattutto, via hardware. Nei computer esistono adattatori di rete detti **Network Interface Card (NIC)**, con un chip specializzato (il **controller**) che implementa le funzionalità del livello di collegamento. Un tempo queste schede erano separate dalla scheda madre e inserite in slot PCI; recentemente l'approccio sta cambiando verso la soluzione LAN-on-motherboard.

Software

Il software esegue sulla CPU e fornisce le funzionalità di alto livello: indirizzamento, gestione di interrupt e hardware, gestione degli errori, incapsulamento e decapsulamento dei datagram. Al livello di collegamento esiste un indirizzo aggiuntivo, l'**indirizzo MAC**, che identifica la NIC. I datagram devono inoltre essere memorizzati in memoria per essere usati dal protocollo di livello superiore: vengono recuperati dalla memoria durante l'incapsulamento (lato mittente) e inseriti in memoria durante il decapsulamento (lato ricevente).

Hardware

L'hardware funziona come un tipico dispositivo di I/O, convertendo i dati nel segnale da trasmettere in base al protocollo fisico (Ethernet, wireless, ecc.).

Tecnologie fisiche

Il protocollo del livello di collegamento, e quindi la struttura del frame risultante, dipende dalla tecnologia del collegamento fisico. Le tecnologie comuni sono tre:

- **Ethernet 802.3**: connessione cablata basata su impulsi elettrici su 4 doppini intrecciati in rame (la più comune) oppure su fotoni tramite cavo in fibra ottica.
- **WiFi 802.11**: connessione wireless basata su radiofrequenza (2,4 GHz, 5 GHz, 6 GHz).
- **Bluetooth**: connessione wireless basata su radiofrequenza (2,4 GHz).

Le tecnologie fisiche evolvono continuamente e i loro limiti in termini di distanza e banda vengono aggiornati spesso; ogni tecnologia ha pro e contro:

Tipo	Segnale	Distanza	Banda	Criticità
Wireless	Radio	30–50 m	1,3 Gbps	Sicurezza, velocità
Doppino in rame	Elettricità	30–100 m	1–25 Gbps	Interferenze
Fibra ottica	Luce	100–200 km	1–400+ Gbps	Fragilità

Frame (richiamo)

Il livello di collegamento accetta i pacchetti dal livello di rete, li incapsula in frame e li invia tramite il livello fisico; la ricezione è il processo opposto.

Metodi di framing

Gli approcci possibili, logici o fisici, sono quattro:

- Conteggio di byte (*byte count*).
- Byte di flag con *byte stuffing*.
- Bit di flag con *bit stuffing*.
- Violazioni della codifica del livello fisico (uso di simboli non-dato per indicare il frame).

Conteggio di byte (mai usato)

Ogni frame inizia con un **contatore del numero di byte** che contiene. L'approccio è semplice, ma dopo un errore è difficile risincronizzarsi: poiché il ricevitore usa il contatore per delimitare i frame, se il contatore viene corrotto perde l'allineamento anche sui frame successivi.

Byte stuffing

Un **byte di flag speciale** delimita i frame, e le occorrenze del flag nei dati devono essere “riempite” o mascherate (*stuffed*) tramite byte di escape. I frame risultano più lunghi, ma dopo un errore è facile risincronizzarsi, perché il ricevitore cerca semplicemente il flag successivo.

Formato del frame:

FLAG	Header		Campo payload	Trailer	FLAG		
------	--------	--	---------------	---------	------	--	--

Esempi (ESC = byte di escape):

Byte originali			→	Byte dopo stuffing			
A	FLAG	B	→	A	ESC	FLAG	B
A	ESC	B	→	A	ESC	ESC	B
A	ESC	FLAG	B	→	A	ESC	ESC

Bit stuffing

Il flag del frame è la sequenza di **sei 1 consecutivi** (01111110). In trasmissione, dopo **cinque 1** nei dati viene aggiunto uno **0**; in ricezione, uno **0** dopo cinque 1 viene **eliminato**, ricostruendo i dati originali.

Violazione fisica

Questo metodo si affida alla codifica della trasmissione fisica, usando un byte (o parola) **non-dato** al livello fisico. Poiché i segnali delimitatori non possono apparire nei dati, le violazioni della codifica delimitano i frame e non serve alcuno stuffing.

Rilevamento e correzione degli errori

Formulazione del problema

A livello di collegamento è possibile eseguire **rilevamento e correzione degli errori a livello di bit**. Sia D , di dimensione d , il dato da proteggere: per difenderlo da errori di bit si includono nel messaggio alcuni bit di rilevamento e correzione, detti **EDC**. L'obiettivo è capire se i D' e EDC' ricevuti differiscono dagli originali D ed EDC e, in tal caso, eventualmente recuperare gli originali. Bisogna tuttavia ricordare che, anche usando bit di rilevamento, possono comunque esserci errori non rilevati (in casi “patologici”).

Parity check

Il **controllo di parità** è forse la forma più semplice di rilevamento degli errori: si aggiunge **un solo bit di parità** al messaggio, tale che il numero totale di 1 tra gli $d + 1$ bit sia pari (schema a parità pari) o dispari (schema a parità dispari).

Messaggio	Bit di parità (pari)	Bit di parità (dispari)
1 0 1 0 1 1 0 0	0	1
1 1 0 1 0 0 0 0	1	0

Il ricevitore deve solo contare il numero di 1 nei $d + 1$ bit ricevuti: se con schema a parità pari trova un numero dispari di bit a 1 (o viceversa), sa che è avvenuto almeno un errore. L'approccio funziona quindi solo con un numero dispari di errori. Se la probabilità di errore sui bit fosse piccola e gli errori fossero indipendenti l'uno dall'altro, la probabilità di errori multipli sarebbe minima — ma non è il caso delle reti di calcolatori: in pratica gli errori tendono a **raggrupparsi in “burst”**. In condizioni di burst error la probabilità di errori non rilevati con la parità a bit singolo può avvicinarsi al **50%**, quindi la tecnica non è davvero utile.

Parity check bidimensionale

Un modo per migliorare l'approccio è usare più bit di parità. La **parità bidimensionale** divide il messaggio in n righe e m colonne, ciascuna con un proprio bit di parità. In questo modo il ricevitore può rilevare l'errore e individuare il bit errato, potendo quindi tentare la correzione. La parità bidimensionale può inoltre rilevare (ma non correggere) qualsiasi combinazione di due errori in un pacchetto.

Cyclic Redundancy Check (CRC)

Il **CRC** è una tecnica di rilevamento errori molto usata. Mittente e destinatario concordano un **pattern di $r + 1$ bit chiamato generatore (G)**, con il bit più significativo (il più a sinistra) a 1. Per un dato D di d bit, il mittente sceglie r **bit aggiuntivi (bit CRC)** da appendere al messaggio, in modo che il messaggio M — cioè D concatenata con i bit CRC — sia divisibile modulo-2 per G (resto = 0). Il ricevitore divide M per G : se il resto è zero il messaggio è corretto, altrimenti si assume un errore. L'operazione si implementa spostando G in corrispondenza dell'1 più significativo del messaggio ed eseguendo uno XOR bit-a-bit.

Esempio

Con un messaggio di 6 bit e un CRC di 3 bit: $d = 6$, $D = 101110$, $r = 3$, $G = 1001$.

Lato mittente si calcola il CRC da D e G , tramite la divisione modulo-2 di D seguito da r zeri per G : gli ultimi r bit del resto sono i bit CRC, che vengono attaccati al messaggio e trasmessi. **Lato ricevente** si divide per G il messaggio ricevuto ($D + \text{CRC}$): se il resto è 0, il messaggio è intatto.

Standard e prestazioni

Sono stati definiti **standard internazionali** per generatori a 8, 12, 16 e 32 bit (cioè $r = 8, 12, 16, 32$). Lo **standard CRC-32 a 32 bit**, adottato in diversi protocolli IEEE di livello di collegamento, usa il seguente generatore (33 bit):

$$G_{\text{CRC-32}} = 100000100110000010001110110110111$$

Ogni standard CRC rileva con certezza burst error di meno di $r + 1$ bit; per errori più lunghi di $r + 1$ esiste una probabilità P di individuare l'errore:

$$P = 1 - 0.5^r$$

La probabilità di trovare l'errore cresce quindi con r .

Distanza di Hamming

Un codice di correzione degli errori trasforma dati di n bit in **codeword di $n + k$ bit**. La **distanza di Hamming** è il minimo numero di inversioni di bit necessarie per trasformare una codeword valida in un'altra qualsiasi codeword valida.

Esempio con 4 codeword di 10 bit ($n = 2$, $k = 8$):

```
0000000000, 0000011111, 1111100000, 1111111111
```

La distanza di Hamming è **5**. Per un codice con distanza d valgono i seguenti limiti:

- $2d + 1$ bit permettono di **correggere** d errori (nell'esempio, 2 errori).
- $d + 1$ bit permettono di **rilevare** d errori (nell'esempio, 4 errori).

I **codici di Hamming** si basano su questo principio, ma sono più complessi.

Lezione 23: Livello di collegamento — Sotto-livello MAC

Richiamo: servizi del livello di collegamento

Il livello di collegamento può offrire fino a quattro servizi, tra loro indipendenti:

1. **Framing**: incapsulare/decapsulare i datagram in/da frame.
2. **Accesso al collegamento**: regolare l'accesso ai collegamenti condivisi (broadcast), compito del protocollo **Medium Access Control (MAC)**.
3. **Consegna affidabile**: garantire che i frame trasmessi sul collegamento vengano ricevuti (per lo più opzionale).
4. **Rilevamento e correzione degli errori**: rilevare gli errori nei frame ed eventualmente correggerli (parity check, checksum, CRC).

Tipi di collegamento

Esistono fondamentalmente due tipi di collegamento. Nel **collegamento punto-a-punto** vi è un solo mittente a un'estremità e un solo ricevitore all'altra; il **Point-to-Point Protocol (PPP)** è un esempio di protocollo che gestisce tali collegamenti (es. collegamento Ethernet diretto tra due computer). Nel **collegamento broadcast**, invece, più nodi trasmettenti e riceventi sono connessi allo stesso canale condiviso: si dice broadcast perché, quando un nodo trasmette un frame, questo viene ricevuto da tutti i nodi sul canale (bus Ethernet, Ethernet half-duplex — raro, oggi la maggior parte dei cavi è full-duplex — o LAN wireless). Poiché più comunicazioni sullo stesso collegamento possono interferire tra loro, l'accesso ai collegamenti broadcast deve essere coordinato: è il cosiddetto **problema degli accessi multipli**.

Collisioni

Il problema principale del collegamento broadcast è la **collisione**: se più nodi trasmettono simultaneamente frame sullo stesso canale, tali frame si sovrappongono diventando inintelligibili. I frame collisi vengono ricevuti da tutti i nodi del canale e scartati come errori (nessun danno), però tutti i frame trasmessi sono persi e l'intervallo di tempo è sprecato, poiché il canale è stato usato per trasmettere dati inutili. L'accesso al mezzo fisico, così come il framing, è quindi regolato dal protocollo **MAC (Medium Access Control)**.

Protocolli di accesso multiplo

Nelle reti di calcolatori i **protocolli di accesso multiplo** (*multiple access*) regolano le trasmissioni sui canali broadcast, così che le collisioni siano gestite, ogni nodo abbia una possibilità di trasmettere — quindi nessun nodo monopolizzi il collegamento — e le connessioni stabilite non vengano interrotte, ad esempio verificando se il collegamento è occupato. Sono necessari in vari contesti, cablati, wireless o satellitari, dove centinaia o migliaia di nodi comunicano direttamente su un canale broadcast.

Ruolo e approcci

Il ruolo primario di un protocollo di accesso multiplo è in qualche modo **evitare le collisioni**, e ne esistono decine per le diverse tecnologie. Gli approcci principali sono tre:

- **Partizionamento del canale** (*channel partitioning*): la banda è partizionata tra i diversi nodi.
- **Accesso casuale** (*random access*): i nodi “scommettono” per l'accesso.
- **Turni** (*taking-turns*): i nodi aspettano il proprio turno.

Desiderata

Un protocollo di accesso multiplo per un **canale broadcast di velocità R bps** dovrebbe avere le seguenti proprietà:

- **Massimizzare l'uso del canale:** se M nodi hanno dati da inviare, ciascuno dovrebbe avere in media un throughput di R/M bps (se $M=1$, il throughput dovrebbe essere R).
- **Essere decentralizzato:** un nodo master sarebbe un singolo punto di guasto.
- **Essere semplice e leggero:** vengono inviati tantissimi frame, quindi non ci deve essere overhead.

Protocolli a partizionamento del canale

TDMA (Time Division Multiple Access)

Considerando un canale con N nodi e velocità di trasmissione R bps, il TDMA divide il tempo in **frame temporali** (step) e divide ulteriormente ogni frame temporale in **N slot** temporali. Ogni slot è assegnato a uno degli N nodi, quindi quando un nodo ha un pacchetto da inviare aspetta il proprio slot; tipicamente gli slot sono dimensionati così che un intero pacchetto possa essere trasmesso durante uno slot.

FDMA (Frequency Division Multiple Access)

Il FDMA divide il canale da R bps in diverse frequenze, ciascuna con banda R/N , e assegna ogni frequenza a uno degli N nodi. FDMA e TDMA condividono pro e contro: evitano le collisioni e dividono equamente la banda tra gli N nodi, però un nodo è limitato alla banda R/N anche quando è l'unico nodo con pacchetti da inviare.

CDMA (Code Division Multiple Access)

Il CDMA assegna a ogni nodo un **codice diverso** usato per codificare e decodificare i bit dei dati inviati. Se i codici sono scelti con cura, le reti CDMA permettono a nodi diversi di trasmettere simultaneamente senza interferenze. Funziona soprattutto su canali wireless; è stato usato in sistemi militari per le sue proprietà anti-jamming e anche nella telefonia cellulare (es. 3G). Le sue criticità sono la complessità di implementazione e il fatto che non scala bene con il numero di nodi.

Protocolli ad accesso casuale

Nei protocolli ad accesso casuale un nodo trasmittente **trasmette sempre alla velocità piena** del canale (R bps). Se avviene una collisione, cioè se almeno due nodi stanno trasmettendo, tutti i nodi trasmittenti aspettano un **ritardo casuale** prima di ritentare la ritrasmissione. Poiché la scelta è fatta indipendentemente, è possibile che i due nodi scelgano ritardi sufficientemente diversi, permettendo a uno dei due di far passare il messaggio; se invece viene scelto un ritardo simile, avviene una nuova collisione e il processo è iterato.

Slotted ALOHA

Il protocollo **slotted ALOHA** si basa su quattro regole:

- Il **tempo è diviso in slot** (come in TDMA), ciascuno abbastanza grande da contenere un frame.
- I nodi sono sincronizzati: ogni nodo trasmette i frame solo all'inizio di uno slot.
- Se nessuna collisione è rilevata nello slot, la comunicazione prosegue.
- Se viene rilevata una collisione, ogni nodo in collisione ha una probabilità $p \in [0, 1]$ di reinviare il frame in ciascuno degli slot successivi, finché il frame non è inviato con successo.

Le sue caratteristiche: se c'è un solo nodo, questo usa l'intera velocità R del canale senza collisioni; è **decentralizzato**, poiché i nodi sono totalmente indipendenti (a parte la sincronizzazione); è semplice da implementare ed eseguire, dato che la selezione casuale è rapidissima; ed è improbabile avere collisioni consecutive multiple.

CSMA e CSMA/CD

Una debolezza di ALOHA è che si può iniziare a trasmettere — producendo una collisione — anche se il canale è già occupato, e lo si capisce solo dall'effetto delle collisioni. Una soluzione è **monitorare il canale** e tentare la trasmissione solo se è inattivo. I protocolli **CSMA (Carrier Sense Multiple Access)** e **CSMA con rilevamento delle collisioni (CSMA/CD)** si basano su due principi:

- **Carrier sensing:** i nodi ascoltano il canale prima di trasmettere; se è in corso la trasmissione di un frame di un altro nodo, aspettano finché non viene rilevata nessuna trasmissione.
- **Collision detection:** i nodi ascoltano il canale mentre trasmettono; se rilevano una collisione, interrompono la trasmissione e aspettano un tempo casuale prima di ripartire.

Resta una domanda chiave: se tutti i nodi fanno carrier sensing, perché le collisioni avvengono comunque? La risposta, legata al ritardo di propagazione, è nella lezione successiva.

Lezione 24: Sotto-livello MAC, accesso casuale, protocolli senza collisioni, switch e ARP

Protocolli ad accesso casuale: CSMA

Perché le collisioni avvengono nonostante il carrier sensing?

La causa è il **ritardo nella trasmissione del segnale**. Anche se la propagazione dei segnali nel canale avviene tipicamente vicino alla velocità della luce, serve tempo per raggiungere tutti gli altri nodi: un secondo nodo rileva la trasmissione solo dopo che è iniziata. Per questo ritardo tra inizio della trasmissione e rilevamento, un nodo può ritenere libero un canale in realtà già in uso, producendo una collisione.

CSMA puro

1. Il nodo verifica se il canale è occupato.
2. Se il canale è inattivo, invia un frame.

Nel CSMA puro le collisioni non vengono rilevate ma restano possibili: si capisce che un frame è perso solo perché l'**ACK non arriva**.

CSMA/CD (implementato attualmente su Ethernet)

1. Il nodo verifica se il canale è occupato.
2. Se il canale è inattivo, invia un frame.
3. Mentre trasmette, controlla eventuali collisioni.
4. Se rileva una collisione, interrompe la trasmissione e aspetta un periodo casuale $K \in \{0, \dots, 2^n - 1\}$, dove n è il numero di collisioni rilevate sul frame corrente (**binary exponential backoff**).

Grazie al backoff, la probabilità di inviare con successo il frame cresce con il numero di collisioni.

Protocolli a turni (taking-turns)

Protocollo di polling

Esiste un **nodo master che seleziona, in modalità round-robin, un nodo alla volta** autorizzato a trasmettere (fino al throughput massimo); il processo è iterato ogni volta che la trasmissione si ferma (es. Bluetooth). Il pro è l'assenza di collisioni; i contro sono il **ritardo di polling** (il tempo per selezionare i nodi) e l'approccio **centralizzato**, che introduce un singolo punto di guasto.

Protocollo token-passing

Non c'è un nodo master: i nodi si scambiano un **frame speciale chiamato token**, e chi lo riceve è autorizzato a trasmettere, per poi passarlo a un altro nodo. I pro sono l'assenza di collisioni e l'approccio **decentralizzato**; il contro è che sorgono problemi se un nodo dimentica di rilasciare il token, monopolizzando il collegamento.

Indirizzi MAC

A livello di collegamento i dispositivi sono identificati dagli **indirizzi MAC**: ogni interfaccia di rete ha un MAC specifico (progettato per essere fisso, ma cambiabile) e ogni produttore possiede il proprio insieme di MAC. L'**indirizzo MAC** (o indirizzo fisico) è un indirizzo di livello collegamento composto da **6 byte** (2^8 indirizzi possibili), spesso in notazione esadecimale:

1A:23:F9:CD:06:9B oppure 1A-23-F9-CD-06-9B

Gli indirizzi MAC sono **locali**, mentre gli IP, fuori dalle LAN, sono globali: tutte le interfacce hanno un MAC, ma questo viene usato solo all'interno di una LAN.

Comunicazione in una LAN

In una LAN, due interfacce A e B comunicano così: A inserisce il MAC di B nel frame e lo trasmette; B riceve il frame e confronta il proprio MAC con il MAC di destinazione del frame; se i due MAC coincidono, il frame è accettato, altrimenti è **scartato** senza coinvolgere il resto dello stack. Esiste anche la possibilità di inviare **messaggi broadcast**, accettati da qualunque MAC: per le LAN con indirizzi da 6 byte (es. Ethernet e 802.11) l'indirizzo broadcast è una stringa di 48 uno consecutivi, cioè FF-FF-FF-FF-FF-FF. Poiché in una LAN è tutt'altro che raro che un'interfaccia riceva frame diretti ad altre, il ruolo dell'indirizzo MAC è di **filtrare i frame non intesi** senza disturbare l'host.

Switch

Gli switch sono l'**equivalente di livello collegamento dei router**: non implementano alcun algoritmo di routing e usano **solo indirizzi MAC**, non gli IP. Il loro ruolo è ricevere i frame in arrivo sui collegamenti in ingresso e inoltrarli sui collegamenti in uscita. Lo switch è **trasparente** agli host e ai router della sottorete, ha **buffer sulle interfacce** e possiede **tabelle di inoltro** che associano indirizzi MAC a interfacce; la tabella viene aggiornata **automaticamente e dinamicamente** (*self-learning*) man mano che si scoprono nuovi dispositivi.

LAN con switch

Le LAN usano in genere uno o più switch per collegare più dispositivi locali. A differenza dei router, gli switch sono **veloci e plug-and-play**, poiché non eseguono alcun algoritmo di routing e coinvolgono solo due livelli dello stack. D'altro canto, le LAN con switch sono **limitate in dimensione** e devono avere **struttura ad albero**: i MAC sono difficili da raggruppare (le tabelle di inoltro possono crescere rapidamente) e, non essendoci routing, i loop sono difficili da evitare.

ARP (Address Resolution Protocol)

Poiché i protocolli dei livelli superiori lavorano con indirizzi IP, è necessario tradurre IP in MAC: se ne occupa l'**Address Resolution Protocol (ARP)**. Ogni interfaccia ha un **modulo ARP** con una **tabella ARP** che associa ogni IP della LAN a un MAC, con un **tempo di vita (TTL)** dopo il quale la voce viene eliminata (tipicamente 5-20 minuti).

Esempio di tabella ARP:

Indirizzo IP	Indirizzo MAC	TTL
222.222.222.221	88-B2-2F-54-1A-0F	13:45:00
222.222.222.223	5C-66-AB-90-75-B1	13:52:00

Poiché i MAC sono locali, anche ARP funziona solo su reti locali (LAN).

Esempio: da C verso A

L'host C (222.222.222.220) vuole inviare un messaggio all'host A (222.222.222.222) e gli serve conoscere anche il MAC associato. Prima di inviare, se A non è presente in tabella, C invia un pacchetto di **ARP request in broadcast** a tutti i dispositivi della rete, cercando l'IP giusto. Tutti i nodi ricevono il pacchetto, ma **solo l'IP cercato risponde** con un messaggio diretto di **ARP reply** (unicast). Un host malevolo può intercettare la richiesta e rispondere con una risposta "contraffatta" (**ARP poisoning**), uno degli attacchi più comuni sulle LAN.

ARP e gateway (IP non locale)

Se l'IP è fuori dalla rete, il meccanismo passa dal router: il router che collega le due reti deve avere **almeno due interfacce** (2 IP, 2 MAC e 2 tabelle ARP), una dentro ciascuna sottorete. I frame diretti fuori dalla sottorete vengono quindi inviati alla prima interfaccia del router, spostati alla seconda interfaccia e dirottati verso l'host giusto usando la seconda tabella ARP.

Frame Ethernet

Il frame **Ethernet** è composto dai seguenti campi:

Campo	Dimensione	Descrizione
Preamble	8 byte	Blocco di bit di “sveglia” usato per sincronizzare gli orologi degli adattatori sorgente e destinazione
Destination address	6 byte	MAC dell'adattatore di destinazione
Source address	6 byte	MAC dell'adattatore sorgente che trasmette il frame sulla LAN
Type field	2 byte	Specifica il protocollo di livello rete usato nel frame (potrebbero esserci alternative a IP: ad es. i pacchetti ARP hanno un tipo specifico — 0x0806)
Data field	46–1500 byte	Contiene il datagram IP; il limite massimo è dato dalla MTU di Ethernet: se il datagram la supera, viene frammentato
CRC	4 byte	Contiene il numero di CRC

Lezione 25: Sicurezza di rete — Fondamenti

Richiamo: alcuni protocolli applicativi

Alcune applicazioni scambiano **informazioni sensibili** (credenziali utente, dati personali); poiché viaggiano su reti pubbliche, i messaggi devono essere protetti.

Applicazione	Descrizione
DHCP	Dynamic Host Configuration Protocol, assegna indirizzi IP
DNS	Domain Name System, traduce i nomi dei siti in indirizzi IP
HTTP/HTTPS	HyperText Transfer Protocol (Secure), trasferisce pagine web
SMTP/SMTS	Simple Mail Transfer Protocol (Secure), invia messaggi e-mail
SNMP	Simple Network Management Protocol, gestisce dispositivi di rete
Telnet/SSH	Teletype Network (Secure SHell), interfaccia a riga di comando con host remoti
FTP/FTPS	File Transfer Protocol (Secure), usato per trasferire file

Introduzione

La **sicurezza di rete** è il campo che studia i possibili attacchi alle reti e i modi di prevenirli. Poiché il networking è ormai parte della nostra vita — la connessione Internet è considerata quasi come acqua o elettricità — una grande quantità di dati sensibili viaggia sulle reti. Esistono diversi tipi di attacchi, con scopi e meccanismi differenti, che evolvono insieme alla tecnologia e all’audience delle reti.

Attacchi: malware

Un **malware** è un software dannoso che può essere trasferito su un computer tramite la rete (file scaricato, allegato e-mail, ecc.). Una volta infettato il dispositivo, può danneggiarlo in diversi modi: forzare un sistema a mostrare pubblicità commerciali (**adware**); mostrare falsi messaggi di allarme per indurre l’utente a scaricare malware (**scareware**); cancellare (**wiper**) o cifrare (**ransomware**) i file; raccogliere informazioni private come password o numeri di sicurezza (**spyware**) — ad esempio i **keylogger** registrano i tasti premuti sulla tastiera; ottenere privilegi di root (**rootkit**); trasformare il dispositivo in uno “schiavo” o punto d’appoggio per attaccare altri dispositivi (**zombie** o **botnet**).

I malware odierni sono spesso **auto-replicanti**: infettato un host, da quello cercano di entrare in altri host su Internet, e da questi in ancora altri. Possono diffondersi come virus o worm. I **virus** richiedono una qualche interazione dell’utente per infettare il dispositivo (es. un allegato e-mail con codice eseguibile dannoso che si auto-replica inviando mail simili ai contatti) e sono tipicamente mascherati da software legittimo (**cavalli di Troia**). I **worm**, invece, entrano nel dispositivo senza alcuna interazione esplicita dell’utente: un’applicazione di rete vulnerabile accetta il worm senza intervento, e il worm poi scansiona la rete in cerca di host con la stessa applicazione.

Attacchi: DoS

Gli attacchi **Denial-of-Service (DoS)** sono molto comuni e mirano a rendere inutilizzabile una rete, un host o un altro componente dell'infrastruttura (server web, DNS, ecc.) agli utenti legittimi. Esistono tre tipi di attacco DoS:

1. **Vulnerability attack**: invio di messaggi ad hoc ad applicazioni o sistemi operativi vulnerabili per farli bloccare o crashare.
2. **Bandwidth flooding**: invio di un'enorme quantità di pacchetti all'host bersaglio, impedendo ai pacchetti legittimi di raggiungere il server.
3. **Connection flooding**: apertura di un gran numero di connessioni TCP half-open o completamente aperte sull'host bersaglio, che smette di accettare connessioni legittime.

Una variante è il **DDoS** (Distributed DoS), che usa una **botnet** di zombie.

Attacchi: packet sniffing

Il **packet sniffing** prevede un ricevitore passivo (**sniffer**) che registra una copia dei pacchetti rilevanti da un host bersaglio per rubare informazioni sensibili (detto anche **eavesdropping**). Gli sniffer possono essere installati in ogni tipo di rete **broadcast**, cablata o wireless, semplicemente copiando i pacchetti destinati ad altri invece di scartarli. Nelle reti **non broadcast**, invece, uno sniffer può essere incorporato in un malware (spyware) e usato per infettare dispositivi di rete come i router, così che tutto il traffico inoltrato viene copiato. Poiché sono **passivi** — non iniettano alcun traffico aggiuntivo — gli sniffer sono molto difficili da rilevare. La contromisura principale è la **crittografia**. Un esempio noto di sniffer è **Wireshark** (ex Ethereal).

Attacchi: IP spoofing

L'**IP spoofing** consente a host malevoli di iniettare in rete pacchetti con **indirizzi sorgente falsificati**. Può essere usato in combinazione con vulnerabilità delle applicazioni per attaccare host specifici mascherandosi da un altro utente, oppure per attacchi DoS come alternativa alle botnet: poiché i messaggi provengono da IP sorgente diversi, sono più difficili da filtrare. Spoofing e sniffing possono inoltre servire per attacchi **man-in-the-middle** (MitM): l'attaccante si pone tra due host in comunicazione spacciandosi per entrambi, quindi i due host credono di parlare tra loro ma comunicano in realtà con l'attaccante. Le contromisure sono le **verifiche di integrità dei messaggi** e l'**autenticazione degli endpoint**, che permettono di capire se il messaggio è stato modificato e se proviene dalla fonte giusta.

Fondamenti della sicurezza di rete

Una **comunicazione sicura** dovrebbe garantire quattro proprietà:

- **Riservatezza** (*confidentiality*): solo mittente e destinatario previsto devono poter comprendere il contenuto del messaggio (contro lo sniffing).
- **Integrità dei messaggi**: il contenuto della comunicazione non deve essere alterato, né dolosamente né accidentalmente.
- **Autenticazione degli endpoint**: mittente e destinatario devono poter confermare l'identità dell'altra parte (contro lo spoofing).
- **Sicurezza operativa**: poter contare su un'infrastruttura di rete che impedisce a host malevoli di intrufolarsi nella comunicazione.

Le prime tre proprietà sono **software-based**, mentre la sicurezza operativa si basa tipicamente su **hardware dedicato** (firewall, intrusion detection systems).

Crittografia

Una **tecnica crittografica** consente a un mittente di camuffare i dati rendendoli incomprensibili a un intruso, così che l'intruso non ricavi informazioni dai dati intercettati, mentre il destinatario deve poter recuperare i dati originali. Nella forma iniziale il messaggio si chiama **plaintext** (o cleartext), leggibile da chiunque; prima

di immetterlo nel canale, l'host applica un algoritmo di cifratura che lo trasforma in una forma illeggibile, il **ciphertext**, che va decifrato alla ricezione.

Chiavi

In molti sistemi crittografici moderni, inclusi quelli usati su Internet, la tecnica di cifratura è nota e standard per tutti, intruso compreso: la parte sconosciuta sono le **chiavi** di cifratura e decifratura. Una **chiave** è una stringa alfanumerica fornita all'algoritmo di cifratura/decifratura; le chiavi possono essere identiche (crittografia simmetrica) o diverse (asimmetrica).

Crittografia simmetrica

Nella crittografia simmetrica esiste **una sola chiave** usata sia per cifrare che per decifrare. In cifratura, il plaintext e la chiave vengono dati all'algoritmo di cifratura, generando un ciphertext da inviare in rete in sicurezza; in decifratura, il ciphertext e la chiave vengono dati all'algoritmo di decifratura per ricostruire il plaintext iniziale. Il limite principale è il **problema dello scambio delle chiavi** (*key exchange problem*): la chiave segreta deve essere comunicata in qualche modo, usando un canale sicuro oppure un protocollo che permetta di “convergere” su una chiave condivisa. Se due parti non riescono a fare uno scambio iniziale sicuro, non potranno comunicare in sicurezza, poiché una terza parte che ottiene la chiave durante lo scambio può intercettare e decifrare i messaggi.

Crittografia asimmetrica

La crittografia asimmetrica è un sistema a **due chiavi**, pubblica e privata: un messaggio cifrato con una chiave va decifrato con l'altra, e viceversa. La **chiave pubblica** può essere trasmessa su canali non sicuri o condivisa pubblicamente, mentre la **chiave privata** è disponibile solo al proprietario. L'approccio tipico è usare la **chiave pubblica per cifrare e la chiave privata per decifrare**: se l'intruso intercetta la chiave pubblica, non può comunque decifrare i messaggi; A usa infatti la chiave pubblica di B per cifrare messaggi leggibili solo tramite la chiave privata di B, che è solo nelle mani di B.

Certification Authority (CA)

Nella crittografia a chiave pubblica è utile **verificare che una chiave pubblica appartenga davvero all'entità** con cui si vuole comunicare: altrimenti potremmo avere la chiave di un attaccante e cifrare messaggi leggibili da entità illegittime. Il legame tra chiave pubblica ed entità è stabilito da una **Certification Authority (CA)**, il cui lavoro è validare le identità ed emettere certificati:

1. La CA **verifica che un'entità sia chi dice di essere**: non esiste un protocollo per questo, quindi bisogna fidarsi della CA — funziona come una selezione naturale, poiché una CA inaffidabile non è più fidata. Esistono diverse CA federali e statali ragionevolmente affidabili, ma resta necessario fidarsi.
2. Verificata l'identità, la CA **crea un certificato che lega la chiave pubblica dell'entità alla sua identità**: il certificato contiene la chiave pubblica e un identificatore globalmente univoco del proprietario (es. un nome o un indirizzo IP).

Integrità dei messaggi

L'**integrità dei messaggi** (o **autenticazione dei messaggi**) è il problema di verificare che il messaggio non sia stato manomesso e che provenga davvero dall'host atteso. Si può creare un “codice di controllo” in modo analogo a checksum o CRC; tipicamente si usa una **funzione hash**. Una funzione hash mappa dati di dimensione arbitraria in valori a dimensione fissa; una **funzione hash crittografica** è una funzione H che converte un messaggio x in una stringa a dimensione fissa $H(x)$, tale che sia computazionalmente improbabile trovare un altro messaggio y con $H(y) = H(x)$.

Hash semplice (approccio fallace)

1. A crea il messaggio m e calcola l'hash $h = H(m)$.
2. A appende h al messaggio, creando il messaggio esteso (m, h) , e lo invia a B.
3. B riceve (m, h) e calcola $H(m)$: se $H(m) = h$, il messaggio è intatto.

L'approccio è ovviamente fallace: un intruso può falsificare l'intero messaggio, creandone uno “ad hoc” (m', h') comunque consistente con H .

Message Authentication Code (MAC)

Per evitare ciò, A e B hanno bisogno di un **segreto condiviso** s (**chiave condivisa** o **password**), noto solo a loro — di fatto funziona come una cifratura simmetrica dove s è l'unica chiave privata. La procedura è la seguente:

1. A crea il messaggio $m + s$ (concatenazione di messaggio e segreto) e calcola l'hash $h = H(m + s)$, detto **message authentication code (MAC)**.
2. A appende il MAC al messaggio, creando il messaggio esteso $(m, H(m + s))$, e lo invia a B.
3. B riceve (m, h) e, conoscendo s , calcola il MAC $H(m + s)$: se $H(m + s) = h$, il messaggio è intatto.

Come in tutti gli approcci simmetrici, bisogna scambiare il segreto; lo scambio può avvenire combinando crittografia asimmetrica e certificati.

Lezione 26: Sicurezza di rete — Autenticazione, SSL, firewall

Richiamo: fondamenti della sicurezza di rete

Una **comunicazione sicura** dovrebbe garantire quattro proprietà:

- **Riservatezza**: solo mittente e destinatario previsto devono poter comprendere il contenuto del messaggio (contro lo sniffing).
- **Integrità dei messaggi**: il contenuto della comunicazione non deve essere alterato, né dolosamente né accidentalmente.
- **Autenticazione degli endpoint**: mittente e destinatario devono poter confermare l'identità dell'altra parte (contro lo spoofing).
- **Sicurezza operativa**: poter contare su un'infrastruttura di rete che impedisce a host malevoli di intrufolarsi nella comunicazione.

Le prime tre proprietà sono **software-based**, mentre la sicurezza operativa si basa tipicamente su **hardware dedicato** (firewall, intrusion detection systems).

Autenticazione degli endpoint

L'autenticazione degli endpoint è il processo con cui **un'entità prova la propria identità a un'altra entità** su una rete di calcolatori. Tipicamente un **protocollo di autenticazione (AP)** viene eseguito prima che le due parti eseguano un altro protocollo (trasferimento dati affidabile su TCP, scambio di informazioni di routing DV/LS, e-mail con SMTP, ecc.), così da stabilire le identità prima di iniziare a lavorare. Si può fare affidamento sulla **Certification Authority (CA)**, ma non basta.

Autenticazione tramite indirizzo IP

Se A e B hanno già comunicato e l'IP di A non è cambiato, B può autenticare A controllando l'**indirizzo IP** nel datagram: se è quello “noto”, si assume che sia A a inviare il messaggio. Questo funziona con intrusi “ingenui”, ma è chiaramente insufficiente, poiché un intruso “meno ingenuo” può falsificare l'indirizzo nel pacchetto (spoofing). Un modo per evitare l'IP spoofing è agire sui router: un router può essere configurato per inoltrare solo datagram legittimi, cioè con IP sorgenti che appartengono davvero agli host della rete, scartando i datagram falsificati. Purtroppo questa capacità non è diffusa ovunque né imposta, quindi bisogna assumere lo spoofing come possibile.

Autenticazione tramite password

L'approccio più comune è usare una **password segreta**, che funziona come **segreto condiviso** tra autenticatore e autenticando (Gmail, Facebook, telnet, FTP e molti altri servizi la usano); il segreto condiviso può servire anche al controllo di integrità. Il primo problema è che l'intruso può **intercettare (eavesdropping) la password** dai messaggi, usando poi password e IP falsificato. La soluzione è **cifrare la password**, rendendola illeggibile all'intruso; con la cifratura simmetrica, peraltro, non serve una password aggiuntiva, poiché la chiave condivisa stessa può fare da password.

Attacco playback (replay)

Anche cifrando, un intruso “molto astuto” può intrufolarsi nella comunicazione spacciandosi per A con un **attacco playback**: sniffa i pacchetti da A a B per ottenere la **versione cifrata della password di A** e, se ci riesce, può “riprodurre” (ri-trasmettere) la password cifrata a B in una nuova sessione, usando anche senza comprenderla. A differenza degli attacchi MiM, che avvengono in tempo reale, sniffing e playback avvengono separatamente. Il problema è che B non può dire se A “era vivo”, cioè se esiste una sessione attiva tra il vero A e B. È una situazione simile all'instaurazione di connessione TCP, dove il three-way handshake assicura che entrambi gli endpoint siano vivi e TCP usa **numeri di sequenza iniziali casuali** per evitare che vecchie ritrasmissioni vengano interpretate come SYN/ACK; un'idea analoga vale per l'autenticazione.

Nonce

Si usa quindi un **nonce**: un numero casuale o pseudo-casuale che un protocollo usa **una sola volta nella sua vita**. Esempio con A e B che condividono una chiave simmetrica:

1. A invia a B il messaggio “Io sono A”.
2. B sceglie un **nonce R** e lo invia ad A.
3. A cifra il nonce insieme alla password usando la chiave simmetrica segreta A-B e rimanda il nonce cifrato a B.
4. B decifra il messaggio: se il nonce decifrato è uguale a quello inviato, A è autenticato.

In questo modo la password diventa di fatto una **one-time password**: poiché ogni password cifrata contiene quel numero di nonce, non può essere riutilizzata in altre sessioni da altri. Se B riceve correttamente password e nonce, si può ragionevolmente assumere che dietro c'è un A “vivo”.

SSL (Secure Socket Layer)

Le tecniche crittografiche viste si integrano per fornire riservatezza, integrità dei dati e autenticazione degli endpoint sulle connessioni TCP: sono implementate nel **Secure Socket Layer (SSL)**, una **versione potenziata di TCP** con servizi di sicurezza. Esiste una versione leggermente modificata di SSL v3 detta **Transport Layer Security (TLS)**, e l'equivalente per UDP è il **Datagram TLS (DTLS)**. SSL è usato da molti protocolli applicativi — web, e-mail, messaggistica istantanea, VoIP — e il browser spesso usa **HTTPS (HTTP + SSL/TLS)** invece di HTTP. Poiché SSL protegge TCP, può essere usato da qualsiasi applicazione che gira su TCP, fornendo un'API con socket simile a quella di TCP (l'applicazione include le classi e le librerie SSL). Tecnicamente risiede nel livello applicativo, ma allo sviluppatore sembra un protocollo di trasporto, con i servizi di TCP più quelli di sicurezza. Le sue **tre fasi principali** sono handshake, derivazione delle chiavi (*key derivation*) e trasferimento dati.

Fase 1: Handshake

Un client B vuole comunicare via SSL con un server A: deve stabilire una connessione TCP con A, verificare che A sia davvero A e inviare ad A un master secret (segreto condiviso). La procedura è la seguente:

1. Stabilita la connessione TCP, B invia un messaggio di **hello** ad A.
2. Il server A risponde con un **certificato** contenente la sua chiave pubblica (asimmetrica).
3. Poiché il certificato è certificato da una CA, B si fida che la chiave pubblica nel certificato sia di A.
4. B genera un **Master Secret (MS)**, valido solo per questa sessione SSL (funge da nonce), lo cifra con la chiave pubblica di A e lo invia.
5. A decifra con la chiave privata e ottiene l'MS: al termine, solo B e A conoscono il master secret della sessione.

Fase 2: Key Derivation

L'MS è un segreto condiviso tra B e A, quindi potrebbe essere usato come chiave simmetrica per tutta la cifratura e la verifica di integrità della sessione. Tuttavia è più sicuro usare chiavi diverse per cifratura e controllo di integrità nei due flussi ($A \rightarrow B$ e $B \rightarrow A$), quindi dall'MS si creano **quattro chiavi**:

- $E_{B \rightarrow A}$ = chiave di **cifratura** di sessione per i dati da B verso A.
- $M_{B \rightarrow A}$ = chiave di **MAC** di sessione per i dati da B verso A.
- $E_{A \rightarrow B}$ = chiave di **cifratura** di sessione per i dati da A verso B.
- $M_{A \rightarrow B}$ = chiave di **MAC** di sessione per i dati da A verso B.

Fase 3: Trasferimento dati

SSL protegge TCP spezzando il flusso di dati dell'applicazione in **record**. Per ogni record:

1. SSL appende un **MAC** per il controllo di integrità ($\text{record}_i + \text{MAC}$).
2. Il record modificato viene **cifrato**.
3. Il record cifrato è passato a TCP per la trasmissione.

Questo garantisce l'integrità dei singoli record, ma non dello stream intero: poiché si cifra solo il payload, un MiM tra A e B può ancora danneggiare lo stream invertendo o rimuovendo segmenti — ad esempio cattura due segmenti consecutivi, inverte il loro ordine insieme ai numeri di sequenza TCP e li invia così al ricevente, che se ne accorge solo quando i dati arrivano all'applicazione. La soluzione è integrare anche un **numero di**

sequenza nel messaggio prima della cifratura: il messaggio cifrato è $\text{record}_i + \text{MAC} + \text{sequenceNumber}_i$, quindi conoscendo la procedura e il MAC, il ricevente può certificare l'integrità del messaggio.

Firewall

Un **firewall** è una combinazione di hardware e software che isola (protegge) una rete da Internet consentendo o negando il passaggio dei pacchetti. Tutto il traffico in ingresso e in uscita deve passare dal firewall, e solo il traffico autorizzato — definito dalla policy di sicurezza locale — può passare. Il firewall è quindi il **singolo punto di accesso alla rete pubblica**, anche se le grandi organizzazioni possono avere più livelli o firewall distribuiti. Il firewall stesso deve essere **immune alle penetrazioni**: se compromesso, dà un falso senso di sicurezza. Gli approcci sono tre: filtraggio tradizionale dei pacchetti, filtraggio stateful e application gateway.

Filtraggio tradizionale dei pacchetti

Un **packet filter** è tipicamente implementato sul **gateway**, cioè il router che connette la rete locale all'ISP. Esamina ogni singolo datagram e decide se accettarlo o scartarlo in base a regole definite dall'amministratore, specifiche per datagram in ingresso/uscita o per singola interfaccia. Le regole si basano tipicamente su:

- Indirizzo IP sorgente o destinazione.
- Tipo di protocollo nel campo del datagram IP (TCP, UDP, ICMP, ecc.).
- Porta sorgente e destinazione.
- Flag TCP: SYN, ACK, ecc.

Esempi di policy: per consentire solo traffico web si bloccano tutti i segmenti TCP SYN con porta di destinazione diversa da 80; per negare alcuni servizi di streaming si blocca il traffico UDP non critico, spesso usato per lo streaming; per negare i ping in ingresso si bloccano le risposte ICMP ping in uscita. Le policy possono inoltre combinare indirizzi e numeri di porta, ad esempio inoltrando tutti i datagram Telnet (porta 23) tranne quelli verso o da un elenco di IP specifici; tali policy però non proteggono dallo spoofing.

Access Control List (ACL)

Le regole si implementano nei router tramite **access control list**. Esempio per far passare solo traffico web (interfaccia router-ISP):

Azione	Indirizzo sorgente	Indirizzo dest.	Protocollo	Porta src	Porta dest	Flag
allow	222.22/16	fuori da 222.22/16	TCP	> 1023	80	any
allow	fuori da 222.22/16	222.22/16	TCP	80	> 1023	ACK
allow	222.22/16	fuori da 222.22/16	UDP	> 1023	53	—
allow	fuori da 222.22/16	222.22/16	UDP	53	> 1023	—
deny	all	all	all	all	all	all

Con la regola 1 i pacchetti TCP con porta destinazione 80 possono uscire; con la regola 2 i pacchetti TCP con porta sorgente 80 e ACK=1 possono entrare, poiché tutti i messaggi web portano numeri ACK; con le regole 3–4 i pacchetti DNS (UDP porta 53) entrano ed escono.

Problema: filtraggio stateless

Il filtraggio tradizionale è **stateless**: la tabella, pur restrittiva, lascia passare qualsiasi pacchetto dall'esterno con ACK=1 e porta sorgente 80 — e tali pacchetti sono noti per essere usati in **attacchi DoS**. La soluzione ingenua sarebbe bloccare anche i pacchetti TCP ACK, ma ciò impedirebbe agli utenti interni di navigare sul Web.

Filtraggio stateful

I filtri **stateful** risolvono il problema tracciando **tutte le connessioni TCP attive** in una tabella delle connessioni, per capire se il traffico appartiene a una connessione legittima. Il firewall riconosce l'inizio di una nuova connessione (la sequenza SYN–SYNACK–ACK del three-way handshake) e la fine (il pacchetto FIN);

può anche assumere, conservativamente, che la connessione sia terminata quando non vede attività per un certo tempo (es. 60 secondi). La tabella stateful include tipicamente una colonna “**check connection**” che specifica se i pacchetti rientrano in una connessione già stabilita: solo i pacchetti ACK dentro connessioni stabilite vengono ammessi.

Azione	Ind. sorgente	Ind. dest.	Protocollo	Porta src	Porta dest	Flag	Check connessione
allow	222.22/16	fuori da 222.22/16	TCP	> 1023	80	any	
allow	fuori da 222.22/16	222.22/16	TCP	80	> 1023	ACK	X
allow	222.22/16	fuori da 222.22/16	UDP	> 1023	53	—	
allow	fuori da 222.22/16	222.22/16	UDP	53	> 1023	—	
deny	all	all	all	all	all	all	

Application Gateway

I metodi di filtraggio fin qui visti controllano soprattutto IP e porte, ma può servire consentire o negare funzionalità **in base all’utente o all’applicazione**: ad esempio, solo i tecnici potrebbero usare certi protocolli (es. Telnet verso l’esterno), negati agli utenti normali, oppure solo certi tipi di messaggi o comandi sono ammessi dentro un protocollo. Questo va oltre le capacità dei filtri tradizionali e stateful, poiché l’identità degli utenti interni è un dato del **livello applicativo**, non presente negli header IP/TCP/UDP. Un **application gateway** (o application-level gateway) filtra i pacchetti in base ai dati del livello applicativo ed è tipicamente un server separato che lavora insieme al firewall. Il suo funzionamento prevede che si neghi ogni connessione di un protocollo specifico (es. Telnet) tranne quelle da o verso l’application gateway; chi vuole usare il gateway — e quindi il protocollo riservato — deve autenticarsi su di esso (user ID e password); il server verifica se l’utente ha il permesso per quel protocollo e, in tal caso, tutte le richieste e risposte passano dal gateway (proxy). Le reti interne hanno spesso più gateway per applicazioni diverse (Telnet, HTTP, FTP, e-mail). Gli svantaggi sono tre:

- Serve un **gateway diverso per ogni applicazione**.
- C’è una **penalizzazione delle prestazioni** dovuta al proxying, soprattutto con molti utenti.
- Il software sugli host deve **sapere usare l’application gateway**.

Intrusion Detection Systems (IDS)

Per rilevare certi tipi di attacchi serve il **deep packet inspection**, cercando pattern di attacco noti o traffico sospetto. L’**Intrusion Detection System (IDS)** è un insieme di dispositivi specializzati che monitorano la rete cercando pacchetti sospetti; il suo ruolo primario è riconoscere i pacchetti potenzialmente malevoli e avvisare l’amministratore di rete, che poi deve agire. Se può anche bloccare (filtrare) i pacchetti potenzialmente malevoli, si parla di **Intrusion Prevention System (IPS)**. Le organizzazioni si affidano agli IDS per rilevare un’ampia gamma di attacchi: network/port scan (es. nmap), flooding DoS, worm e virus. L’architettura tipica prevede uno o più **sensori IDS** e un **processore IDS centrale** che raccoglie e integra le informazioni e invia gli allarmi. Esistono due tipi di IDS:

- **Signature-based**: mantiene un’ampia banca dati di **firme di attacco**, cioè insiemi di regole legate a un’attività di intrusione (il più comune).
- **Anomaly-based**: crea un **profilo del traffico** e segnala i flussi statisticamente anomali.

Demilitarized Zone (DMZ)

Nelle grandi reti esiste spesso il problema di avere due livelli di sicurezza diversi per dispositivi diversi: ad esempio, i server che devono comunicare con l’esterno possono usare un filtraggio più morbido. Una **demilitarized zone (DMZ)** è una regione a bassa sicurezza in cui le restrizioni sono limitate e si possono ospitare i server. L’approccio tipico è porre la DMZ tra due firewall: uno esterno, meno restrittivo, e uno

interno, più restrittivo. In una DMZ con IDS, la **regione ad alta sicurezza** è protetta da un packet filter e da un application gateway e monitorata dai sensori IDS, mentre la **regione a bassa sicurezza** (la DMZ) è protetta solo dal filtro e monitorata dai sensori.